

**KLASIFIKASI *MULTILAYER* JENIS TUMOR OTAK BERDASARKAN
CITRA *MAGNETIC RESONANCE IMAGING* (MRI) MENGGUNAKAN
MODEL EFFICIENTNETV2-S**

SKRIPSI

diajukan untuk menempuh ujian sarjana
pada Fakultas Matematika dan Ilmu Pengetahuan Alam
Universitas Padjadjaran

ROBERT WILLIAM HENDRA TJUTJU
NPM 140810220087



UNIVERSITAS PADJADJARAN
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
PROGRAM STUDI TEKNIK INFORMATIKA
SUMEDANG
2026

SKRIPSI

KLASIFIKASI *MULTILAYER* JENIS TUMOR OTAK BERDASARKAN CITRA *MAGNETIC RESONANCE IMAGING (MRI)* MENGGUNAKAN MODEL *EFFICIENTNETV2-S*

MULTILAYER CLASSIFICATION OF BRAIN TUMOR TYPES BASED ON MAGNETIC RESONANCE IMAGING (MRI) USING THE EFFICIENTNETV2-S MODEL

Telah dipersiapkan dan disusun oleh

ROBERT WILLIAM HENDRA TJUTJU
NPM 140810220087

Telah dipertahankan di depan Tim Penguji
pada tanggal 8 Juli 2026

Susunan Tim Penguji

- | | | |
|--|-------------------|-------|
| 1. <u>Dr. Akmal, S.Si., M.T.</u>
NIP. 19680717 199303 1 003 | Ketua Tim Penguji | |
| 2. <u>Dr. Afrida Helen, S.T., M.Kom.</u>
NIP. 19650128 199703 2 001 | Pembimbing | |
| 3. <u>Dr. Akmal, S.Si., M.T.</u>
NIP. 19680717 199303 1 003 | Co-Pembimbing | |
| 4. <u>Dr. Setiawan Hadi, M.Sc.CS.</u>
NIP. 19620701 199302 1 001 | Penguji | |
| 5. <u>Erick Paulus, S.Si, M.Kom.</u>
NIP. 19820318 200604 1 001 | Penguji | |
| 6. <u>Akik Hidayat, Drs., M.Kom.</u>
NIP. 19611018 198603 1 002 | Penguji | |

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, yang telah melimpahkan rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan penyusunan skripsi yang berjudul **“KLASIFIKASI *MULTILAYER* JENIS TUMOR OTAK BERDASARKAN CITRA *MAGNETIC RESONANCE IMAGING* (MRI) MENGGUNAKAN MODEL *EFFICIENTNETV2-S*”** sebagai salah satu syarat menempuh sarjana pada Program Studi S-1 Teknik Informatika Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Padjadjaran.

Dalam proses penyusunan dan penulisan skripsi ini tidak terlepas dari bantuan, bimbingan, serta dukungan dari berbagai pihak. Oleh karena itu dalam kesempatan ini penulis mengucapkan terima kasih kepada Ibu Dr. Afrida Helen, S.T., M.Kom., sebagai pembimbing utama, Bapak Dr. Akmal, S.Si., MT. , sebagai pembimbing pendamping yang telah meluangkan waktu dan pikirannya sehingga penulis dapat menyelesaikan skripsi ini. Ucapan terima kasih juga diberikan kepada keluarga penulis yang selalu memberikan motivasi dan doa yang menjadi pendorong dalam penyelesaian skripsi ini. Penulis juga mengucapkan terima kasih sebanyak-banyaknya kepada:

1. Prof. Dr. Desi Harneti Putri Huspa, S.Si., M.Si., selaku Dekan Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Padjadjaran.

2. Dr. Afrida Helen, S.T., M.Kom., selaku Kepala Departemen Ilmu Komputer Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Padjadjaran.
3. Dr. Akmal, S.Si., MT, selaku Ketua Program Studi S-1 Teknik Informatika Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Padjadjaran
4. Bapak Dr. Setiawan Hadi, M.Sc.CS., selaku Dosen Penguji 1, Bapak Erick Paulus, S.Si., M.Kom., selaku Dosen Penguji 2 dan Bapak Akik Hidayat, Drs., M.Kom., selaku Dosen Penguji 3.
5. Dosen-dosen Teknik Informatika Unpad yang telah mengajar dan memberikan ilmu kepada penulis selama masa perkuliahan yang membawa penulis pada posisi sekarang ini.
6. Keluarga penulis yang senantiasa memberikan doa, semangat, dan motivasi.

Akhir kata, penulis berharap semoga skripsi ini dapat bermanfaat bagi pembaca.

Jatinangor, 8 Juli 2026

Penulis

ABSTRAK

Tumor otak merupakan kondisi kritis yang membutuhkan diagnosis cepat dan presisi melalui citra *Magnetic Resonance Imaging* (MRI). Analisis visual secara manual sering kali memakan waktu dan rentan terhadap kesalahan subjektif. Selain itu, sistem klasifikasi otomatis mungkin mengalami kegagalan saat menerima data masukan yang tidak relevan. Oleh karena itu, penelitian ini bertujuan untuk mengimplementasikan sistem klasifikasi *multilayer* yang terdiri dari lapisan validasi input dan lapisan klasifikasi utama menggunakan arsitektur EfficientNetV2-S untuk mengkategorikan citra ke dalam empat kelas: *glioma*, *meningioma*, *pituitary*, dan *no tumor*.

Penelitian ini menerapkan metode eksperimental dengan menggabungkan kompilasi dataset publik dari Figshare, Br35H, Sartaj, dan *Brain Tumors Dataset*, yang menghasilkan total 29.500 citra. Tahapan komputasi meliputi preprocessing dan augmentasi citra untuk meningkatkan variabilitas data latih. Model dilatih menggunakan pendekatan Transfer Learning dengan optimizer AdamW. Evaluasi kinerja dan ketangguhan sistem diukur secara ketat menggunakan metode 10-Fold Cross Validation.

Hasil pengujian menunjukkan performa klasifikasi yang sangat optimal dengan nilai rata-rata akurasi, *Presisi*, *Recall*, dan *F1-Score* yang konsisten mencapai 99,78%. Kesimpulannya, pendekatan *multilayer* dengan EfficientNetV2-S terbukti sangat tangguh dalam menyaring input yang tidak valid dan sangat akurat dalam mengenali jenis tumor. Sistem ini berpotensi untuk diimplementasikan sebagai alat bantu *Computer-Aided Diagnosis* (CAD) yang andal bagi tenaga kesehatan.

Kata Kunci: *10-Fold Cross Validation*, *Deep Learning*, EfficientNetV2-S, MRI, Tumor Otak

ABSTRACT

A brain tumor is a critical condition that requires rapid and precise diagnosis through Magnetic Resonance Imaging (MRI). Manual visual analysis is often time-consuming and prone to subjective errors. Furthermore, automated detection systems may fail when receiving irrelevant input data. Therefore, this study aims to implement a multilayer classification system consisting of an input validation layer and a main classification layer using the EfficientNetV2-S architecture to categorize MRI scans into four classes: Glioma, Meningioma, Pituitary, and No tumor.

This research applies an experimental method by combining public datasets from Figshare, Br35H, Sartaj, and the Brain Tumors Dataset, resulting in a total of 29,500 images. The computational stages include image preprocessing and augmentation to increase the variability of the training data. The model was trained using the Transfer Learning approach with the AdamW optimizer. System performance and robustness were rigorously evaluated using the 10-Fold Cross-Validation method.

The experimental results demonstrated highly optimal classification performance, achieving a consistent weighted average of 99.78% across Accuracy, Precision, Recall, and F1-Score. In conclusion, the multilayer approach with EfficientNetV2-S is proven to be highly robust in filtering invalid inputs and highly accurate in recognizing tumor types. This system has a potential to be implemented as a reliable Computer-Aided Diagnosis (CAD) tool for healthcare professionals.

Keywords: 10-Fold Cross-Validation, Brain Tumor, Deep Learning, EfficientNetV2-S, MRI

DAFTAR ISI

KATA PENGANTAR	iii
ABSTRAK	v
<i>ABSTRACT</i>	vi
DAFTAR ISI	vii
DAFTAR TABEL	x
DAFTAR GAMBAR	xi
DAFTAR LAMPIRAN	xii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Identifikasi Masalah.....	3
1.3 Batasan Masalah	4
1.4 Maksud dan Tujuan Penelitian	5
1.5 Manfaat Penelitian	6
1.6 Metodologi Penelitian	6
1.7 Sistematika Penulisan	8
BAB II TINJAUAN PUSTAKA	11
2.1 Tumor Otak	11
2.2 <i>Magnetic Resonance Imaging</i> (MRI)	12
2.3 <i>Deep Learning</i> dan <i>Convolutional Neural Network</i> (CNN)	13
2.4 <i>Transfer Learning</i>	14
2.5 Arsitektur EfficientNet.....	15
2.5.1 EfficientNet.....	15
2.5.2 EfficientNetV2	17
2.6 <i>Optimizer</i>	20
2.6.1 <i>Adaptive Moment Estimation</i> (Adam)	21
2.6.2 <i>Adam with Decoupled Weight Decay</i> (AdamW)	22
2.7 Penjadwalan Laju Pembelajaran <i>ReduceOnLROnPlateu</i>	23
2.8 Fungsi Kerugian <i>CrossEntropyLoss</i>	24

2.9	Perangkat Lunak yang Digunakan.....	25
2.9.1	Python.....	26
2.9.2	PyTorch.....	26
2.9.3	Streamlit.....	27
2.10	Penelitian Terkait.....	28
2.11	Evaluasi dan Pengujian Sistem.....	29
2.11.1	<i>K-Fold Cross Validation</i>	29
2.11.2	Metrik Evaluasi	30
2.11.3	<i>Black Box Testing</i>	31
BAB III	ANALISIS DAN PERANCANGAN	33
3.1	Alur Penelitian.....	33
3.2	Analisis Kebutuhan.....	36
3.2.1	Kebutuhan Data.....	36
3.2.2	Kebutuhan Perangkat Keras.....	43
3.2.3	Kebutuhan Perangkat Lunak.....	44
3.3	Perancangan Skenario Eksperimen.....	45
3.3.1	Pengujian Model EfficientNetV2-S pada Dataset <i>Baseline</i>	45
3.3.2	Pengujian Model EfficientNetV2-S pada Dataset Campuran.....	46
3.3.3	Pengujian Model EfficientNetV2-S pada Dataset COCO untuk Validasi Input Citra.....	47
3.4	Perancangan Model Deep Learning.....	48
3.4.1	Arsitektur Model (EfficientNetV2-S).....	48
3.4.2	Konfigurasi <i>Hyperparameter</i> Pelatihan.....	49
3.5	Perancangan Prototipe Aplikasi	51
3.5.1	Rancangan Antarmuka Aplikasi	51
3.5.2	<i>Use Case Diagram</i> Aplikasi	53
3.6	Pengujian Fungsionalitas Aplikasi (<i>Black Box Testing</i>).....	54
BAB IV	HASIL DAN PEMBAHASAN	57
4.1	Lingkungan dan Implementasi Model	57
4.1.1	Pra-pemrosesan Data (<i>Preprocessing</i> & Augmentasi).....	58
4.1.2	Arsitektur dan Konfigurasi Model	59

4.1.3	Skema Pengujian (<i>Cross Validation</i>)	61
4.2	Implementasi Pengujian pada Dataset Baseline	62
4.2.1	Pengujian pada Dataset <i>Brain Tumor Classification</i> (MRI).....	63
4.2.2	Pengujian pada Dataset <i>Crystal Clean</i>	68
4.2.3	Pengujian pada 3 Dataset Penelitian Wong	73
4.3	Fase Produksi Implementasi Sistem	79
4.3.1	Implementasi Model pada Dataset Campuran (4 Dataset)	80
4.3.2	Implementasi Model pada Validasi Biner Citra Otak	87
4.3.3	Implementasi <i>Web</i>	90
4.4	Evaluasi <i>Black Box Testing</i>	95
BAB V KESIMPULAN DAN SARAN.....		102
5.1	Kesimpulan.....	102
5.2	Saran	104
DAFTAR PUSTAKA		106
LAMPIRAN		109
RIWAYAT HIDUP		110

DAFTAR TABEL

Tabel 2.1 Penelitian Terkait	28
Tabel 3.1 Skenario Black Box Testing	55
Tabel 4.1 Hasil dan Metode Penelitian Terdahulu	63
Tabel 4.2 Hasil <i>Fold</i> pada Dataset Brain Tumor Classification (MRI)	64
Tabel 4.3 Detail <i>Epoch</i> pada <i>Fold 1</i>	66
Tabel 4.4 Hasil <i>Fold</i> pada Dataset Crystal Clean.....	69
Tabel 4.5 Detail <i>Epoch</i> pada <i>Fold 7</i>	71
Tabel 4.6 Detail Sumber Datasaset	74
Tabel 4.7 Hasil 10 <i>Fold</i> pada Dataset Wong.....	75
Tabel 4.8 Detail <i>Epoch</i> pada <i>Fold 2</i>	77
Tabel 4.9 Distribusi Kelas dan Dataset	80
Tabel 4.10 Hasil 10 <i>Fold</i> pada Dataset Campuran	80
Tabel 4.11 Sampel Kesalahan Prediksi Model.....	84
Tabel 4.12 Pembagian Data pada Validasi Biner Citra Otak	88
Tabel 4.13 Hasil <i>Black Box Testing</i>	96
Tabel 4.14 Uji Validasi Input	98

DAFTAR GAMBAR

Gambar 2.1 Arsitektur CNN	13
Gambar 2.2 Ilustrasi Perbandingan Penskalaan Model	16
Gambar 2.3 Arsitektur EfficientNetV2-S	19
Gambar 3.1 Diagram Alur Penelitian	34
Gambar 3.2 Contoh Citra Kelas Glioma pada Dataset Figshare	37
Gambar 3.3 Contoh Citra Kelas Meningioma pada Dataset Figshare	38
Gambar 3.4 Contoh Citra Kelas Pituitary pada Dataset Figshare	38
Gambar 3.5 Contoh Citra Glioma pada Dataset Crystal Clean	39
Gambar 3.6 Contoh Citra Meningioma pada Dataset Crystal Clean	39
Gambar 3.7 Contoh Citra Pituitary pada Dataset Crystal Clean	40
Gambar 3.8 Contoh Citra <i>No tumor</i> pada Dataset Crystal Clean	40
Gambar 3.9 Contoh Citra <i>No tumor</i> pada Dataset BR35H	41
Gambar 3.10 Contoh Citra Glioma pada Dataset Brain Tumor Classification	42
Gambar 3.11 Citra Meningioma pada Dataset Brain Tumor Classification	42
Gambar 3.12 Contoh Citra Pituitary pada Dataset Brain Tumor Classification ...	42
Gambar 3.13 Contoh Citra <i>No Tumor</i> pada Dataset Brain Tumor Classification ..	42
Gambar 3.14 Contoh Citra pada COCO Dataset	43
Gambar 4.1 Grafik <i>Loss</i> dan <i>Akurasi</i> pada <i>Fold 1</i>	65
Gambar 4.2 Grafik <i>Loss</i> dan <i>Akurasi</i> pada <i>Fold 7</i>	70
Gambar 4.3 Grafik <i>Loss</i> dan <i>Akurasi</i> pada <i>Fold 2</i>	76
Gambar 4.4 <i>Confusion Matrix</i> pada Dataset Campuran	82
Gambar 4.5 Grafik <i>Loss</i> dan <i>Akurasi</i> pada <i>Fold 8</i>	87
Gambar 4.6 <i>Confusion Matrix</i> Validasi Biner Citra Otak	89
Gambar 4.7 Klasifikasi Gambar Jamak	94
Gambar 4.8 Klasifikasi Gambar Tunggal	95

DAFTAR LAMPIRAN

Lampiran 1 Link Github Berisi <i>Source Code</i>	109
--	-----

BAB I

PENDAHULUAN

Pada bab ini dijelaskan tentang latar belakang dari topik penulisan skripsi, pokok permasalahan berupa identifikasi dan batasan masalah, maksud dan tujuan penelitian, serta metodologi yang digunakan serta sistematika penulisan.

1.1 Latar Belakang

Tumor otak adalah salah satu penyakit paling fatal akibat pertumbuhan sel abnormal yang tidak terkendali di dalam otak. Pertumbuhan ini dapat menyebabkan pembengkakan, mengganggu fungsi sistem saraf, dan menimbulkan gejala serius hingga kematian, tergantung pada jenis dan ukurannya. Tiga jenis tumor otak primer yang paling umum adalah glioma, meningioma, dan pituitary. Mengingat dampaknya yang signifikan, deteksi dini dan klasifikasi jenis tumor yang akurat menjadi langkah krusial untuk menentukan rencana pengobatan yang efektif dan meningkatkan harapan hidup pasien.

Untuk mendiagnosis tumor otak, teknologi pencitraan medis seperti *Magnetic Resonance Imaging* (MRI) adalah metode yang paling umum digunakan (Badža & Barjaktarović, 2020). Meskipun begitu, interpretasi citra MRI secara manual oleh ahli radiologi merupakan proses yang kompleks, memakan waktu, dan rentan terhadap kesalahan subjektif yang sangat bergantung pada keahlian dan pengalaman individu (Azzahra et al., 2023). Keterbatasan ini mendorong pengembangan metode diagnosis yang lebih objektif, otomatis, dan andal.

Perkembangan pesat dalam bidang kecerdasan artifisial, khususnya *Deep Learning* dengan arsitektur *Convolutional Neural Network* (CNN), menawarkan solusi yang menjanjikan. Namun, salah satu tantangan dalam implementasi sistem deteksi medis otomatis adalah menjaga integritas data masukan. Sistem sering kali rentan menghasilkan prediksi yang keliru atau error apabila pengguna secara tidak sengaja memasukkan citra yang tidak relevan. Untuk mengatasi celah tersebut, diperlukan sebuah pendekatan klasifikasi *multilayer*. Mekanisme *multilayer* ini dirancang agar sistem memiliki tahapan validasi input di lapisan awal untuk menyaring dan memastikan bahwa citra yang diunggah benar-benar merupakan citra MRI otak. Setelah citra tervalidasi, lapisan klasifikasi utama baru akan bekerja untuk mendeteksi dan membedakan jenis tumor.

Melatih arsitektur yang tangguh dari awal membutuhkan komputasi dan dataset berjumlah masif, yang sering kali menjadi kendala dalam bidang medis. Untuk mengatasi masalah ini, teknik *transfer learning* menjadi pendekatan yang sangat efektif karena memanfaatkan model yang telah dilatih pada dataset gambar berskala besar dan mengadaptasinya untuk tugas klasifikasi tumor otak yang spesifik.

Salah satu arsitektur yang terbukti memiliki performa tinggi adalah EfficientNet. Arsitektur ini dikenal karena efisiensinya yang tinggi dengan secara seimbang meningkatkan kedalaman, lebar, dan resolusi lapisan jaringan untuk mencapai akurasi maksimal dengan sumber daya komputasi yang lebih sedikit (Karno A S B et al., 2023). Generasi terbarunya, EfficientNetV2, dirancang untuk memiliki kecepatan training yang lebih cepat dan efisiensi parameter yang lebih

baik dibandingkan versi sebelumnya (Waskita et al., 2023). Beberapa penelitian telah menunjukkan bahwa varian EfficientNet berhasil mencapai akurasi tinggi untuk klasifikasi tumor otak, menjadikannya kandidat kuat untuk diaplikasikan pada lapisan utama penelitian ini.

Meskipun demikian, performa sebuah model sangat dipengaruhi oleh kualitas dan keragaman data latih. Identifikasi tumor otak multi-kelas masih menjadi tantangan yang lebih sulit dibandingkan klasifikasi biner, dan solusi yang ada sering kali menderita akurasi yang rendah (Iqbal et al., 2024). Untuk menguji ketangguhan dan kemampuan generalisasi model dalam skenario yang lebih mendekati dunia nyata, penting untuk mengevaluasi performanya tidak hanya pada satu dataset, tetapi juga pada dataset campuran yang bervariasi.

Berdasarkan uraian tersebut, penelitian ini bertujuan untuk mengimplementasikan sistem klasifikasi *multilayer* menggunakan model EfficientNetV2-S berbasis *transfer learning*. Lapisan awal sistem ditujukan untuk validasi input citra, sementara lapisan klasifikasi utama bertugas mengkategorikan citra MRI ke dalam empat kelas: glioma, meningioma, pituitary, dan *no tumor*. Penelitian ini juga akan mengukur efektivitas model, membandingkan performanya dengan arsitektur lain yang relevan, serta menganalisis pengaruh penggunaan dataset campuran terhadap akurasi dan ketangguhan sistem secara keseluruhan.

1.2 Identifikasi Masalah

Berdasarkan latar belakang yang telah dijelaskan sebelumnya, masalah yang akan dicari solusinya dalam penelitian ini adalah sebagai berikut:

1. Bagaimana cara mengimplementasikan model transfer learning EfficientNetV2-S untuk melakukan klasifikasi citra MRI tumor otak?
2. Bagaimana cara membangun mekanisme *filter* (validasi input) agar sistem hanya memproses citra otak dan menolak citra bukan otak?
3. Apakah model EfficientNetV2-S efektif dalam mengklasifikasikan jenis tumor otak (glioma, meningioma, pituitary, dan *no tumor*) dari citra MRI?
4. Bagaimana perbandingan performa akurasi model EfficientNetV2-S dengan model-model lain yang telah diimplementasikan dalam penelitian sebelumnya pada dataset yang identik?
5. Bagaimana pengaruh penggunaan dataset campuran (gabungan dari beberapa sumber dataset) terhadap performa akurasi, generalisasi, dan robustitas model EfficientNetV2-S?

1.3 Batasan Masalah

Dari identifikasi masalah tersebut, maka dalam penelitian ini terdapat beberapa batasan masalah pada sebagai berikut:

1. Penelitian ini berfokus pada implementasi model *transfer learning* untuk klasifikasi citra, dan tidak akan melibatkan pelatihan model *Convolutional Neural Network* (CNN) dari nol.
2. Model yang akan digunakan terbatas pada arsitektur EfficientNetV2-S.
3. Objek penelitian adalah klasifikasi citra MRI tumor otak ke dalam empat kategori: glioma, meningioma, pituitary, dan *no tumor*. Penelitian ini tidak mencakup segmentasi atau deteksi lokasi tumor dalam citra.

4. Sistem dilengkapi dengan model klasifikasi biner tambahan sebagai filter awal untuk membedakan citra MRI otak dan citra bukan MRI otak sebelum masuk ke tahap klasifikasi jenis tumor.
5. Data yang digunakan adalah dataset citra MRI yang sudah tersedia secara publik.
6. *Epoch* yang dilakukan selama pelatihan akan dibatasi pada *max 20 epoch* untuk klasifikasi tumor dan 5 *epoch* untuk validasi karena keterbatasan sumber daya *hardware*.
7. Analisis kinerja model akan dibatasi pada metrik kuantitatif seperti akurasi, *presisi*, *recall*, dan *F1-Score*.
8. Penelitian ini mencakup pembuatan prototipe aplikasi *web* sederhana menggunakan Streamlit untuk mendemonstrasikan hasil klasifikasi model.

1.4 Maksud dan Tujuan Penelitian

Maksud dari penelitian ini adalah mengimplementasikan model *transfer learning* EfficientNetV2-S untuk melakukan deteksi dan klasifikasi jenis tumor otak dari citra MRI.

Tujuan yang ingin dicapai oleh penulis dari penelitian ini adalah:

1. Mengimplementasikan model *transfer learning* EfficientNetV2-S untuk klasifikasi tumor otak berbasis citra MRI.
2. Mengimplementasikan model *transfer learning* EfficientNetV2-S untuk memvalidasi citra masukan agar sistem menolak gambar yang bukan merupakan citra otak.
3. Menganalisis performa model EfficientNetV2-S dalam klasifikasi jenis tumor otak berdasarkan metrik akurasi, *presisi*, *recall*, dan *F1-Score*.

4. Membangun prototipe aplikasi *web* interaktif menggunakan Streamlit untuk mendemonstrasikan hasil klasifikasi model.

1.5 Manfaat Penelitian

Manfaat yang diharapkan dari penelitian ini adalah:

1. Memberikan kontribusi dalam pengembangan ilmu pengetahuan di bidang *Deep Learning* dan *Computer Vision*.
2. Menjadi referensi ilmiah bagi peneliti lain yang ingin mengembangkan sistem klasifikasi citra medis berbasis *transfer learning* untuk diagnosis penyakit.
3. Menghasilkan model klasifikasi yang dapat membantu ahli medis dalam mengidentifikasi jenis tumor otak secara objektif dan lebih cepat. Hal ini dapat mengurangi proses manual yang memakan waktu.
4. Menyediakan prototipe aplikasi *web* interaktif yang dapat digunakan sebagai alat bantu demonstrasi, sehingga hasil penelitian dapat divisualisasikan dengan lebih mudah dan intuitif.

1.6 Metodologi Penelitian

Penelitian ini akan menggunakan metode eksperimental, dengan tahapan yang disesuaikan untuk menghasilkan sebuah model *deep learning* yang fungsional dan prototipe aplikasi *web*. Tahapan penelitian yang akan dilalui adalah sebagai berikut:

1. Studi Literatur

Pada tahap awal, akan dilakukan studi literatur untuk mengumpulkan referensi terkait konsep dasar *Deep Learning* dan *Convolutional Neural Network* (CNN).

Selain itu, studi ini juga akan berfokus pada pemahaman metode *transfer learning* dan arsitektur model EfficientNetV2, serta meninjau penelitian-penelitian terdahulu mengenai klasifikasi tumor otak menggunakan citra MRI.

2. Pengumpulan dan Pra-pemrosesan Data

Dataset yang akan digunakan adalah citra MRI tumor otak yang diperoleh dari sumber public yaitu Kaggle dan Figshare. Data untuk klasifikasi tumor otak mencakup empat kategori: tumor glioma, meningioma, pituitary, dan *no tumor*. Sedangkan data untuk klasifikasi citra otak atau bukan menggunakan COCO dataset yang didapat dari Kaggle. Setelah data terkumpul, akan dilakukan pra-pemrosesan yang meliputi:

- a. *Pembagian Data*: Dataset yang digunakan sudah terbagi menjadi set pelatihan (*training*) dan *validation*.
- b. *Normalisasi dan Resizing*: Seluruh citra akan dinormalisasi dan diubah ukurannya agar sesuai dengan spesifikasi input model EfficientNetV2-S (misalnya, menjadi 224x224 piksel).
- c. *Data Augmentation*: Untuk memperkaya variasi data dan mencegah *overfitting*, akan diterapkan teknik *data augmentation* seperti rotasi dan *horizontal flip*.

3. Perancangan dan Implementasi Model

Pada tahap ini, akan diimplementasikan model *transfer learning* EfficientNetV2-S. Model *pretrained* akan dimuat, dan akan ditambahkan lapisan- lapisan baru (*head*) di atasnya untuk menyesuaikan dengan tugas

klasifikasi tumor otak. Model ini akan di *fine-tune* dengan melatih sebagian atau seluruh lapisannya untuk mendapatkan performa yang optimal.

4. Pelatihan dan Evaluasi Model

Model yang telah dirancang akan dilatih menggunakan dataset yang sudah diproses. Proses pelatihan akan diatur dengan parameter tertentu (seperti *epoch*, *optimizer*, dan *learning rate*). Setelah pelatihan selesai, model akan dievaluasi menggunakan metrik performa seperti akurasi, presisi, *recall*, dan *F1-Score*.

5. Pembuatan Prototipe Sistem

Model terbaik yang dihasilkan dari tahap evaluasi akan disimpan. Kemudian, akan dibuat sebuah prototipe aplikasi *web* sederhana menggunakan *library* Streamlit untuk mendemonstrasikan kemampuan klasifikasi model secara interaktif.

6. Pelaporan

Seluruh proses penelitian, mulai dari studi literatur hingga hasil evaluasi model dan prototipe, akan didokumentasikan secara sistematis ke dalam laporan akhir skripsi.

1.7 Sistematika Penulisan

Untuk memberi gambaran yang jelas tentang penelitian ini, maka disusunlah sistematika penulisan yang berisi materi yang akan dibahas pada setiap bab. Sistematika dalam penulisan tugas akhir ini adalah sebagai berikut:

BAB I PENDAHULUAN

Pada bab ini dijelaskan tentang latar belakang dari topik penulisan skripsi, pokok

permasalahan berupa identifikasi dan batasan masalah, maksud dan tujuan penelitian, serta metodologi yang digunakan serta sistematika penulisan.

BAB II TINJAUAN PUSTAKA

Pada bab ini akan dibahas mengenai landasan-landasan teori yang menjadi dasar dalam penelitian. Pembahasan mencakup penjelasan mengenai tumor otak dan pencitraan medis *Magnetic Resonance Imaging* (MRI), *Deep Learning*, *Convolutional Neural Network* (CNN), *transfer learning*, arsitektur model, *optimizer*, penjadwalan laju pembelajaran, fungsi kerugian, perangkat lunak yang digunakan, penelitian-penelitian terdahulu, serta evaluasi pengujian sistem.

BAB III ANALISIS DAN PERANCANGAN

Pada bab ini dijelaskan mengenai tahapan penelitian dan metode yang digunakan dalam membangun model klasifikasi. Pembahasan meliputi alur penelitian, analisis kebutuhan (kebutuhan data, perangkat keras, dan perangkat lunak), perancangan scenario eksperimen, perancangan model *deep learning*, perancangan prototipe aplikasi, dan pengujian fungsionalitas aplikasi.

BAB IV HASIL DAN PEMBAHASAN

Pada bab ini dipaparkan hasil eksperimen yang telah dilakukan, meliputi lingkungan dan implementasi model, implementasi pengujian pada dataset *baseline*, fase produksi dan implementasi sistem, dan evaluasi *black box testing*.

BAB V KESIMPULAN DAN SARAN

Bab ini merupakan penutup yang berisi kesimpulan dan saran dari penelitian yang sudah dilakukan.

BAB II

TINJAUAN PUSTAKA

Pada bab ini akan dibahas mengenai landasan-landasan teori yang menjadi dasar dalam penelitian. Pembahasan mencakup penjelasan mengenai tumor otak dan pencitraan medis *Magnetic Resonance Imaging* (MRI), *Deep Learning*, *Convolutional Neural Network* (CNN), *transfer learning*, arsitektur model, *optimizer*, penjadwalan laju pembelajaran, fungsi kerugian, perangkat lunak yang digunakan, penelitian-penelitian terdahulu, serta evaluasi pengujian sistem.

2.1 Tumor Otak

Tumor otak adalah salah satu penyakit paling fatal yang disebabkan oleh pertumbuhan sel abnormal yang tidak terkendali di dalam otak (Badža & Barjaktarović, 2020). Pertumbuhan jaringan abnormal ini tidak memiliki fungsi fisiologis dan kehadirannya dapat menyebabkan pembengkakan otak, menghalangi aliran cairan serebrospinal, dan mengganggu sistem neurologis. Terdapat dua jenis utama tumor otak: jinak (*benign*) dan ganas (*malignant*). Tumor jinak umumnya tumbuh lambat dan jarang menyebar, sementara tumor ganas tumbuh cepat, bersifat kanker, dan dapat menyebar ke jaringan di sekitarnya.

Tiga jenis tumor otak primer yang paling umum adalah glioma, meningioma, dan pituitary. Meningioma dan tumor pituitary umumnya bersifat jinak, sedangkan glioma sebagian besar bersifat ganas. Mengingat dampaknya yang signifikan, deteksi dini dan klasifikasi jenis tumor yang akurat menjadi langkah krusial untuk

menentukan rencana pengobatan yang efektif dan meningkatkan harapan hidup pasien.

2.2 *Magnetic Resonance Imaging (MRI)*

Magnetic Resonance Imaging (MRI) adalah salah satu metode pencitraan medis yang telah menjadi standar utama dalam diagnosis dan evaluasi klinis tumor otak. Berbeda dengan beberapa metode pemindaian lain, MRI lebih disukai karena sama sekali tidak menggunakan radiasi pengion yang berbahaya bagi pasien. Teknologi ini memanfaatkan medan magnet dan gelombang radio untuk menghasilkan citra penampang lintang anatomi otak dengan resolusi spasial yang sangat tinggi. Keunggulan utama MRI terletak pada kemampuannya memberikan kontras jaringan lunak yang sangat detail, sehingga memungkinkan tenaga medis untuk membedakan secara presisi antara jaringan otak yang sehat, area pembengkakan, dan massa tumor itu sendiri. Detail anatomi yang tajam ini sangat krusial dalam menentukan letak, ukuran, dan karakteristik penyebaran tumor sebelum tindakan medis atau operasi dilakukan.

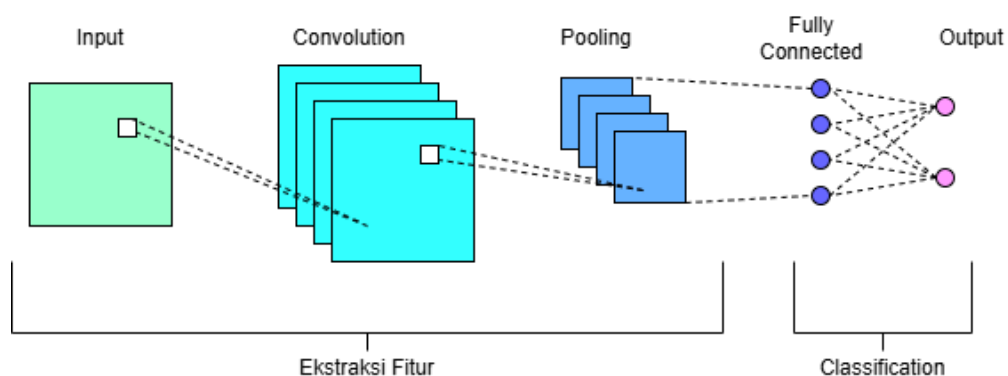
Meskipun menyajikan informasi visual yang komprehensif, interpretasi citra MRI secara manual oleh ahli radiologi tetap merupakan proses yang menantang dan memakan banyak waktu. Hal ini sebagian besar disebabkan oleh volume data yang masif, di mana pemindaian satu pasien saja dapat menghasilkan puluhan hingga ratusan irisan (*slice*) gambar yang harus dianalisis satu per satu. Selain itu, proses diagnosis manual ini rentan terhadap kesalahan subjektif dan kelelahan visual, mengingat akurasinya sangat bergantung pada tingkat keahlian dan jam terbang individu (Azzahra et al., 2023). Kompleksitas bentuk anatomi tumor yang tidak

beraturan serta kemiripan batas visual antara jaringan abnormal dan sehat seringkali semakin menyulitkan proses identifikasi.

2.3 *Deep Learning* dan *Convolutional Neural Network (CNN)*

Seiring berkembangnya teknologi, Kecerdasan Buatan dianggap sebagai solusi yang tepat untuk mengatasi keterbatasan metode konvensional. Salah satu cabangnya, *Deep Learning*, telah menunjukkan dampak signifikan pada pemrosesan citra medis. *Convolutional Neural Network (CNN)* adalah salah satu jenis arsitektur deep learning yang umum digunakan untuk memproses data gambar (Azzahra et al., 2023).

CNN dirancang khusus untuk mendeteksi dan mengenali objek dalam sebuah citra dengan cara menangkap pola-pola spesifik melalui serangkaian lapisan konvolusi, *pooling*, dan *fully connected*. Keunggulan utama CNN adalah kemampuannya untuk melakukan ekstraksi fitur secara otomatis, sehingga tidak memerlukan segmentasi manual yang dapat menurunkan efisiensi dan akurasi. Berikut adalah Gambar 2.1 yang menunjukkan arsitektur CNN.



Gambar 2.1 Arsitektur CNN

Arsitektur CNN terdiri dari rangkaian lapisan yang mentransformasi volume input menjadi volume output berupa skor kelas. Alur kerja arsitektur tersebut dapat dijelaskan sebagai berikut:

1. Lapisan Input: Lapisan awal yang memuat data citra mentah yang akan diproses.
2. Ekstraksi Fitur: Bagian ini berfungsi sebagai mata jaringan yang belajar melihat pola.
 - a. Lapisan Konvolusi: Lapisan ini melakukan operasi konvolusi matematis antara citra input dan kernel untuk menghasilkan peta fitur. Semakin dalam lapisannya, semakin kompleks fitur yang dapat dikenali.
 - b. *Pooling Layer*: Lapisan yang berfungsi untuk mengurangi resolusi spasial data (seperti *Max Pooling* atau *Average Pooling*). Ini membuat model lebih invarian terhadap pergeseran kecil dan distorsi pada gambar.
3. Klasifikasi: Bagian ini berfungsi sebagai otak yang membuat keputusan berdasarkan fitur yang ditemukan.
 - a. *Fully Connected Layer*: Lapisan di mana setiap neuron terhubung dengan semua neuron di lapisan sebelumnya. Lapisan ini menggunakan fitur hasil ekstraksi untuk mengklasifikasikan citra ke dalam label yang sesuai.
 - b. *Output Layer*: Lapisan terakhir yang menggunakan fungsi aktivasi (seperti *Softmax* atau *Sigmoid*) untuk menghasilkan probabilitas akhir dari prediksi model.

2.4 *Transfer Learning*

Salah satu tantangan dalam melatih model CNN dari awal adalah kebutuhan akan dataset dalam jumlah yang sangat besar, yang seringkali menjadi kendala

dalam bidang medis. Untuk mengatasi masalah ini, teknik *transfer learning* menjadi pendekatan yang sangat efektif.

Transfer learning memanfaatkan model yang telah dilatih sebelumnya pada dataset gambar berskala besar seperti ImageNet dan mengadaptasinya untuk tugas spesifik, seperti klasifikasi tumor otak (Azzahra et al., 2023). Dengan menggunakan bobot dan bias dari model yang sudah terlatih, proses pembelajaran menjadi lebih efektif dan dapat mencapai akurasi tinggi bahkan dengan dataset yang relatif terbatas.

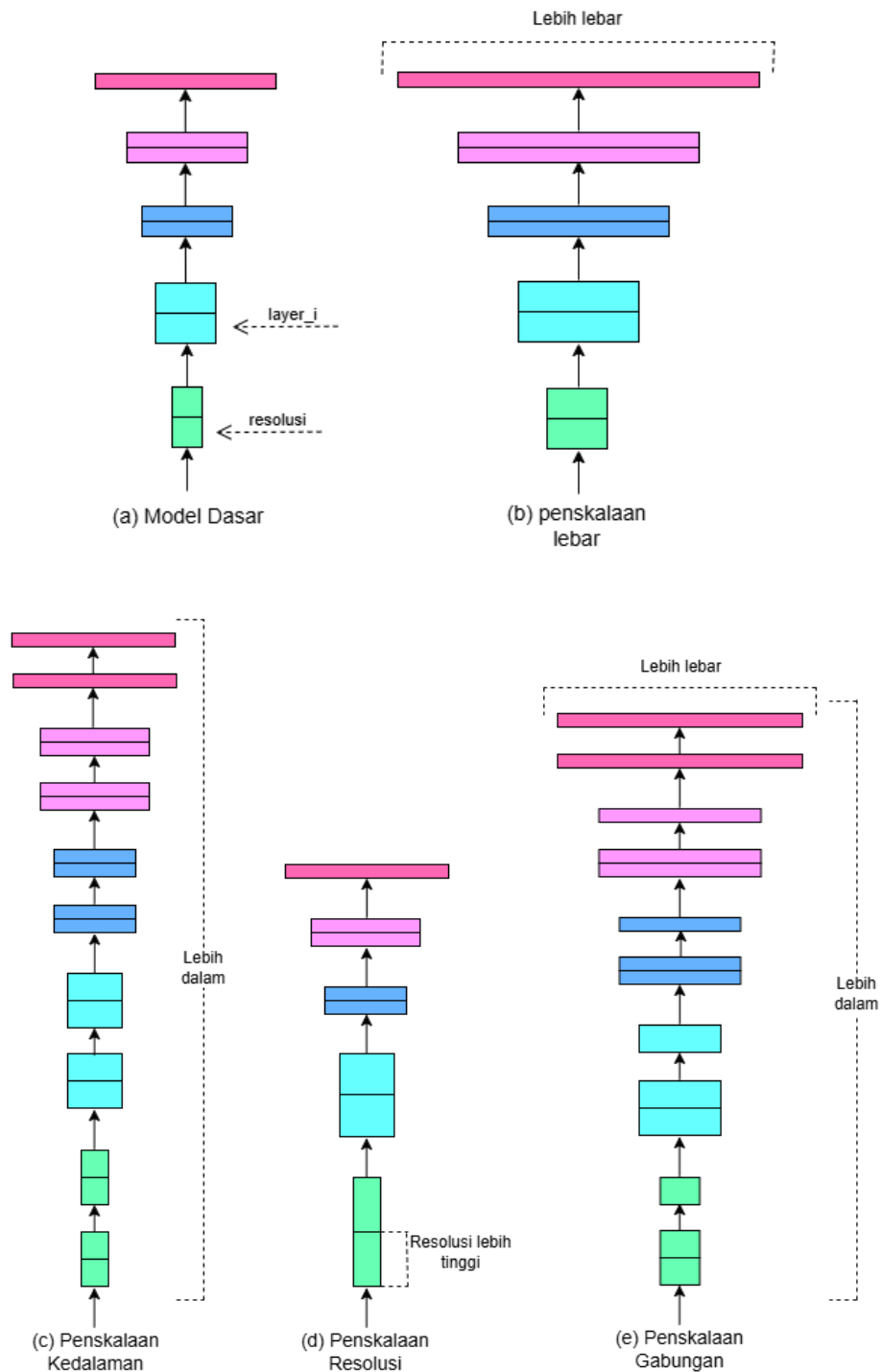
2.5 Arsitektur EfficientNet

EfficientNet adalah keluarga model *Convolutional Neural Network* (CNN) yang diperkenalkan oleh (Tan & Le, 2019) dengan tujuan mencapai akurasi tinggi sekaligus efisiensi komputasi yang lebih baik dibandingkan arsitektur CNN konvensional. Inovasi utama dari EfficientNet terletak pada pendekatan penskalaan model yang sistematis.

2.5.1 EfficientNet

Secara tradisional, untuk meningkatkan akurasi model CNN, peneliti biasanya melakukan penskalaan pada satu dimensi saja, seperti menambah kedalaman, memperlebar channel, atau meningkatkan resolusi citra input. Namun, (Tan & Le, 2019) menemukan bahwa penskalaan dimensi tunggal ini memiliki keterbatasan dan efektivitasnya akan menurun seiring membesarnya model.

Perbedaan antara metode konvensional dengan metode yang diusulkan EfficientNet dapat dilihat pada Gambar 2.2.



Gambar 2.2 Ilustrasi Perbandingan Penskalaan Model

Berdasarkan ilustrasi di atas, terdapat perbedaan mendasar dalam strategi penskalaan model:

1. Model *Dasar* (a): Merupakan arsitektur awal jaringan saraf tiruan.
2. Penskalaan Konvensional (b-d): Menambah dimensi secara terpisah. Gambar (b) menunjukkan *width scaling* (jaringan lebih lebar), Gambar (c) menunjukkan *depth scaling* (jaringan lebih dalam/banyak *layer*), dan Gambar (d) menunjukkan *resolution scaling* (resolusi input lebih tinggi). Metode ini sering kali menghasilkan kinerja yang suboptimal karena tidak memperhitungkan ketergantungan antar dimensi.
3. Penskalaan Gabungan (e): Metode yang diperkenalkan oleh EfficientNet, di mana ketiga dimensi (kedalaman, lebar, dan resolusi) ditingkatkan secara serentak dan seragam menggunakan koefisien majemuk yang tetap (Tan & Le, 2019).

Logika di balik *compound scaling* ini didasarkan pada intuisi bahwa jika resolusi citra input diperbesar, jaringan membutuhkan lebih banyak *layer* untuk memperluas *receptive field*, serta lebih banyak *channel* untuk menangkap pola detail yang lebih halus pada citra yang lebih besar tersebut.

2.5.2 EfficientNetV2

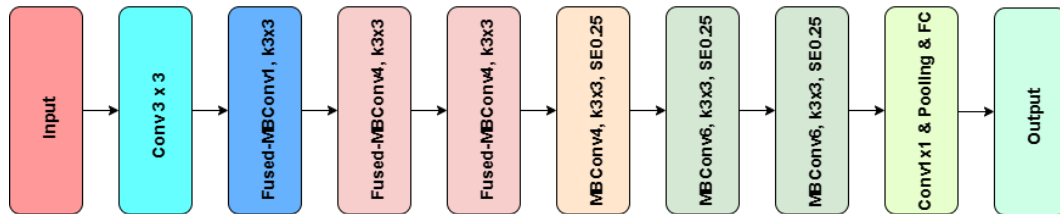
EfficientNetV2 merupakan pengembangan lebih lanjut yang dirancang untuk mengatasi beberapa keterbatasan pada versi pertama, terutama dengan tujuan untuk memiliki kecepatan pelatihan yang lebih cepat dan efisiensi parameter yang lebih baik (Waskita et al., 2023). Untuk mencapai ini, EfficientNetV2 memperkenalkan beberapa optimasi arsitektur dan teknik pelatihan. Salah satu peningkatannya adalah penggunaan lapisan Fused-MBConv pada tahap-tahap awal jaringan, yang lebih efisien secara komputasi daripada lapisan MBConv standar yang digunakan

di EfficientNetB0. Selain itu, model ini menerapkan *progressive learning*, yaitu sebuah strategi pelatihan di mana ukuran gambar dan kekuatan regularisasi ditingkatkan secara bertahap untuk mempercepat konvergensi. Keluarga EfficientNetV2 terdiri dari beberapa varian dengan ukuran yang berbeda untuk kebutuhan yang beragam:

1. EfficientNetV2-S (*Small*): Varian yang lebih kecil dan dioptimalkan untuk pelatihan yang lebih cepat serta penggunaan memori yang lebih rendah. Model ini sangat cocok untuk skenario yang membutuhkan inferensi cepat. Sebuah studi komparatif oleh (Gencer, 2025) menemukan bahwa EfficientNetV2-S mengungguli model lain dan mencapai akurasi tertinggi sebesar 98.20% dalam klasifikasi tumor otak, dengan kinerja yang seimbang di semua kelas.
2. EfficientNetV2-M (*Medium*): Varian ini menyeimbangkan antara ukuran model dan performa. Ia memperluas kapabilitas EfficientNetV2-S dengan meningkatkan kedalaman dan lebar jaringan, sehingga cocok untuk tugas yang membutuhkan akurasi tinggi dengan sumber daya komputasi yang moderat.
3. EfficientNetV2-L (*Large*): Merupakan varian terbesar yang menawarkan performa superior dengan menskalakan dimensi jaringan secara signifikan. Model ini dirancang untuk aplikasi dengan akurasi sangat tinggi di mana sumber daya komputasi bukan menjadi batasan utama, seperti pada tugas-tugas pencitraan medis tingkat lanjut.

Secara keseluruhan, seri EfficientNetV2 menunjukkan peningkatan progresif dalam hal efisiensi komputasi dan akurasi, serta menghasilkan kurva pembelajaran yang lebih stabil dibandingkan dengan EfficientNetB0. Oleh karena itu, arsitektur

ini, khususnya varian EfficientNetV2-S, sangat menjanjikan untuk aplikasi klinis otomatis yang andal. Berikut adalah Gambar 2.3 yang menunjukkan arsitektur EfficientNetV2-S.



Gambar 2.3 Arsitektur EfficientNetV2-S

Berdasarkan Gambar 2.3, arsitektur EfficientNetV2-S yang diusulkan menerima input citra MRI berukuran piksel. Struktur jaringan ini terbagi menjadi dua bagian utama: ekstraktor fitur dan klasifikasi. Berikut adalah rincian tahapan pemrosesan pada arsitektur tersebut:

1. Tahap Awal (Stage 0-3): Ekstraksi Fitur Cepat

Pada layer-layer awal, model menggunakan blok *Fused-MBConv*. Blok ini menggabungkan operasi konvolusi *expand* dan *depthwise* menjadi satu konvolusi standar (Tan & Le, 2021).

- a. *Fused-MBConv1*: Menggunakan rasio ekspansi 1 (tanpa pelebaran channel) untuk efisiensi memori pada resolusi tinggi.
- b. *Fused-MBConv4*: Menggunakan rasio ekspansi 4 untuk mulai menangkap fitur visual dasar yang lebih kompleks. Penggunaan *Fused-MBConv* di tahap ini bertujuan mempercepat proses pelatihan dibanding arsitektur versi sebelumnya (Tan & Le, 2021).

2. Tahap Lanjut (Stage 4-6): Ekstraksi Fitur Mendalam

Pada layer yang lebih dalam, model beralih menggunakan blok *MBConv* standar. Blok ini dilengkapi dengan modul SE 0.25 (*Squeeze-and-Excitation* dengan rasio 0.25).

- a. Mekanisme SE berfungsi memberikan bobot atensi pada *channel* fitur yang memuat informasi tumor penting dan menekan *noise* pada *channel* yang kurang relevan.
- b. Perbedaan antara MBConv4 dan MBConv6 terletak pada rasio ekspansi internalnya (4 kali vs 6 kali lipat), di mana MBConv6 mampu menampung representasi fitur yang lebih abstrak dan detail.

3. Tahap Akhir (Stage 7): Klasifikasi Tumor

Bagian terakhir adalah *head classifier* yang telah dimodifikasi. Output dari ekstraktor fitur melalui lapisan *Conv1x1* dan *Pooling* untuk mereduksi dimensi menjadi vektor fitur. Akhirnya, lapisan *Fully Connected* memetakan fitur tersebut ke dalam 4 neuron output, yang merepresentasikan probabilitas kelas: glioma, meningioma, pituitary, dan no tumor.

2.6 Optimizer

Dalam pelatihan *deep learning*, *optimizer* adalah algoritma inti yang bertugas memperbarui parameter internal dari sebuah jaringan syaraf tiruan, yang dikenal sebagai bobot dan bias. Tujuan utama dari proses ini adalah untuk meminimalkan fungsi kerugian. Fungsi kerugian ini pada dasarnya adalah skor yang mengukur seberapa besar kesalahan yang dibuat model saat memprediksi. Semakin rendah skor kerugiannya, semakin akurat dan baik performa model tersebut.

Proses penyesuaian ini dilakukan secara iteratif. Pada setiap iterasi, *optimizer* menghitung gradien, yang secara konseptual adalah sinyal yang menunjukkan arah di mana kesalahan model akan meningkat paling tajam. Berbekal informasi ini, *optimizer* kemudian mengambil langkah kecil untuk menyesuaikan parameter ke arah yang berlawanan, yaitu ke arah di mana kesalahan akan menurun. Proses ini diulang berkali-kali hingga *optimizer* menemukan titik terendah (minimum) dari fungsi kerugian, di mana model telah mencapai performa paling akurat.

2.6.1 Adaptive Moment Estimation (Adam)

Adam adalah algoritma optimisasi berbasis gradien urutan pertama yang diperkenalkan oleh (Kingma & Ba, 2015). Algoritma ini dirancang untuk menggabungkan keunggulan dari dua metode populer sebelumnya, yaitu AdaGrad yang bekerja baik pada gradien jarang (*sparse*), dan RMSProp yang efektif dalam pengaturan non-stasioner. Secara konseptual, Adam bekerja dengan menghitung estimasi adaptif dari momen pertama (rata-rata gradien) dan momen kedua (varians gradien yang tidak terpusat). Algoritma ini juga menerapkan mekanisme koreksi bias untuk memastikan estimasi momen tetap akurat pada tahap awal pelatihan ketika gradien diinisialisasi dengan nol. Inti dari algoritma Adam adalah aturan pembaruan parameter yang menyesuaikan bobot jaringan saraf berdasarkan rasio antara estimasi momen pertama dan akar kuadrat dari estimasi momen kedua. Secara matematis, pembaruan parameter pada setiap langkah iterasi () dirumuskan sebagai berikut:

: Parameter (bobot dan bias) dari *neural network* pada *timestep* ke-.

: Parameter model sebelum iterasi ke

: Estimasi momen pertama yang telah dikoreksi biasanya

: Estimasi momen kedua yang telah dikoreksi biasanya

: *learning rate*

: konstanta yang sangat kecil (misalnya)

Dengan mekanisme ini, Adam mampu melakukan penskalaan laju pembelajaran secara otomatis untuk setiap parameter, sehingga proses konvergensi menjadi lebih efisien dibandingkan metode *Stochastic Gradient Descent* (SGD) standar (Kingma & Ba, 2015).

2.6.2 Adam with Decoupled Weight Decay (AdamW)

Loshchilov & Hutter, (2019) mengidentifikasi adanya masalah dalam implementasi regularisasi (yang sering disebut *weight decay*). Pada *optimizer* adaptif seperti Adam, *weight decay* standar menjadi tidak efektif karena terikat (*coupled*) dengan estimasi momen kedua (). Hal ini menyebabkan bobot dengan gradien historis yang besar menerima *weight decay* yang lebih kecil, padahal seharusnya tidak.

AdamW adalah varian Adam yang memperbaiki masalah ini dengan memisahkan mekanisme *weight decay* dari langkah optimisasi gradien adaptif. Dalam AdamW, gradien murni digunakan untuk menghitung dan (seperti pada rumus di bawah). Kemudian, pembaruan bobot dan penerapan *weight decay* (dengan faktor) dilakukan secara terpisah seperti pada rumus berikut:

- : Parameter (bobot dan bias) dari *neural network* pada *timestep* ke-.
- : Parameter model setelah iterasi ke
- : Estimasi momen pertama yang telah dikoreksi biasnya
- : Estimasi momen kedua yang telah dikoreksi biasnya
- : *learning rate*
- : konstanta yang sangat kecil (misalnya)

Pemisahan ini memastikan bahwa *weight decay* diterapkan secara seragam ke semua parameter, sebanding dengan besaran parameter itu sendiri, seperti pada formulasi aslinya. Modifikasi ini terbukti secara signifikan meningkatkan kinerja generalisasi model dan membuatnya lebih stabil, terutama pada tugas-tugas *computer vision* (Loshchilov & Hutter, 2019). Oleh karena itu, AdamW sering menjadi pilihan utama dibandingkan Adam standar untuk pelatihan model modern.

2.7 Penjadwalan Laju Pembelajaran *ReduceLROnPlateau*

ReduceLROnPlateau adalah algoritma penjadwalan laju pembelajaran yang bekerja secara dinamis dengan memantau kinerja model pada metrik validasi selama proses pelatihan. Dalam ekosistem pengembangan *Deep Learning* modern seperti PyTorch, teknik ini digunakan untuk mengatasi masalah stagnasi, di mana model kesulitan mencapai penurunan kerugian yang signifikan meskipun pelatihan terus berjalan. Berbeda dengan pendekatan peluruhan statis yang menurunkan laju pembelajaran berdasarkan interval waktu tetap, *ReduceLROnPlateau* bersifat responsif terhadap data. Ia hanya akan melakukan intervensi ketika metrik yang dipantau (seperti fungsi kerugian pada data validasi) berhenti membaik setelah melewati periode toleransi yang disebut *patience*.

Mekanisme utama algoritma ini adalah mengurangi nilai laju pembelajaran saat ini dengan mengalikannya menggunakan faktor peluruhan (α) yang bernilai antara 0 dan 1 (misalnya 0,1 atau 0,5) (Paszke et al., 2019). Penurunan laju pembelajaran ini bertujuan untuk memperkecil langkah pembaruan bobot dalam ruang optimisasi, sehingga memungkinkan model untuk melakukan penelusuran yang lebih halus di sekitar titik minimum lokal atau global. Secara matematis, pembaruan laju pembelajaran (α) saat kondisi stagnasi terdeteksi dirumuskan sebagai berikut:

Dimana:

α_{new} : Laju pembelajaran baru yang akan digunakan pada *epoch* berikutnya.

α_{old} : Laju pembelajaran pada *epoch* saat ini.

α : Faktor peluruhan (*decay factor*), sebuah konstanta .

2.8 Fungsi Kerugian *CrossEntropyLoss*

CrossEntropyLoss merupakan fungsi kerugian standar yang digunakan dalam tugas klasifikasi multi-kelas untuk mengukur kinerja model klasifikasi yang outputnya berupa nilai probabilitas antara 0 dan 1. Fungsi ini mengukur divergensi atau perbedaan antara dua distribusi probabilitas: distribusi prediksi yang dihasilkan oleh model (biasanya setelah melewati fungsi aktivasi *Softmax*) dan distribusi label aktual (*ground truth*). Penggunaan *Cross Entropy* dalam jaringan saraf tiruan berakar pada prinsip *Maximum Likelihood Estimation*, di mana tujuannya adalah memaksimalkan probabilitas bahwa model memprediksi label yang benar untuk data pelatihan yang diberikan (Adugna et al., 2024).

CrossEntropyLoss bekerja dengan memberikan penalti yang bersifat logaritmik terhadap kesalahan prediksi. Jika model memprediksi kelas target dengan probabilitas yang rendah (mendekati 0), nilai kerugian (*loss*) akan meningkat secara drastis menuju tak hingga, yang memaksa *optimizer* untuk mengoreksi bobot model secara agresif. Sebaliknya, jika probabilitas prediksi mendekati 1, nilai *loss* akan mendekati 0. Rumus matematis untuk *CrossEntropyLoss* () pada kasus klasifikasi multi-kelas didefinisikan sebagai:

Dimana:

: Jumlah total kelas (dalam penelitian ini adalah 4: Glioma, Meningioma, Pituitary, *No tumor*).

: Indikator biner (0 atau 1) jika label kelas adalah klasifikasi yang benar untuk observasi .

: Probabilitas prediksi model bahwa observasi termasuk dalam kelas .

2.9 Perangkat Lunak yang Digunakan

Penelitian ini memanfaatkan beberapa perangkat lunak utama untuk membangun, melatih, dan menyajikan model klasifikasi. Bahasa pemrograman inti yang digunakan adalah Python, yang dipilih karena ekosistemnya yang matang dan dukungan *library* yang luas untuk komputasi ilmiah dan *deep learning*. Untuk implementasi model *Convolutional Neural Network* (CNN) dan proses *transfer learning* dengan arsitektur EfficientNetV2-S, penelitian ini menggunakan *framework* PyTorch. PyTorch menyediakan fleksibilitas dan kontrol tingkat tinggi yang diperlukan untuk eksperimen dan pengembangan model. Selanjutnya, untuk

membangun antarmuka pengguna berbasis *web* yang interaktif guna mendemonstrasikan hasil klasifikasi model, digunakan Streamlit. Pustaka ini memungkinkan pembuatan aplikasi data secara cepat dan efisien langsung dari skrip Python. Penjelasan lebih rinci mengenai ketiga perangkat lunak ini akan diuraikan pada sub-bab berikut.

2.9.1 Python

Python adalah bahasa pemrograman interpretatif, *high-level*, dan *general-purpose* yang dikenal luas karena sintaksnya yang bersih, sederhana, dan mudah dibaca. Dalam beberapa dekade terakhir, Python telah menjadi bahasa dominan dalam komputasi ilmiah dan Kecerdasan Buatan, terutama karena ekosistem pustakanya yang matang (Harris et al., 2020).

Kekuatan utama Python dalam penelitian *Deep Learning* bukan hanya terletak pada kesederhanaan bahasanya, tetapi pada pustaka komputasi berkinerja tinggi. Pustaka fundamental seperti NumPy menyediakan struktur data *array* multi-dimensi yang efisien untuk operasi numerik. Di atas pondasi inilah kerangka kerja *Deep Learning* seperti PyTorch dibangun, yang memungkinkan peneliti merancang dan melatih arsitektur *neural network* yang kompleks (Paszke et al., 2019).

2.9.2 PyTorch

PyTorch adalah salah satu kerangka kerja (*framework*) *Deep Learning open-source* berbasis Python yang paling populer, yang dikembangkan oleh laboratorium riset AI Meta (Paszke et al., 2019). PyTorch dirancang untuk memberikan

fleksibilitas tinggi dan kecepatan dalam alur kerja penelitian hingga penerapan produksi.

Dua fitur inti yang mendefinisikan PyTorch adalah:

1. **Komputasi Tensor:** Mirip dengan NumPy, PyTorch menyediakan struktur data Tensors, yaitu *array* multi-dimensi. Keunggulan utamanya adalah Tensors ini dapat dengan mudah dieksekusi pada *Graphics Processing Units* (GPU), yang secara drastis mempercepat operasi matriks paralel yang masif yang diperlukan dalam *Deep Learning*.
2. **Diferensiasi Otomatis (*Autograd*):** PyTorch menggunakan mekanisme yang disebut *tape-based autograd* untuk membangun *dynamic computation graphs* (grafik komputasi dinamis). Ini berarti bahwa alur komputasi (maju dan mundur) didefinisikan saat dieksekusi (*run-time*), memberikan fleksibilitas yang sangat besar bagi peneliti untuk mengimplementasikan arsitektur model yang kompleks dan mempermudah proses *debugging*.

Dalam penelitian ini, PyTorch digunakan untuk memuat arsitektur *pre-trained* EfficientNetV2-S, memodifikasi lapisan klasifikasi akhir (*head*), dan melakukan seluruh proses pelatihan (*fine-tuning*) dan evaluasi model.

2.9.3 Streamlit

Streamlit adalah sebuah kerangka kerja (*framework*) *open-source* berbasis Python yang dirancang untuk membangun dan membagikan aplikasi *web* interaktif untuk proyek *machine learning* dan ilmu data dengan cepat dan mudah. Dalam penelitian ini, Streamlit akan digunakan untuk membuat prototipe aplikasi *web*

sederhana guna mendemonstrasikan hasil klasifikasi dari model yang telah dilatih secara visual dan intuitif.

Keunggulan utama Streamlit terletak pada kesederhanaannya yang memungkinkan peneliti untuk mengubah skrip data menjadi aplikasi *web* hanya dengan beberapa baris kode, tanpa memerlukan pengetahuan mendalam tentang pengembangan *front-end* seperti HTML, CSS, atau JavaScript. Aplikasi yang akan dibangun akan memiliki antarmuka yang interaktif, di mana pengguna dapat mengunggah data baru (seperti gambar atau teks) melalui sebuah *widget*. Setelah data diunggah, aplikasi akan memprosesnya menggunakan model *machine learning* yang telah dilatih dan menampilkan hasil prediksi klasifikasi secara langsung di halaman *web*, sehingga mempermudah proses validasi dan presentasi model.

2.10 Penelitian Terkait

Beberapa penelitian telah dilakukan untuk mengklasifikasikan tumor otak menggunakan berbagai arsitektur *deep learning* guna meningkatkan akurasi diagnosis medis secara otomatis. Tabel 2.1 merangkum empat penelitian terdahulu yang menggunakan variasi dataset dan metode model sebagai referensi utama dalam dokumen ini.

Tabel 2.1 Penelitian Terkait

Peneliti (Tahun)	Objektif	Metode	Akurasi	Dataset
Azzahra, dkk. (2023)	Klasifikasi 4 jenis citra MRI (glioma, meningioma, pituitary, <i>no tumor</i>)	CNN (Inception Resnet v2)	93.4%	Brain Tumor Classification (MRI) dari kaggle.com

Peneliti (Tahun)	Objektif	Metode	Akurasi	Dataset
Hamidja, dkk. (2024)	Klasifikasi tumor otak pada dataset publik untuk efisiensi komputasi	CNN (KKDNet)	>90%	Brain Tumor Classification (MRI) dari kaggle.com
Waskita, dkk. (2023)	Klasifikasi 4 jenis citra MRI (glioma, meningioma, pituitary, normal)	CNN (EfficientNetV2 B0, B1, B2, B3)	>99%	Crystal Clean: Brain Tumors MRI Dataset dari kaggle.com
Wong, dkk. (2023)	Klasifikasi tumor otak menggunakan <i>Deep Learning</i> dengan dataset skala besar	CNN (VGG16 + Augmentasi pada 3 Dataset Publik Gabungan)	99.24%	Brain tumor dataset (figshare), Br35H :: Brain Tumor Detection 2020 (kaggle.com), dan Brain Tumor Classification (MRI) dari kaggle.com

2.11 Evaluasi dan Pengujian Sistem

Evaluasi dan pengujian dalam penelitian ini dilakukan secara menyeluruh untuk memastikan keandalan model klasifikasi serta fungsionalitas sistem yang dibangun. Proses ini mencakup tiga pendekatan utama, yaitu penerapan metode *K-Fold Cross Validation* untuk memvalidasi kekokohan model dalam mempelajari data guna menghindari bias, penggunaan berbagai metrik evaluasi untuk mengukur performa klasifikasi secara kuantitatif, serta pengujian *Black Box Testing* untuk memverifikasi bahwa seluruh fitur dan antarmuka pada aplikasi *web* dapat berjalan sesuai dengan kebutuhan pengguna tanpa meninjau struktur kode internal.

2.11.1 K-Fold Cross Validation

K-Fold Cross Validation adalah metode statistik yang digunakan untuk memperkirakan keterampilan model pembelajaran mesin pada data yang belum pernah dilihat sebelumnya. Dalam metode ini, dataset tunggal dibagi secara acak

menjadi bagian yang berukuran sama, yang disebut sebagai *fold*. Proses pelatihan dan validasi kemudian diulangi sebanyak kali (iterasi). Pada setiap iterasi, satu *fold* digunakan sebagai data uji (validasi), sedangkan *Fold* sisanya digunakan sebagai data latih. Validasi silang (*cross-validation*) ini bertujuan untuk menilai seberapa baik hasil analisis statistik dapat digeneralisasi ke kumpulan data independen, serta untuk mendeteksi masalah seperti *overfitting* atau *selection bias* (Friedl et al., 2002).

Keunggulan utama dari metode ini adalah kemampuannya dalam menghasilkan evaluasi yang lebih akurat dan tidak bias dibandingkan dengan metode pembagian sederhana (*hold-out*). Karena setiap titik data dalam dataset memiliki kesempatan tepat satu kali untuk menjadi data uji dan kali menjadi data latih, varians dari estimasi performa model dapat dikurangi. Perkiraan kesalahan prediksi model diperoleh dengan merata-ratakan hasil kesalahan dari setiap iterasi *fold*, sehingga memberikan gambaran kinerja yang lebih representatif terhadap keseluruhan data. Hal ini sangat krusial dalam dataset medis terbatas, seperti pada kasus tumor otak, untuk memastikan model tidak hanya menghafal data tertentu.

2.11.2 Metrik Evaluasi

Untuk menganalisis kinerja model secara kuantitatif, metrik evaluasi berikut akan digunakan, yang dihitung berdasarkan nilai dari confusion matrix (TP: *True Positive*, TN: *True Negative*, FP: *False Positive*, FN: *False Negative*) (Azzahra et al., 2023).

1. Akurasi: Mengukur proporsi prediksi yang benar dari keseluruhan data.

2. Presisi: Mengukur tingkat ketepatan dari prediksi positif.
3. *Recall*: Mengukur kemampuan model untuk mengidentifikasi semua sampel positif yang sebenarnya.
4. *F1-Score*: Rata-rata harmonik dari presisi dan *recall*, memberikan skor tunggal yang menyeimbangkan *presisi* dan *recall*.

2.11.3 *Black Box Testing*

Black Box Testing merupakan metode pengujian perangkat lunak yang berfokus sepenuhnya pada fungsionalitas sistem tanpa memperhatikan struktur logika internal atau kode program yang membangunnya. Dalam pendekatan ini, penguji memandang aplikasi sebagai sebuah kotak tertutup di mana evaluasi dilakukan hanya berdasarkan kesesuaian antara input yang diberikan dengan output yang dihasilkan oleh sistem. Tujuan utama dari teknik ini adalah untuk memeriksa apakah perangkat lunak memenuhi persyaratan fungsional tanpa perlu memahami detail implementasi teknis yang rumit di dalamnya (Nidhra, 2012).

Penerapan *Black Box Testing* sangat krusial dalam pengembangan aplikasi berbasis *web* untuk mendeteksi berbagai jenis kesalahan yang mungkin terjadi pada antarmuka pengguna maupun fungsi sistem. Teknik ini bertujuan menemukan kesalahan dalam kategori fungsi yang tidak benar atau hilang, kesalahan antarmuka, kesalahan pada struktur data atau akses basis data, serta kesalahan inisialisasi dan

terminasi program (Khan, 2011). Khusus dalam penelitian ini, pengujian akan difokuskan pada elemen-elemen interaktif seperti tombol navigasi, formulir pengunggahan citra MRI, serta tampilan hasil prediksi untuk memastikan tidak ada kendala saat digunakan. Dengan demikian, metode ini menjamin bahwa aplikasi dapat beroperasi dengan stabil dan memberikan respons yang akurat kepada pengguna sebelum sistem tersebut disebarluaskan atau digunakan secara nyata.

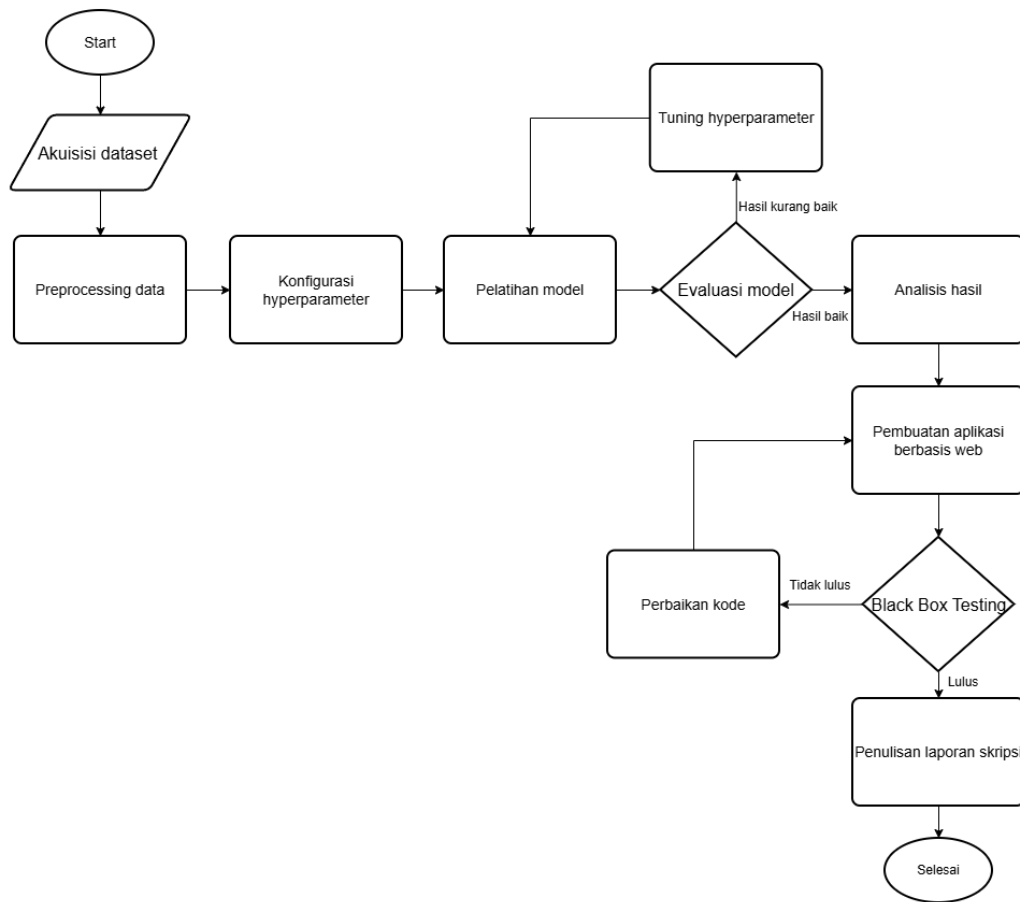
BAB III

ANALISIS DAN PERANCANGAN

Pada bab ini dijelaskan mengenai tahapan penelitian dan metode yang digunakan dalam membangun model klasifikasi. Pembahasan meliputi alur penelitian, analisis kebutuhan (kebutuhan data, perangkat keras, dan perangkat lunak), perancangan scenario eksperimen, perancangan model *deep learning*, perancangan prototipe aplikasi, dan pengujian fungsionalitas aplikasi.

3.1 Alur Penelitian

Secara garis besar, tahapan penelitian ini dirancang secara sistematis mulai dari identifikasi masalah hingga penulisan laporan akhir. Alur penelitian digambarkan dalam diagram alir Gambar 3.1. Proses ini terdiri dari beberapa fase utama, yaitu studi pendahuluan, pengumpulan data, pengembangan model, evaluasi, implementasi aplikasi, dan penyusunan laporan. Setiap tahapan dalam alur penelitian tersebut saling terintegrasi untuk memastikan bahwa model yang dikembangkan memiliki landasan teoritis yang kuat serta performa yang teruji sebelum diimplementasikan ke dalam sistem. Pendekatan terstruktur ini bertujuan agar seluruh target penelitian dapat dicapai secara efektif dengan meminimalisir risiko kesalahan pada setiap fase pengembangan aplikasi. Berdasarkan Gambar 3.1, rincian dari setiap tahapan penelitian adalah sebagai berikut.



Gambar 3.1 Diagram Alur Penelitian

1. Studi Literatur dan Analisis Masalah

Penelitian diawali dengan studi literatur untuk memahami konsep dasar klasifikasi tumor otak, arsitektur *Convolutional Neural Network* (CNN), serta karakteristik model EfficientNetV2-S. Pada tahap ini juga dilakukan analisis masalah untuk menentukan rumusan masalah dan tujuan penelitian.

2. Pengumpulan Data (Akuisisi Dataset)

Tahap pengumpulan data dilakukan melalui dua jalur. Pertama, mengunduh dataset *baseline* dan dataset untuk validasi input. Kedua, menyusun dataset campuran guna meningkatkan keragaman data latih.

3. Pra-pemrosesan Data (*Preprocessing*)

Data yang telah dikumpulkan kemudian masuk ke tahap pra-pemrosesan. Proses ini meliputi *resizing* citra agar sesuai dengan input model, normalisasi nilai piksel, *random rotation*, *random horizontal flip*, serta pembagian data (*data splitting*) menjadi data latih (*train*) dan validasi (*validation*).

5. Konfigurasi dan Pelatihan Model

Sebelum pelatihan dimulai, dilakukan konfigurasi *hyperparameter* seperti penentuan laju pembelajaran, *optimizer* (AdamW), dan *loss function*. Selanjutnya, model EfficientNetV2-S dilatih menggunakan lingkungan komputasi Kaggle.

6. Evaluasi dan *Tuning Hyperparameter*

Setelah proses pelatihan, model dievaluasi menggunakan data validasi. Terdapat proses pengambilan keputusan pada tahap ini:

- Jika hasil evaluasi kurang baik, maka penelitian akan kembali ke tahap *tuning hyperparameter* untuk menyesuaikan konfigurasi ulang dan melakukan pelatihan ulang.
- Jika hasil evaluasi baik dan optimal, proses dilanjutkan ke tahap analisis hasil akhir pada data uji.

7. Pembuatan Aplikasi dan Pengujian

Model terbaik yang telah dilatih kemudian diimplementasikan ke dalam antarmuka berbasis *web* menggunakan *framework* Streamlit. *Web* ini kemudian diuji fungsionalitasnya menggunakan metode *Blackbox Testing*.

8. Penulisan Laporan dan Selesai

Tahap terakhir adalah mendokumentasikan seluruh proses, hasil analisis, dan kesimpulan ke dalam laporan skripsi, yang menandai selesainya seluruh rangkaian penelitian ini.

3.2 Analisis Kebutuhan

Tahap analisis kebutuhan dilakukan untuk memetakan sumber daya yang diperlukan dalam penelitian ini. Identifikasi ini mencakup spesifikasi perangkat keras dan perangkat lunak yang akan digunakan. Selain itu, kebutuhan data juga didefinisikan secara jelas agar sesuai dengan model yang dibangun. Spesifikasi yang tepat sangat penting untuk mendukung proses pelatihan dan pengujian sistem. Dengan demikian, seluruh rangkaian eksperimen dapat berjalan lancar tanpa kendala teknis yang berarti.

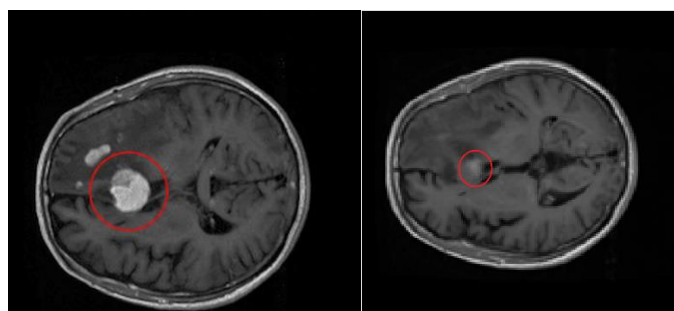
3.2.1 Kebutuhan Data

Sistem klasifikasi jenis tumor otak dalam penelitian ini memerlukan dataset citra *Magnetic Resonance Imaging* (MRI) yang mencakup berbagai variasi kondisi tumor dan *no tumor*. Data mentah diperoleh dari empat sumber dataset publik yang berbeda untuk membangun skenario eksperimen *baseline* dan campuran. Selain citra medis, penelitian ini juga mengintegrasikan dataset citra objek umum, yakni *Common Objects in Context* (COCO), yang difungsikan secara spesifik untuk melatih model pada tahap validasi input. Penggunaan dataset COCO sangat krusial sebagai kelas representatif *non-MRI* pada mekanisme *multilayer*, sehingga sistem mampu menyaring dan menolak citra masukan yang tidak relevan sebelum

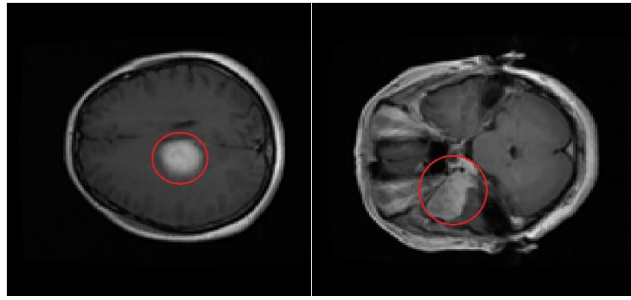
diteruskan ke tahap klasifikasi tumor utama. Rincian dari seluruh sumber data tersebut adalah sebagai berikut:

1. Dataset Figshare (Brain Tumor Dataset)

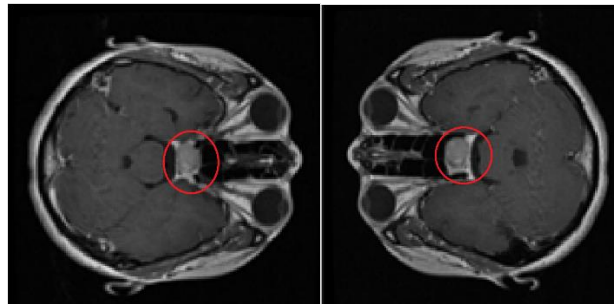
Dataset ini merupakan dataset publik yang disusun oleh Jun Cheng dan dipublikasikan pada tahun 2024. Dataset ini berisi 3.064 citra *T1-tertimbang contrast-enhanced* yang mencakup tiga jenis tumor otak, yaitu 1426 citra Glioma, 708 citra Meningioma, dan 930 citra Pituitary. Data disediakan dalam format .mat yang dilengkapi dengan kode MATLAB untuk konversi ke format citra .png, serta protokol akuisisi citra untuk menjaga standar kualitas data. Dalam penelitian ini, dataset tersebut digunakan sebagai dataset campuran dan juga sebagai salah satu sumber dataset pada penelitian Wong et al., (2025). Sebagai representasi visual dari dataset Figshare ini, sampel citra untuk masing-masing kategori tumor disajikan secara terpisah. Contoh karakteristik visual untuk kelas glioma ditunjukkan pada Gambar 3.2, kelas meningioma pada Gambar 3.3, dan kelas pituitary pada Gambar 3.4. Melalui visualisasi tersebut, dapat diamati dengan jelas perbedaan pada struktur dari masing-masing tumor yang nantinya menjadi fitur krusial untuk dipelajari oleh model.



Gambar 3.2 Contoh Citra Kelas Glioma pada Dataset Figshare



Gambar 3.3 Contoh Citra Kelas Meningioma pada Dataset Figshare

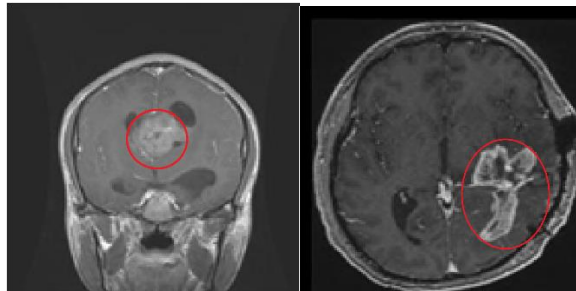


Gambar 3.4 Contoh Citra Kelas Pituitary pada Dataset Figshare

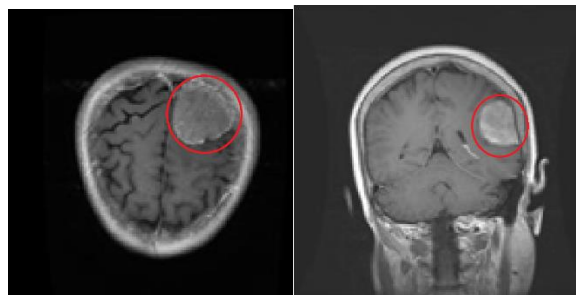
2. Dataset Crystal Clean

Dataset ini berasal dari Kaggle yang merupakan versi pengembangan dari dataset asli milik Sartaj Bhuvaji dan telah melalui pembersihan data. Perbedaan utama dataset ini terletak pada integritas datanya, di mana pengembang melakukan penghapusan sampel duplikat menggunakan metode *image vector comparison* serta koreksi terhadap label yang salah berdasarkan pengetahuan domain medis. Seluruh citra dalam dataset ini telah diseragamkan ukurannya menjadi piksel agar selaras dengan spesifikasi input model EfficientNetV2-S. Dalam penelitian ini, dataset tersebut digunakan sebagai dataset campuran dan juga sebagai sumber dataset pada penelitian (Waskita et al., 2023). Berdasarkan rincian distribusi data, dataset ini memiliki total 21.672 citra yang terdiri dari 6.307 sampel Glioma, 6.391

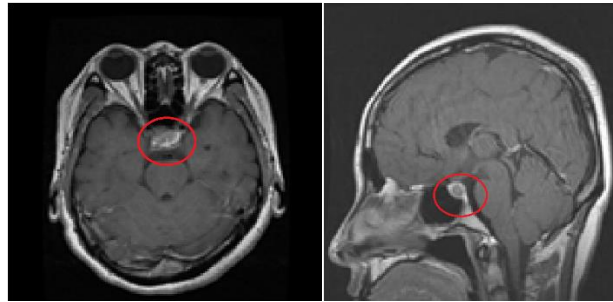
Meningioma, 5.908 Pituitary, dan 3.066 sampel *No tumor*. Proses kurasi yang ketat pada dataset ini bertujuan untuk meminimalisir risiko *overfitting* dan bias pada model akibat adanya data yang redundan atau salah pelabelan. Dengan menggunakan data yang telah terstandarisasi, model diharapkan dapat mempelajari fitur-fitur pembeda yang lebih representatif dari setiap jenis tumor otak untuk menghasilkan prediksi yang lebih konsisten. Sebagai gambaran representatif dari hasil kurasi dataset Crystal Clean ini, sampel visual untuk kelas Glioma ditunjukkan pada Gambar 3.5, kelas Meningioma pada Gambar 3.6, kelas Pituitary pada Gambar 3.7, dan kelas *No tumor* pada Gambar 3.8.



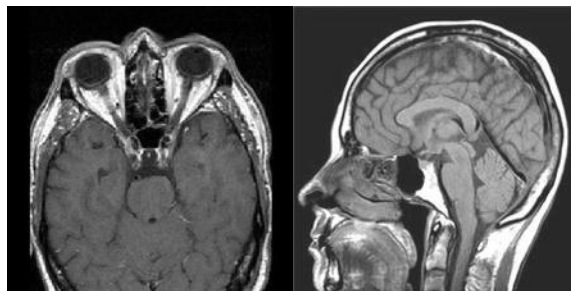
Gambar 3.5 Contoh Citra Glioma pada Dataset Crystal Clean



Gambar 3.6 Contoh Citra Meningioma pada Dataset Crystal Clean



Gambar 3.7 Contoh Citra Pituitary pada Dataset Crystal Clean

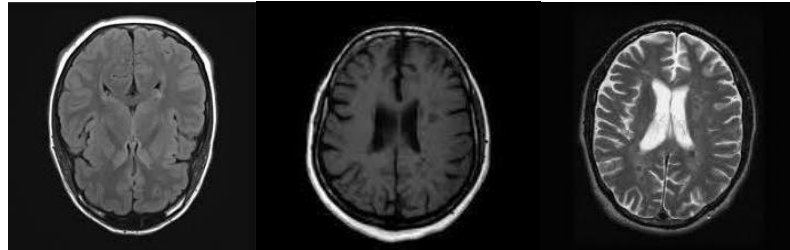


Gambar 3.8 Contoh Citra *No tumor* pada Dataset Crystal Clean

3. Dataset Br35H (Brain Tumor Detection 2020)

Dataset ini merupakan dataset publik yang berasal dari Kaggle dan disusun oleh Ahmed Hamada yang dirancang khusus untuk tugas deteksi keberadaan tumor pada citra MRI otak. Dataset ini terdiri dari total 3.000 citra yang terbagi secara seimbang ke dalam dua kategori utama, yaitu 1.500 citra dengan kondisi tumor dan 1.500 citra tanpa tumor. Dalam penelitian ini, dataset Br35H dimanfaatkan secara spesifik untuk memperkuat basis data pada kelas *No tumor* guna mendukung skenario eksperimen dataset campuran. Penggunaan sumber ini sejalan dengan penelitian yang dilakukan oleh (Wong et al., 2025), yang juga mengintegrasikan dataset Br35H sebagai salah satu penyedia sampel citra untuk kelas *No tumor* dalam penelitian mereka. Berikut

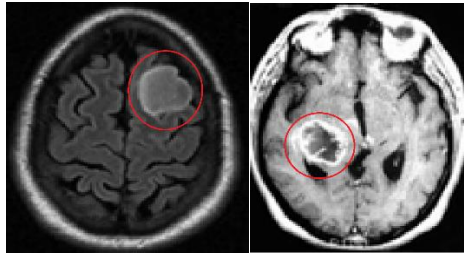
adalah Gambar 3.9 yang menunjukkan sampel gambar kelas *No tumor* pada dataset ini.



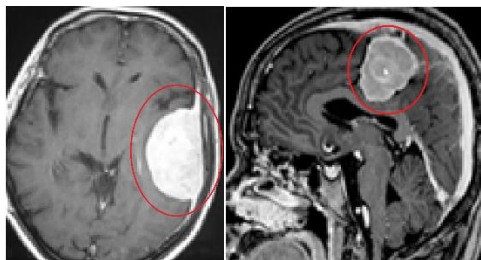
Gambar 3.9 Contoh Citra *No tumor* pada Dataset BR35H

4. Dataset Brain Tumor Classification (MRI)

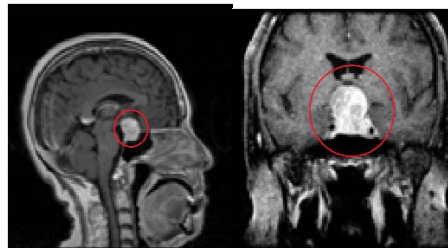
Dataset ini merupakan dataset publik yang disusun oleh Sartaj Bhuvaji dkk. untuk mengatasi kompleksitas analisis manual citra MRI yang rentan terhadap kesalahan serta membantu diagnosis di wilayah dengan keterbatasan tenaga ahli. Dataset ini menyediakan empat kelas lengkap dengan total 3.264 citra yang terdiri dari 926 sampel Glioma, 937 Meningioma, 901 Pituitary, dan 500 sampel *No tumor*. Dalam penelitian ini, dataset ini digunakan sebagai salah satu sumber dataset campuran. Dataset ini digunakan sebagai salah satu sumber dalam penelitian (Wong et al., 2025), juga digunakan sebagai satu-satunya sumber dataset pada penelitian (Azzahra et al., 2023), dan (Hamidja et al., 2024). Sebagai representasi visual dari dataset *Brain Tumor Classification* ini, contoh citra untuk kelas Glioma ditunjukkan pada Gambar 3.10, kelas Meningioma pada Gambar 3.11, kelas Pituitary pada Gambar 3.12, dan kelas *No Tumor* pada Gambar 3.13.



Gambar 3.10 Contoh Citra Glioma pada Dataset Brain Tumor Classification



Gambar 3.11 Citra Meningioma pada Dataset Brain Tumor Classification



Gambar 3.12 Contoh Citra Pituitary pada Dataset Brain Tumor Classification



Gambar 3.13 Contoh Citra *No Tumor* pada Dataset Brain Tumor Classification

5. COCO Dataset 2017

Sebagai tambahan untuk melengkapi keandalan sistem klasifikasi, penelitian ini juga memanfaatkan dataset citra objek umum (bukan citra otak),

yaitu COCO Dataset 2017 yang diakses melalui repositori publik Kaggle. Berbeda dengan keempat dataset sebelumnya yang digunakan untuk mengklasifikasikan jenis tumor, dataset COCO secara spesifik digunakan sebagai representasi kelas "Bukan Citra Otak" pada tahap pelatihan model klasifikasi biner dan jumlah citra yang digunakan dari dataset berjumlah 31.861 gambar. Model biner ini berfungsi sebagai mekanisme filter awal untuk memvalidasi apakah citra masukan dari pengguna merupakan citra MRI otak atau bukan, sehingga sistem dapat menolak pemrosesan gambar yang tidak relevan. Sebagai representasi visual dari kelas objek umum yang digunakan dalam tahap validasi tersebut, contoh sampel citra dari dataset COCO ditunjukkan pada Gambar 3.14.



Gambar 3. 14 Contoh Citra pada COCO Dataset

3.2.2 Kebutuhan Perangkat Keras

Demi menjaga integritas eksperimen, keamanan penyimpanan model, dan efisiensi implementasi, penelitian ini menggunakan tiga lingkungan komputasi terpisah dengan spesifikasi sebagai berikut:

1. Lingkungan Pembuatan Model (Kaggle)

Lingkungan ini digunakan khusus untuk melakukan *training* terhadap model EfficientNetV2-S yang nantinya akan digunakan baik untuk komparasi penelitian terdahulu menggunakan dataset *baseline*, maupun implementasi

model pada aplikasi. *Platform* Kaggle dipilih karena menyediakan lingkungan yang stabil dan dataset yang tersedia berasal dari Kaggle sehingga dapat langsung digunakan. Spesifikasi yang digunakan adalah sebagai berikut:

- GPU: NVIDIA Tesla P100 (16GB VRAM).
- CPU: Intel(R) Xeon(R) CPU @ 2.00GHz (2 Cores).
- RAM: 32 GB.
- Platform: Kaggle Kernels.

2. Lingkungan Implementasi dan Aplikasi (*Deployment*)

Untuk tahap pengujian akhir dan menjalankan antarmuka aplikasi berbasis Streamlit, digunakan perangkat komputer lokal (laptop). Penggunaan perangkat ini bertujuan untuk membuktikan bahwa model yang telah dilatih cukup ringan dan efisien untuk dijalankan pada perangkat konsumen standar. Spesifikasi yang digunakan Adalah sebagai berikut:

- Sistem Operasi: Windows 11 64-bit.
- Processor (CPU): Snapdragon(R) X Plus - X1P64100 - Qualcomm(R) Oryon (TM) CPU (3.42 GHz).
- RAM: 16 GB.
- Kartu Grafis: Integrated Qualcomm (R) Adreno (TM) X1-85 GPU.
- Penyimpanan: SSD 1 TB.

3.2.3 Kebutuhan Perangkat Lunak

Perangkat lunak inti yang digunakan untuk implementasi eksperimen adalah:

1. Python: Bahasa pemrograman utama.

2. PyTorch: Framework *deep learning* untuk membangun dan melatih semua model.
3. Streamlit: Untuk pembuatan protipe *website* program klasifikasi.
4. Pustaka Pendukung: NumPy, Scikit-learn (untuk metrik evaluasi), dan Matplotlib (untuk visualisasi).
5. Kaggle: *Cloud environment* untuk melatih model pada dataset *baseline* dan campuran.

3.3 Perancangan Skenario Eksperimen

Desain eksperimen ini difokuskan pada dua skenario komputasi utama untuk mengevaluasi model EfficientNetV2-S. Hasil dari eksperimen pertama akan dibandingkan dengan *benchmark* literatur yang telah diidentifikasi di Bab II, sementara eksperimen kedua akan menguji robustisitas model tersebut.

3.3.1 Pengujian Model EfficientNetV2-S pada Dataset *Baseline*

Skenario pengujian ini berfokus pada implementasi, pelatihan, dan evaluasi model EfficientNetV2-S menggunakan dataset standar (*baseline*) yang umum digunakan dalam komunitas penelitian, yaitu dataset MRI dari Kaggle. Pada tahap ini, evaluasi performa model dilakukan menggunakan metode *10-Fold Cross-Validation*. Pendekatan ini diterapkan untuk mengukur kinerja murni arsitektur model dalam mempelajari fitur-fitur tumor otak secara menyeluruh dan objektif, serta memastikan model memiliki kemampuan generalisasi yang baik dengan meminimalkan bias pada pemilihan data.

Hasil evaluasi dari skenario ini mencakup metrik akurasi, presisi, *recall*, dan *F1-Score* yang akan digunakan sebagai tolak ukur utama untuk menjawab rumusan masalah penelitian terkait efektivitas model. Nilai performansi rata-rata yang diperoleh dari kesepuluh *Fold* akan dibandingkan secara langsung dengan hasil yang dilaporkan oleh penelitian terdahulu yang menggunakan dataset identik. Perbandingan ini krusial untuk memvalidasi apakah arsitektur EfficientNetV2-S yang diusulkan mampu memberikan peningkatan kinerja atau setidaknya kompetitif dibandingkan metode-metode sebelumnya.

3.3.2 Pengujian Model EfficientNetV2-S pada Dataset Campuran

Skenario kedua bertujuan untuk menganalisis dampak diversifikasi data terhadap ketangguhan model. Pada tahap ini, proses pelatihan dan evaluasi menerapkan metode *10-Fold Cross-Validation* menggunakan Dataset Campuran yang merupakan gabungan dari dataset *baseline*. Penggabungan ini dilakukan untuk memperkaya variasi citra, baik dari segi karakteristik tumor, kontras gambar, maupun resolusi spasial yang mungkin berbeda antar-sumber. Dengan melatih model pada data yang lebih heterogen, penelitian ini berupaya mengatasi keterbatasan dataset tunggal yang sering kali memicu model untuk hanya mengenali pola spesifik dari satu sumber data saja.

Fokus utama analisis pada skenario ini adalah mengukur kemampuan generalisasi dan robustitas model. Generalisasi merujuk pada kemampuan model untuk memprediksi dengan benar pada data baru yang memiliki distribusi sedikit berbeda dari data latih, sedangkan robustitas mengacu pada ketahanan model terhadap variasi kualitas citra. Skenario ini dirancang untuk mensimulasikan

kondisi dunia nyata yang lebih kompleks, di mana data medis sering kali berasal dari berbagai perangkat MRI yang berbeda. Sebagai tindak lanjut dari evaluasi ini, model iterasi yang menghasilkan performa terbaik berdasarkan metrik *F1-Score* akan dipilih dan disimpan untuk kemudian diimplementasikan ke dalam pengembangan aplikasi berbasis *web*.

3.3.3 Pengujian Model EfficientNetV2-S pada Dataset COCO untuk Validasi Input Citra

Pengujian ini bertujuan untuk mengevaluasi kinerja model klasifikasi biner yang difungsikan sebagai mekanisme validasi awal sebelum citra masukan diproses oleh sistem klasifikasi utama. Arsitektur EfficientNetV2-S yang telah dimodifikasi pada lapisan pengklasifikasiannya (menjadi dua *node* keluaran) dilatih secara khusus untuk membedakan kelas "MRI Otak" dan kelas "Bukan MRI Otak". Untuk merepresentasikan kelas "Bukan MRI Otak", digunakan dataset *Common Objects in Context* (COCO) yang jumlah datanya telah dipangkas menjadi 31.861 gambar yang digunakan selama proses pelatihan model. Penyesuaian ini sangat krusial untuk menyamai jumlah total dataset citra MRI, sehingga mencegah terjadinya bias akibat ketidakseimbangan kelas selama proses pelatihan. Berbeda dengan model utama yang diuji menggunakan skema *10-Fold Cross-Validation*, pengujian pada model filter ini cukup dievaluasi menggunakan metode *Hold-out* dengan rasio pembagian data 80:10:10 (Pelatihan:Validasi:Pengujian) mengingat tingkat kompleksitas ekstraksi fitur antar kedua kelas ini sangat kontras dan jauh lebih rendah dibandingkan membedakan antar jenis tumor.

3.4 Perancangan Model Deep Learning

Tahap perancangan model merupakan bagian inti untuk mendefinisikan arsitektur jaringan saraf tiruan yang digunakan. Pada penelitian ini, model EfficientNetV2-S dipilih karena keunggulannya dalam hal efisiensi parameter dan akurasi. Implementasi teknis sepenuhnya dilakukan menggunakan bahasa pemrograman Python dengan dukungan framework PyTorch. Penjelasan dalam tahap ini mencakup modifikasi pada layer akhir model agar sesuai dengan jumlah kelas klasifikasi yang dituju. Selain itu, konfigurasi *hyperparameter* pelatihan juga ditentukan secara detail untuk memastikan model dapat bekerja optimal.

3.4.1 Arsitektur Model (EfficientNetV2-S)

Penelitian ini menggunakan arsitektur EfficientNetV2-S yang telah dilatih sebelumnya pada dataset ImageNet. Penggunaan *transfer learning* bertujuan untuk mempercepat proses pelatihan dan meningkatkan akurasi model, mengingat keterbatasan jumlah data citra MRI yang tersedia.

1. Lapisan Input: Model menerima input citra RGB dengan dimensi 224 224 piksel.
2. *Backbone*: Bagian ekstraksi fitur menggunakan lapisan-lapisan konvolusi dari EfficientNetV2-S yang telah dibekukan (*frozen*) sebagian bobotnya pada tahap awal pelatihan untuk mempertahankan fitur-fitur dasar yang telah dipelajari dari ImageNet.
3. *Head (Classifier)*: Lapisan klasifikasi asli dari EfficientNetV2-S diganti dengan lapisan baru yang disesuaikan dengan jumlah kelas tumor otak dalam penelitian

ini, yaitu 4 kelas (Glioma, Meningioma, Pituitary, *No tumor*). Lapisan *head* terdiri dari:

- a. *Dropout Layer*: Dengan probabilitas 0.2 untuk mencegah *overfitting*.
- b. *Linear Layer (Fully Connected)*: Mengubah fitur hasil ekstraksi menjadi 4 *output* yang merepresentasikan probabilitas masing-masing kelas.

3.4.2 Konfigurasi *Hyperparameter* Pelatihan

Proses pelatihan model memerlukan pengaturan *hyperparameter* yang tepat untuk mencapai konvergensi optimal. Berikut adalah rincian konfigurasi yang digunakan:

1. *Optimizer* (Pengoptimal):
 - a. Algoritma: AdamW (*Adaptive Moment Estimation with Decoupled Weight Decay*).
 - b. *Learning Rate* (Laju Pembelajaran): Diatur pada nilai awal 0.001 atau . Nilai ini dipilih sebagai titik awal yang moderat untuk memungkinkan model belajar dengan cepat tanpa menyebabkan *overshooting* pada gradien.
 - c. *Weight Decay*: Sebesar . Parameter ini berfungsi sebagai regularisasi untuk mencegah bobot model menjadi terlalu besar, yang dapat menyebabkan *overfitting*.
2. *Loss Function* (Fungsi Kerugian):
 - a. Fungsi: *CrossEntropyLoss*.
 - b. Deskripsi: Fungsi ini merupakan standar untuk masalah klasifikasi *multi-class*. *CrossEntropyLoss* mengukur perbedaan antara distribusi probabilitas

prediksi model dengan label kelas yang sebenarnya. Semakin kecil nilai *loss*, semakin akurat prediksi model.

3. *Learning Rate Scheduler*:

- a. Metode: *ReduceLROnPlateau*.
- b. Mekanisme: Penjadwal ini akan memantau nilai *validation loss* selama proses pelatihan. Jika *validation loss* tidak menunjukkan penurunan (stagnan) selama sejumlah *epoch* tertentu (*patience*), maka *learning rate* akan dikurangi secara otomatis dengan faktor tertentu.
- c. Konfigurasi:
 - Mode: 'min' (memantau penurunan *loss*).
 - Factor: 0.5 (mengurangi *learning rate* menjadi setengahnya).
 - Patience: 2 *epoch* (memberikan toleransi 2 *epoch* sebelum mengurangi *learning rate* jika tidak ada perbaikan).

4. *Epoch* (Iterasi Pelatihan):

- a. Jumlah Maksimum: 20 *epoch* untuk klasifikasi tumor otak dan 5 *epoch* untuk validasi biner citra otak.
- b. Deskripsi: Satu *epoch* didefinisikan sebagai satu kali proses seluruh dataset pelatihan melewati *neural network*. Jumlah 5 dan 20 dipilih untuk memberikan waktu yang cukup bagi model untuk belajar dan sebagai batas maksimal agar proses *training* tidak memakan waktu yang terlalu lama.
- c. *Early Stopping*: Untuk mencegah *overfitting* dan menghemat waktu komputasi, diterapkan mekanisme *Early Stopping*. Pelatihan akan dihentikan secara otomatis jika *validation loss* tidak membaik selama 10

epoch berturut-turut. Model dengan *validation loss* terendah akan disimpan sebagai model terbaik (*best model*).

5. Batch Size:

- a. Ukuran: 64.
- b. Deskripsi: Jumlah sampel data yang diproses dalam satu kali iterasi pembaruan bobot. Ukuran 64 dipilih untuk menyeimbangkan kecepatan pelatihan dan penggunaan memori GPU.

3.5 Perancangan Prototipe Aplikasi

Perancangan prototipe aplikasi bertujuan untuk menyediakan antarmuka pengguna yang interaktif dan mudah digunakan guna mengimplementasikan model *Deep Learning* yang telah dilatih. Aplikasi ini dibangun menggunakan kerangka kerja Streamlit berbasis bahasa pemrograman Python. Pemilihan Streamlit didasarkan pada efisiensinya dalam mengonversi skrip data sains menjadi aplikasi *web* fungsional tanpa memerlukan pengembangan *backend* yang kompleks, sehingga sangat sesuai untuk kebutuhan demonstrasi model klasifikasi citra medis. Aplikasi ini dirancang sebagai jembatan antara pengguna (tenaga medis atau peneliti) dengan model EfficientNetV2-S, di mana pengguna dapat memasukkan data citra MRI mentah dan mendapatkan hasil diagnosis secara *real-time*.

3.5.1 Rancangan Antarmuka Aplikasi

Rancangan antarmuka pengguna merupakan visualisasi dari fungsionalitas sistem yang dapat diakses langsung oleh pengguna. Antarmuka aplikasi dibagi menjadi empat komponen utama yang disusun secara sistematis dalam satu

halaman untuk memudahkan navigasi. Rincian komponen antarmuka tersebut adalah sebagai berikut:

1. *Widget Pengunggahan Citra (File Uploader)*

Komponen utama yang memungkinkan pengguna untuk memasukkan data citra MRI ke dalam sistem. *Widget* ini dirancang untuk mendukung dua mode input:

- Unggahan Tunggal (*Single Upload*): Pengguna dapat mengunggah satu citra untuk dianalisis secara mendalam.
- Unggahan Jamak (*Multiple Upload*): Pengguna dapat mengunggah beberapa *file* citra sekaligus untuk melakukan klasifikasi secara *batch* (kelompok) guna mempercepat proses pengujian.
- Format *file* yang didukung dibatasi pada format citra standar seperti JPG, JPEG, dan PNG.

2. *Area Pratinjau Citra (Image Preview)*

Setelah citra diunggah, aplikasi akan menampilkan visualisasi citra tersebut pada layar utama. Hal ini berfungsi untuk memastikan bahwa citra yang dimasukkan oleh pengguna sudah sesuai sebelum diproses oleh model.

3. *Tombol Eksekusi Klasifikasi* Sebuah tombol interaktif

Sebuah tombol yang berfungsi sebagai pemicu (*trigger*) untuk menjalankan proses inferensi model EfficientNetV2-S terhadap citra yang telah diunggah.

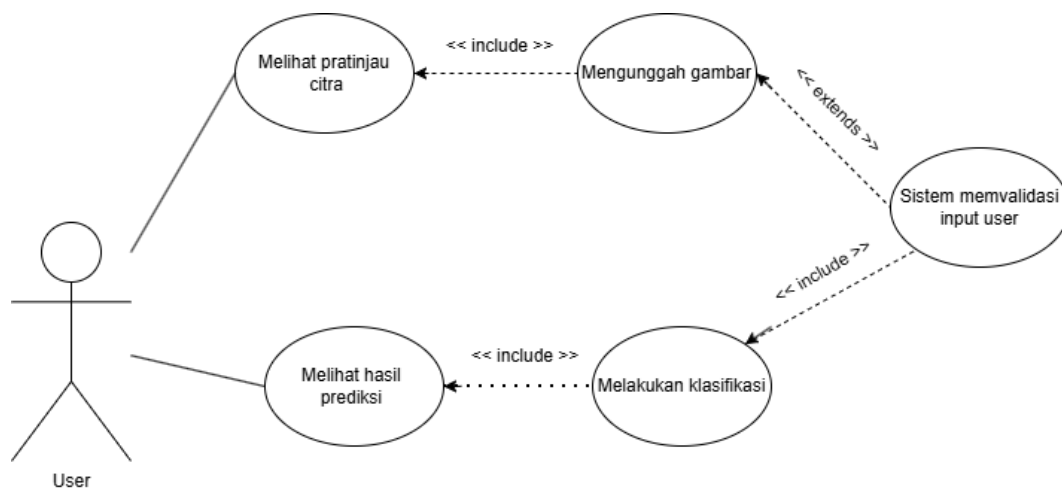
4. *Panel Hasil Prediksi* Area

Berfungsi untuk menampilkan *output* dari model, yang meliputi:

- A. Label Kelas: Jenis tumor yang terdeteksi (Glioma, Meningioma, Pituitary, atau *No tumor*).
- B. Skor Kepercayaan (*Confidence Score*): Persentase probabilitas yang menunjukkan seberapa yakin model terhadap prediksi tersebut.

3.5.2 Use Case Diagram Aplikasi

Perancangan fungsionalitas sistem dimodelkan menggunakan *Use Case Diagram* sebagaimana ditunjukkan pada Gambar 3.2. Diagram ini menggambarkan interaksi antara aktor (pengguna) dengan fitur-fitur utama yang tersedia dalam aplikasi klasifikasi tumor otak. Terdapat satu aktor utama, yaitu User, yang memiliki akses penuh terhadap seluruh fitur sistem tanpa memerlukan proses *login* (autentikasi), mengingat tujuan aplikasi ini sebagai alat bantu diagnosis cepat.



Gambar 3.2 Use Case Diagram

Berdasarkan rancangan pada Gambar 3.2, terdapat empat *use case* utama dengan deskripsi sebagai berikut:

1. Mengunggah gambar: Merupakan fungsi dasar di mana pengguna memasukkan citra MRI ke dalam sistem.
2. Melihat pratinjau citra: Sistem menampilkan visualisasi citra yang telah diunggah untuk diverifikasi oleh pengguna.
3. Sistem memvalidasi input user: Sistem akan mengecek gambar yang diinput pengguna, jika bukan citra otak maka akan ditolak.
4. Melakukan klasifikasi: Pengguna memicu proses inferensi model *Deep Learning* untuk menganalisis citra.
5. Melihat hasil prediksi & *confidence score*: Sistem menyajikan hasil diagnosis berupa label kelas tumor dan tingkat kepercayaan model.

Struktur diagram juga mendefinisikan relasi ketergantungan antar *use case* menggunakan relasi <<include>>, yang bermakna bahwa *use case* asal membutuhkan *use case* tujuan agar dapat dieksekusi.

3.6 Pengujian Fungsionalitas Aplikasi (*Black Box Testing*)

Skenario pengujian dirancang untuk mencakup seluruh alur kerja sistem, termasuk penanganan format file yang valid dan tidak valid, kemampuan pemrosesan *batch* (unggah jamak), serta akurasi tampilan hasil prediksi yang meliputi label kelas dan skor kepercayaan. Selain aspek operasional, skenario pengujian ini juga difokuskan pada validasi fungsionalitas komponen antarmuka dalam merespons hasil dari model EfficientNetV2-S. Pengujian mencakup verifikasi kemunculan label kategori tumor secara tepat, yaitu Glioma, Meningioma, Pituitary, atau *No tumor*, serta tampilan nilai persentase probabilitas sebagai skor kepercayaan model.

Melalui serangkaian pengujian Black Box yang diuraikan pada Tabel 3.6, evaluasi dititikberatkan sepenuhnya pada perspektif pengguna akhir tanpa perlu meninjau kompleksitas kode internal aplikasi. Pendekatan ini sangat esensial untuk menguji tingkat ketahanan dan keandalan sistem antarmuka terhadap berbagai kemungkinan interaksi, termasukantisipasi terhadap potensi kelalaian pengguna seperti pengunggahan format file yang tidak didukung. Khususnya pada skenario validasi input, pengujian ini menjadi pembuktian krusial bagi efektivitas mekanisme filter multilayer yang telah dilatih menggunakan dataset objek umum, guna mencegah model utama memproses citra non-medis. Secara keseluruhan, keberhasilan pemenuhan target fungsionalitas pada skenario pengujian ini akan menjadi indikator utama bahwa sistem yang dikembangkan tidak hanya akurat di atas kertas secara metrik komputasi, tetapi juga interaktif, tahan terhadap anomali masukan, dan siap untuk dioperasikan dalam lingkungan praktis. Rangkuman skenario pengujian fungsionalitas dapat dilihat pada Tabel 3.1.

Tabel 3. 1 Skenario Black Box Testing

No	Fitur	Skenario Pengujian	Masukan (Input)	Hasil yang Diharapkan
1	Widget Pengunggahan (Format Valid)	Mengunggah file citra dengan ekstensi yang didukung	File citra format .jpg, .jpeg, atau .png	File berhasil dimuat dan tidak muncul pesan kesalahan
2	Widget Pengunggahan (Format Invalid)	Mengunggah file non-citra untuk menguji validasi sistem	File dokumen format .pdf, .docx, atau .txt	Sistem menolak file atau menampilkan peringatan "Invalid file format"
3	Mode Unggahan Tunggal (<i>Single Upload</i>)	Mengunggah satu file citra MRI	Satu file citra MRI	Citra berhasil diunggah dan siap untuk diproses secara individual

No	Fitur	Skenario Pengujian	Masukan (Input)	Hasil yang Diharapkan
4	Mode Unggahan Jamak (<i>Multiple Upload</i>)	Mengunggah beberapa file citra sekaligus untuk pemrosesan <i>batch</i>	Memilih 3-5 file citra MRI sekaligus	Seluruh citra berhasil terunggah dan masuk ke dalam antrian proses
5	Area Pratinjau (<i>Image Preview</i>)	Memverifikasi visualisasi citra setelah diunggah	File citra yang telah berhasil diunggah	Citra ditampilkan pada layar utama agar pengguna dapat memverifikasi input
6	Tombol Eksekusi Klasifikasi	Menekan tombol <i>trigger</i> untuk memulai inferensi	Klik tombol "Proses Klasifikasi"	Sistem menampilkan indikator proses (<i>loading</i>) dan menjalankan model EfficientNetV2-S
7	Panel Hasil Prediksi (Label)	Memeriksa tampilan label kelas hasil klasifikasi	Selesainya proses inferensi	Muncul teks label kategori tumor (Glioma/Meningioma/Pituitary/ <i>No tumor</i>)
8	Panel Hasil Prediksi (Confidence Score)	Memeriksa tampilan tingkat keyakinan model	Selesainya proses inferensi	Muncul nilai persentase probabilitas (skor kepercayaan)
9	Validasi Input	Memeriksa apakah gambar input merupakan citra otak	File citra format .jpg, .jpeg, atau .png	Sistem menolak klasifikasi jika gambar yang diinput bukan citra otak

BAB IV

HASIL DAN PEMBAHASAN

Bab ini memaparkan hasil implementasi dan analisis kinerja model EfficientNetV2-S dalam mengklasifikasikan jenis tumor otak melalui serangkaian eksperimen. Pembahasan diawali dengan validasi model pada berbagai skenario dataset dimulai dari replikasi penelitian terdahulu, hingga evaluasi pada dataset gabungan berskala besar yang seluruhnya diuji menggunakan standar *10-Fold Cross Validation*. Rangkaian analisis ini kemudian bermuara pada fase produksi, di mana model dengan performa generalisasi terbaik diintegrasikan ke dalam sistem aplikasi berbasis *web*.

4.1 Lingkungan dan Implementasi Model

Sebelum dilakukan pengujian terhadap berbagai variasi dataset, tahap implementasi model dilakukan secara seragam untuk memastikan konsistensi hasil eksperimen dan validitas perbandingan. Seluruh skenario pengujian dalam penelitian ini, baik pada replikasi dataset *baseline* maupun dataset campuran menggunakan tahapan pra-pemrosesan, arsitektur model, dan konfigurasi *hyperparameter* yang identik. Pendekatan ini bertujuan untuk mengisolasi variabel data sebagai satu-satunya pembeda dalam analisis kinerja. Dengan demikian, setiap metrik evaluasi yang dihasilkan nantinya dapat dipastikan murni berasal dari karakteristik dataset yang diuji, bukan akibat bias dari perbedaan konfigurasi sistem.

4.1.1 Pra-pemrosesan Data (*Preprocessing & Augmentasi*)

Setiap citra masukan mengalami proses transformasi sistematis agar sesuai dengan spesifikasi lapisan input dari arsitektur EfficientNetV2-S. Tahapan ini dimulai dengan pengubahan ukuran seluruh citra menjadi dimensi *pixel*. Selanjutnya, nilai tersebut dinormalisasi menggunakan standar statistik dataset ImageNet, yakni dengan nilai *mean* [0.485, 0.456, 0.406] dan standar deviasi [0.229, 0.224, 0.225]. Penerapan normalisasi ini bertujuan untuk menyelaraskan distribusi data input dengan bobot awal model, yang secara signifikan dapat mempercepat konvergensi model *pre-trained* saat dilatih ulang.

Penerapan transformasi ini dibedakan menjadi dua skema antara data latih dan data validasi. Khusus pada fase pelatihan, dataset diperkaya dengan teknik augmentasi data berupa rotasi acak dan pembalikan horizontal. Strategi ini diterapkan untuk meningkatkan variasi data secara artifisial, sehingga model menjadi lebih tangguh terhadap perubahan orientasi objek sekaligus mencegah terjadinya *overfitting*. Penggunaan fungsi tersebut memungkinkan penggabungan berbagai instruksi modifikasi citra menjadi satu alur eksekusi yang berjalan secara dinamis. Pendekatan ini memastikan model selalu menerima variasi citra yang baru pada setiap iterasi pelatihan tanpa perlu menyimpan atau memodifikasi file dataset mentah aslinya. Implementasi kode transformasi tersebut menggunakan pustaka torchvision sebagai berikut.

script kode program *preprocessing*:

```
# Transformasi untuk Data Latih (dengan Augmentasi)
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),      # Ubah ukuran citra
    transforms.RandomRotation(15),      # Rotasi acak +/- 15
    derajat
    transforms.RandomHorizontalFlip(),  # Flip horizontal acak
    transforms.ToTensor(),              # Konversi ke Tensor
    # Normalisasi standar ImageNet
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225])
])
# Transformasi untuk Data Validasi (Tanpa Augmentasi)
val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225])
])
```

4.1.2 Arsitektur dan Konfigurasi Model

Penelitian ini menggunakan arsitektur EfficientNetV2-S yang telah dilatih sebelumnya pada dataset ImageNet. Strategi *Transfer Learning* diterapkan dengan metode *Partial Freezing*, di mana sebagian besar bobot jaringan dibekukan untuk mempertahankan fitur dasar, sementara dua blok fitur terakhir dibuka untuk proses *fine-tuning*. Selain itu, lapisan klasifikasi (*classifier head*) asli diganti dengan lapisan baru yang disesuaikan untuk klasifikasi 4 kelas tumor otak. Pendekatan *partial freezing* ini dipilih karena sangat efisien dalam meminimalkan jumlah parameter yang harus dilatih ulang, sehingga beban komputasi menjadi jauh lebih ringan. Pembukaan dua blok fitur terakhir tersebut dirancang secara khusus agar model dapat mengekstraksi karakteristik visual tingkat tinggi yang sangat spesifik pada anatomi tumor tanpa merusak hierarki fitur dasar yang telah matang. Sementara itu, pada lapisan klasifikasi yang baru, penyematan fungsi *Dropout* turut

diimplementasikan sebagai bentuk regularisasi proaktif guna mencegah terjadinya *overfitting* saat model mengkalibrasi fitur-fitur medis tersebut. Berikut adalah implementasi modifikasi model tersebut.

script kode program implementasi model:

```
# 1. Memuat Model Pre-trained
model =
efficientnet_v2_s(weights=EfficientNet_V2_S_Weights.DEFAULT)

# 2. Membekukan Seluruh Layer (Freeze)
for param in model.parameters():
    param.requires_grad = False

# 3. Membuka (Unfreeze) 2 Layer Terakhir untuk Fine-Tuning
for param in model.features[-2:].parameters():
    param.requires_grad = True

# 4. Modifikasi Classifier Head (4 Kelas Output)
model.classifier = nn.Sequential(
    nn.Dropout(p=0.2, inplace=True),
    nn.Linear(model.classifier[1].in_features, 4)
)
```

Untuk menjamin konsistensi dan keadilan evaluasi di seluruh skenario eksperimen, konfigurasi *hyperparameter* ditetapkan secara seragam tanpa perubahan. Proses optimisasi bobot model menggunakan algoritma AdamW dengan laju pembelajaran awal sebesar 0.001. Pelatihan dijalankan selama 20 *epoch* dengan ukuran *batch* sebanyak 64 sampel per iterasi, serta menggunakan fungsi kerugian *CrossEntropyLoss* yang sesuai untuk klasifikasi multi-kelas. Untuk menjaga stabilitas konvergensi, diterapkan mekanisme penjadwalan laju pembelajaran (*scheduler*) menggunakan *ReduceLROnPlateau*, yang secara otomatis akan memangkas *learning rate* sebesar 50% (faktor 0.5) apabila nilai

validation loss mengalami stagnasi selama 2 *epoch* berturut-turut. Berikut Implementasi konfigurasi tersebut dalam kode.

script kode program *hyperparameter tuning*:

```
# Konfigurasi Hyperparameter & Training
BATCH_SIZE = 64
EPOCHS = 20

# Setup Loss Function, Optimizer, dan Scheduler
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=0.001,
weight_decay=1e-7)

# Scheduler: Kurangi LR jika val_loss stagnan 2 epoch
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',
    patience=2,
    factor=0.5,
    verbose=True
)
```

4.1.3 Skema Pengujian (*Cross Validation*)

Sebagai upaya mitigasi terhadap potensi bias pembagian data yang sering terjadi pada metode *hold-out* statis, penelitian ini menerapkan skema evaluasi 10-*Fold Stratified Cross Validation*. Dalam skema ini, 100% data digunakan secara bergilir sebagai data latih dan data validasi. Fungsi *StratifiedKFold* digunakan untuk memastikan proporsi kelas di setiap lipatan (*Fold*) tetap seimbang sesuai dengan distribusi data asli. Pada setiap iterasi *Fold*, model terbaik (berdasarkan akurasi validasi tertinggi) disimpan secara otomatis. Mekanisme penyimpanan dinamis yang dikenal sebagai *model checkpointing* ini menjamin bahwa bobot parameter dengan tingkat kesalahan paling minim tidak akan tertimpa, sehingga

siap diekspor untuk kebutuhan pada tahap pengembangan antarmuka akhir.

Implementasi alur pengujian tersebut dapat diringkas dalam potongan kode berikut.

script kode program implementasi *k-Fold*:

```
# Inisialisasi Stratified K-Fold (10 Lipatan)
kFold = StratifiedKFold(n_splits=10, shuffle=True,
random_state=42)

# Loop Evaluasi
for Fold, (train_idx, val_idx) in enumerate(kFold.split(X, y)):
    # ... (Proses Training per Epoch) ...

    # Simpan Model Terbaik (Model Checkpointing)
    if val_acc > best_Fold_acc:
        best_Fold_acc = val_acc
        # Simpan bobot model untuk digunakan pada aplikasi nanti
        torch.save(model.state_dict(),
f'best_model_Fold_{Fold+1}.pth')
```

4.2 Implementasi Pengujian pada Dataset Baseline

Pada bagian ini, dilakukan pengujian kembali terhadap kinerja model menggunakan dataset dari empat penelitian terkait, yaitu (Hamidja et al., 2024), (Azzahra et al., 2023), (Waskita et al., 2023), dan (Wong et al., 2025). Tujuan utama dari eksperimen ini adalah untuk menstandarisasi metode pengujian agar hasil performa dari setiap dataset dapat diperbandingkan secara adil dengan metode dataset campuran yang diusulkan penulis. Sebagai pengganti metode validasi asli dari masing-masing paper, penelitian ini menerapkan skema *10-Fold Cross Validation* secara seragam pada seluruh dataset.

Hal ini dilakukan karena penelitian-penelitian rujukan tersebut menerapkan komposisi pembagian data latih dan uji yang tidak seragam, seperti rasio 70:30, 80:20, bahkan 90:10 dengan metode *Hold-Out*. Ketidakknaan komposisi tersebut berpotensi menghasilkan klaim akurasi yang bias tergantung pada pembagian

datanya. Dengan menggunakan *10-Fold Cross Validation*, seluruh data dipastikan pernah menjadi data uji dan data latih secara bergantian, sehingga hasil rata-rata akurasi yang diperoleh dapat mencerminkan kualitas asli dataset tanpa dipengaruhi oleh keberuntungan pembagian data manual. Hasil dan metode penelitian terdahulu dapat dilihat pada Tabel 4.1.

Tabel 4.1 Hasil dan Metode Penelitian Terdahulu

Peneliti (Tahun)	Objektif	Metode	Akurasi	Dataset
Azzahra, dkk. (2023)	Klasifikasi 4 jenis citra MRI (glioma, meningioma, pituitary, <i>no tumor</i>)	CNN (Inception Resnet v2) dengan pembagian <i>train</i> 70% dan validasi 30%	93,4%	Brain Tumor Classification (MRI) dari kaggle.com
Hamidja, dkk. (2024)	Klasifikasi tumor otak pada dataset publik untuk efisiensi komputasi	CNN (KKDNet) dengan pembagian <i>train</i> 80% dan validasi 20%	>90%	Brain Tumor Classification (MRI) dari kaggle.com
Waskita, dkk. (2023)	Klasifikasi 4 jenis citra MRI (glioma, meningioma, pituitary, normal)	CNN (EfficientNetV2 B0, B1, B2, B3) dengan pembagian <i>train</i> 81%, validasi 9%, dan <i>test</i> 10%	>99%	Crystal Clean: Brain Tumors MRI dari kaggle.com
Wong, dkk. (2023)	Klasifikasi tumor otak menggunakan <i>Deep Learning</i> dengan dataset skala besar	CNN (VGG16 + Augmentasi pada 3 Dataset Publik Gabungan) dengan pembagian <i>train</i> 80%, validasi 10%, dan <i>test</i> 10%	99,24%	Brain tumor dataset (figshare), Br35H: Brain Tumor Detection 2020 (kaggle.com), dan Brain Tumor Classification (MRI) dari kaggle.com

4.2.1 Pengujian pada Dataset *Brain Tumor Classification* (MRI)

Eksperimen pertama difokuskan pada replikasi pengujian dataset berjudul “Brain Tumor Classification (MRI)” yang menjadi basis data utama dalam

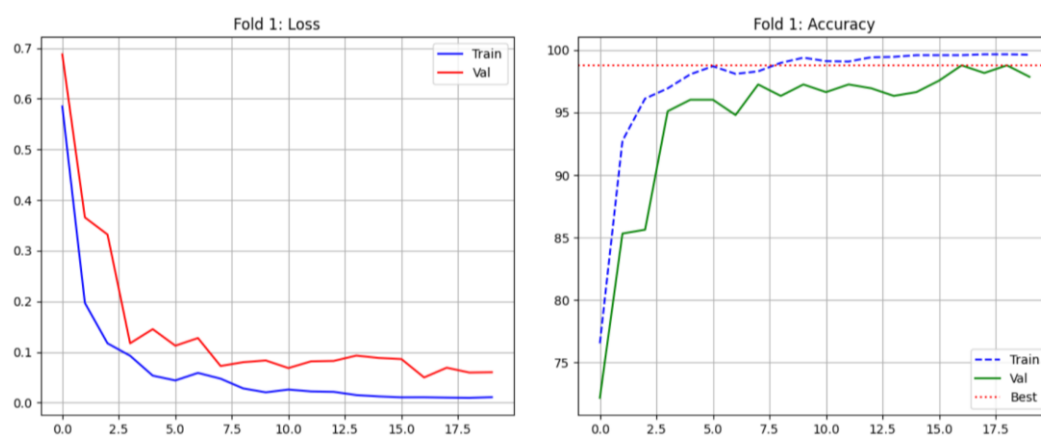
penelitian Hamidja et al., (2024) dan Azzahra et al., (2023). Dataset ini memiliki total 3.264 citra MRI yang terdiri dari empat kelas, yaitu 926 citra glioma tumor, 937 citra meningioma tumor, 901 citra pituitary tumor, dan 500 citra tanpa tumor. Pada penelitian aslinya, (Hamidja et al., 2024) menerapkan skema pembagian data dengan rasio 80:20, sedangkan (Azzahra et al., 2023) menggunakan rasio 70:30. Meskipun kedua penelitian tersebut melaporkan akurasi yang tinggi, penggunaan *single split* manual pada dataset dengan jumlah terbatas ini memiliki risiko bias, di mana model mungkin hanya berkinerja baik pada pembagian data tertentu yang menguntungkan. Oleh karena itu, penelitian ini menggabungkan kembali seluruh data latih dan uji tersebut untuk kemudian diterapkan metode *10-Fold Cross Validation* guna mendapatkan evaluasi yang lebih objektif. Berdasarkan pengujian ulang menggunakan skema *10-Fold CV* tersebut, model EfficientNetV2-S yang diusulkan berhasil mencatatkan performa yang sangat memuaskan. Hasil rekapitulasi pelatihan dari 10 lipatan data yang dihitung berdasarkan rumus akurasi, presisi, *recall*, dan *F1-score* dapat dilihat secara rinci pada Tabel 4.2.

Tabel 4.2 Hasil *Fold* pada Dataset Brain Tumor Classification (MRI)

<i>Fold</i>	<i>Best Epoch</i>	Akurasi	Presisi (Tertimbang)	Recall (Tertimbang)	<i>F1-Score</i> (Tertimbang)
1	17	0,9878	0,9879	0,9878	0,9877
2	18	0,9725	0,9728	0,9725	0,9725
3	12	0,9847	0,9850	0,9847	0,9848
4	13	0,9694	0,9697	0,9694	0,9692
5	15	0,9601	0,9608	0,9601	0,9600
6	15	0,9785	0,9786	0,9785	0,9785
7	15	0,9663	0,9664	0,9663	0,9661
8	9	0,9693	0,9695	0,9693	0,9693
9	13	0,9816	0,9819	0,9816	0,9816
10	15	0,9693	0,9694	0,9693	0,9693
Rata-rata	-	0,9740	0,9742	0,9740	0,739

Evaluasi kinerja model dilakukan menggunakan metode *10-Fold Cross-Validation* untuk memastikan konsistensi dan kemampuan generalisasi model terhadap variasi pembagian data. Tabel 4.2 menyajikan rekapitulasi metrik performa terbaik yang dicapai pada setiap *Fold*, meliputi akurasi, *presisi*, *recall*, dan *F1-Score*. Berdasarkan hasil pengujian keseluruhan, model EfficientNetV2-S menunjukkan performa yang sangat stabil dengan rata-rata akurasi mencapai 97,40% dan rata-rata *tertimbang F1-Score* sebesar 97,39%. Konsistensi nilai metrik antar *Fold* ini mengindikasikan bahwa model mampu mempelajari fitur citra MRI dengan baik tanpa bias yang signifikan terhadap partisi data tertentu.

Di antara seluruh skenario pengujian, *Fold 1* mencatatkan kinerja paling optimal dengan akurasi validasi tertinggi sebesar 98,77%. Oleh karena itu, hasil pelatihan pada *Fold* ini digunakan sebagai representasi untuk menganalisis proses konvergensi model secara lebih mendalam. Gambar 4.1 di bawah ini memperlihatkan grafik pergerakan *Training vs Validation Loss* serta *Akurasi* pada *Fold 1*, yang menunjukkan bahwa model berhasil meminimalkan *loss* dengan stabil seiring bertambahnya *epoch* tanpa indikasi *overfitting* yang ekstrem.



Gambar 4.1 Grafik *Loss* dan *Akurasi* pada *Fold 1*

Gambar 4.1 memvisualisasikan dinamika pelatihan pada *Fold 1* hingga batas maksimum 20 *epoch*. Terlihat bahwa kurva akurasi pelatihan (*training akurasi*) dan validasi (*validation akurasi*) bergerak beriringan dengan tren peningkatan yang stabil. Celah yang sangat tipis antara kedua kurva tersebut mengindikasikan bahwa model memiliki kemampuan generalisasi yang sangat baik dan tidak mengalami *overfitting* yang signifikan terhadap data latih. Meskipun pelatihan dihentikan pada *epoch* ke-20 sesuai batasan parameter yang ditentukan, model telah berhasil mencapai performa yang optimal tanpa menunjukkan tanda-tanda degradasi performa pada data validasi. Rincian performa model pada setiap *epoch* untuk *Fold 1*, yang dihitung berdasarkan rumus akurasi dan *loss*, disajikan secara lengkap pada Tabel 4.3.

Tabel 4.3 Detail *Epoch* pada *Fold 1*

Epoch	Laju Pembelajaran	Loss Pelatihan	Akurasi Pelatihan (%)	Loss Validasi	Akurasi Validasi (%)
1	0,001000	0,5850	76,54	0,6877	72,17
2	0,001000	0,1969	92,75	0,3657	85,32
3	0,001000	0,1172	96,12	0,3320	85,63
4	0,001000	0,0926	96,94	0,1172	95,11
5	0,001000	0,0534	98,06	0,1452	96,02
6	0,001000	0,0439	98,71	0,1125	96,02
7	0,001000	0,0588	98,09	0,1276	94,80
8	0,001000	0,0474	98,30	0,0724	97,25
9	0,001000	0,0282	98,98	0,0799	96,33
10	0,001000	0,0204	99,39	0,0833	97,25
11	0,001000	0,0259	99,11	0,0683	96,64
12	0,001000	0,0222	99,08	0,0816	97,25
13	0,001000	0,0214	99,42	0,0824	96,94
14	0,001000	0,0150	99,46	0,0929	96,33
15	0,000500	0,0123	99,59	0,0883	96,64
16	0,000500	0,0106	99,59	0,0860	97,55
17	0,000500	0,0108	99,59	0,0498	98,78
18	0,000500	0,0100	99,66	0,0690	98,17
19	0,000500	0,0095	99,66	0,0595	98,78
20	0,000500	0,0110	99,63	0,0601	97,86

Analisis lebih mendalam pada kurva *loss* memperlihatkan efektivitas penerapan mekanisme *ReduceLROnPlateau*. Sebelum *epoch* ke-15, penurunan *validation loss* terlihat mulai melambat dan mengalami stagnasi. Sebagai respons terhadap kondisi tersebut, sistem secara otomatis menurunkan *learning rate* dari 0,001 ke angka $\times 10^{-4}$ pada *epoch* ke-15. Penyesuaian ini memberikan dampak langsung yang positif, di mana *validation loss* kembali turun secara signifikan setelahnya dan mencapai *validation akurasi* tertinggi di *epoch* ke-17. Hal ini membuktikan bahwa strategi penurunan *learning rate* adaptif berhasil membantu model untuk melakukan penyesuaian bobot yang lebih halus dan keluar dari titik stagnasi guna mencapai nilai *loss* yang lebih minimal.

Performa tinggi yang dicapai model (rata-rata akurasi validasi yang konsisten tinggi di setiap *Fold*) tidak terlepas dari pemilihan arsitektur dan strategi pelatihan yang diterapkan. Keberhasilan ini didorong oleh tiga faktor teknis utama, yaitu:

1. Efektivitas *Transfer Learning* dan *Partial Freezing*

Penggunaan EfficientNetV2-S memungkinkan model memanfaatkan fitur visual yang telah dipelajari dari dataset ImageNet. Kunci stabilitas model terletak pada strategi *freezing* bobot. Kode berikut menunjukkan bahwa sebagian besar *layer* dibekukan untuk menjaga ekstraksi fitur dasar, sementara hanya dua blok fitur terakhir dan *classifier* yang dilatih ulang agar model beradaptasi spesifik pada citra MRI tumor otak. Strategi ini mencegah *overfitting* yang sering terjadi jika melatih ulang seluruh jaringan pada dataset medis yang jumlahnya terbatas.

2. *Preprocessing* dan Augmentasi Data

Model dapat melakukan generalisasi dengan baik karena input gambar telah dinormalisasi sesuai ukuran dataset ImageNet dan menggunakan statistik (*mean* dan *std*) yang sama dengan data pelatihan awal EfficientNet. Selain itu, augmentasi berupa rotasi acak membantu model mengenali tumor dari berbagai orientasi spasial.

3. Adaptasi *Learning Rate* (*Scheduler*)

Penurunan *Loss* yang signifikan dan lonjakan akurasi (seperti terlihat pada data *epoch* 15 sebelumnya) terjadi berkat mekanisme ReduceLROnPlateau. Ketika model mulai kesulitan menurunkan *loss*, *learning rate* diturunkan secara otomatis agar model dapat mencapai *global minima* dengan lebih presisi.

skema *10-Fold Cross-Validation* ini memberikan evaluasi performa yang lebih komprehensif dibandingkan penelitian terdahulu oleh (Hamidja et al., 2024) dan (Azzahra et al., 2023). Jika penelitian sebelumnya menerapkan metode pemisahan data statis yang memiliki keterbatasan dalam mengukur konsistensi model pada variasi data, hasil penelitian ini membuktikan bahwa model EfficientNetV2-S mampu mempertahankan performa yang stabil (*robust*) di seluruh lipatan pengujian. Stabilitas nilai metrik antar *Fold* ini mengonfirmasi bahwa model yang diusulkan memiliki kemampuan generalisasi yang lebih teruji, sehingga melengkapi kekurangan evaluasi pada studi sebelumnya dan menawarkan tingkat kepercayaan yang lebih tinggi untuk implementasi klinis.

4.2.2 Pengujian pada Dataset *Crystal Clean*

Eksperimen kedua difokuskan pada replikasi pengujian dataset berjudul “Crystal Clean: Brain Tumors MRI Dataset” yang menjadi sumber data dalam

penelitian (Waskita et al., 2023). Berbeda dengan penelitian referensi yang menggunakan metode *hold-out* dengan rasio 81:9:10, eksperimen ini menerapkan standarisasi evaluasi menggunakan *10-Fold Cross Validation* serupa dengan skenario pertama. Penerapan metode yang seragam ini bertujuan untuk memvalidasi ketangguhan model pada distribusi data yang berbeda secara objektif.

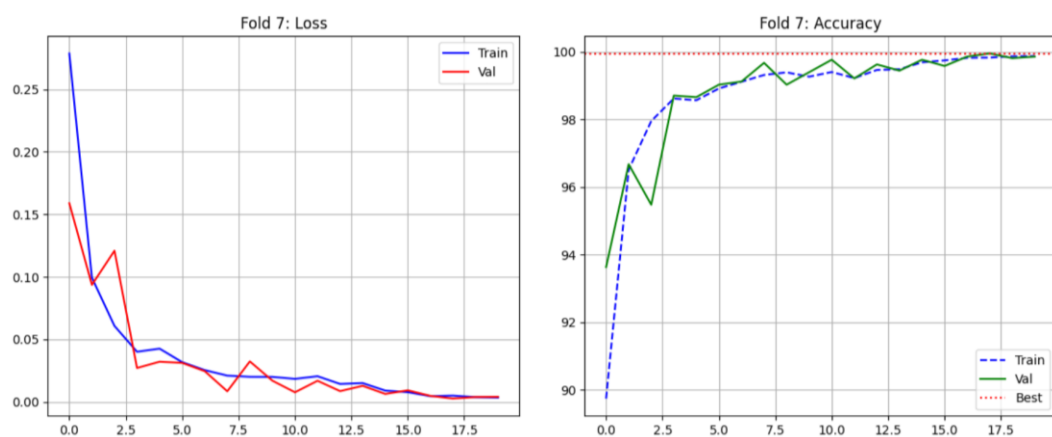
Dataset ini memiliki volume data yang signifikan dengan total akumulasi sebanyak 21.672 citra MRI. Distribusi kelas pada dataset ini terdiri dari 6.307 citra Glioma, 6.391 citra Meningioma, dan 5.908 citra Pituitary, serta dilengkapi dengan 3.066 citra kondisi Normal. Kuantitas data yang besar dan terkurasi ini memberikan ruang eksplorasi yang luas bagi model untuk mempelajari fitur karakteristik tumor secara mendalam. Tabel 4.4 menyajikan rekapitulasi hasil pelatihan dari 10 lipatan data yang dihitung berdasarkan rumus pada Bab 2, yang mencakup informasi mengenai metrik evaluasi berupa akurasi, presisi, *recall*, dan *F1-score*.

Tabel 4.4 Hasil *Fold* pada Dataset Crystal Clean

<i>Fold</i>	<i>Best Epoch</i>	Akurasi	Presisi (Tertimbang)	Recall (Tertimbang)	<i>F1-Score</i> (Tertimbang)
1	18	0,9986	0,9986	0,9986	0,9986
2	19	0,9982	0,9982	0,9982	0,9982
3	18	0,9968	0,9968	0,9968	0,9968
4	18	0,9982	0,9982	0,9982	0,9982
5	15	0,9972	0,9972	0,9972	0,9972
6	17	0,9995	0,9995	0,9995	0,9995
7	18	0,9995	0,9995	0,9995	0,9995
8	19	0,9968	0, 9968	0, 9968	0, 9968
9	20	0,9982	0, 9982	0, 9982	0, 9982
10	17	0,9968	0, 9968	0, 9968	0, 9968
Rata-rata	-	0,9980	0,9980	0,9980	0,9980

Hasil rekapitulasi evaluasi menggunakan *10-Fold Cross Validation* pada Tabel 4.4 menunjukkan performa yang sangat impresif dan konsisten di seluruh

lipatan pengujian. Model EfficientNetV2-S berhasil mencapai rata-rata akurasi akhir sebesar 99,80%, dengan variasi kinerja yang sangat minim antar-*Fold*. Akurasi terendah tercatat pada angka 99,68% (*Fold* 3, 8, dan 10), sedangkan akurasi tertinggi mencapai 99,95% (*Fold* 7). Tingginya nilai rata-rata *Tertimbang Presisi*, *Recall*, dan *F1-Score* yang identik dengan akurasi (99,95%) mengindikasikan bahwa model mampu mengklasifikasikan seluruh kelas tumor dengan sangat presisi tanpa terpengaruh oleh isu ketidakseimbangan kelas. Hasil ini tidak hanya memvalidasi klaim akurasi tinggi (>99%) yang dilaporkan oleh (Waskita et al., 2023) mungkin untuk dicapai, tetapi juga membuktikan bahwa arsitektur yang diusulkan memiliki stabilitas yang sangat baik (*robust*) meskipun diuji dengan rotasi data validasi yang ketat. Gambar 4.2 di bawah ini memvisualisasikan pergerakan kurva *Loss* dan *Akurasi* selama proses pelatihan berlangsung pada lipatan terbaik yaitu *Fold* 7.



Gambar 4.2 Grafik *Loss* dan *Akurasi* pada *Fold* 7

Berdasarkan grafik di atas, dapat diamati perilaku konvergensi model yang sangat baik. Pada fase awal pelatihan (*epoch* 0 hingga 5), terlihat adanya celah yang

cukup lebar antara kurva pelatihan (*training*) dan validasi (*validation*), baik pada metrik *loss* maupun *akurasi*. Kondisi ini wajar terjadi karena model baru mulai mempelajari pola fitur dari data latih. Namun, seiring bertambahnya *epoch*, celah tersebut secara konsisten semakin mengecil dan menyempit.

Memasuki pertengahan hingga akhir pelatihan (di atas *epoch* ke-10), kurva *training* dan *validation* terlihat bergerak beriringan dengan jarak yang sangat tipis, bahkan hampir berimpit pada *epoch* puncak. Fenomena menyempitnya *gap* ini mengindikasikan bahwa model memiliki kemampuan generalisasi yang kuat; model mampu mempelajari data latih dengan baik tanpa kehilangan kemampuannya untuk memprediksi data validasi yang belum pernah dilihat sebelumnya. Tidak adanya divergensi antara kedua kurva di akhir pelatihan juga menjadi bukti kuat bahwa model tidak mengalami *overfitting*. Detail perubahan nilai *loss* dan akurasi pada setiap *epoch* selama proses pelatihan di lipatan terbaik (Fold 7), yang perhitungannya mengacu pada rumus di Bab 2, disajikan secara rinci pada Tabel 4.5.

Tabel 4.5 Detail *Epoch* pada *Fold* 7

Epoch	Laju Pembelajaran	Loss Pelatihan	Akurasi Pelatihan (%)	Loss Validasi	Akurasi Validasi (%)
1	0,001000	0,2786	89,74	0,1589	93,63
2	0,001000	0,0994	96,51	0,0937	96,68
3	0,001000	0,0608	97,95	0,1210	95,48
4	0,001000	0,0402	98,62	0,0272	98,71
5	0,001000	0,0427	98,57	0,0322	98,66
6	0,001000	0,0319	98,92	0,0313	99,03
7	0,001000	0,0255	99,12	0,0247	99,12
8	0,001000	0,0211	99,32	0,0086	99,68
9	0,001000	0,0201	99,40	0,0325	99,03
10	0,001000	0,0201	99,27	0,0170	99,40
11	0,001000	0,0185	99,41	0,0078	99,77
12	0,001000	0,0206	99,23	0,0170	99,22

Epoch	Laju Pembelajaran	Loss Pelatihan	Akurasi Pelatihan (%)	Loss Validasi	Akurasi Validasi (%)
13	0,001000	0,0144	99,47	0,0087	99,63
14	0,001000	0,0151	99,49	0,0130	99,45
15	0,000500	0,0091	99,69	0,0065	99,77
16	0,000500	0,0079	99,75	0,0093	99,58
17	0,000500	0,0047	99,83	0,0049	99,86
18	0,000500	0,0051	99,83	0,0027	99,95
19	0,000500	0,0039	99,87	0,0040	99,82
20	0,000500	0,0036	99,88	0,0041	99,86

Berdasarkan data numerik pada Tabel 4.5, terlihat bahwa model mampu belajar dengan sangat cepat pada fase awal, di mana akurasi validasi melonjak dari 93,63% pada *epoch* pertama menjadi 99,68% hanya dalam delapan *epoch*. Namun, memasuki pertengahan proses pelatihan (*epoch* 12 hingga 14), penurunan *validation loss* mulai melambat dan menunjukkan tanda-tanda saturasi. Pada titik inilah mekanisme *ReduceLROnPlateau* kembali membuktikan efektivitasnya.

Sistem mendeteksi stagnasi tersebut dan secara otomatis menurunkan *learning rate* dari 0,001 menjadi 0,0005 pada *epoch* ke-15. Intervensi ini memberikan dampak instan yang sangat positif; model berhasil keluar dari *local minima* dan *validation loss* turun drastis dari 0,0130 (*epoch* 14) menjadi 0,0065 (*epoch* 15). Momentum perbaikan ini berlanjut hingga model mencapai performa puncaknya pada *epoch* ke-18, dengan akurasi validasi nyaris sempurna sebesar 99,95% dan *loss* terendah di angka 0,0027. Hal ini mengonfirmasi bahwa strategi penurunan laju pembelajaran adaptif sangat krusial dalam membantu model menyempurnakan bobotnya (*fine-tuning*) untuk mencapai konvergensi yang optimal.

Secara komparatif, hasil eksperimen ini memiliki implikasi penting terhadap temuan (Waskita et al., 2023). Penelitian referensi tersebut melaporkan akurasi di atas 99% menggunakan skema pembagian data statis (*hold-out*) 81:9:10. Meskipun angka tersebut sangat tinggi, metode *single split* memiliki celah validitas karena performa model bisa saja bias terhadap partisi data tertentu yang mudah diprediksi.

Penelitian ini berhasil memverifikasi dan memperkuat klaim tersebut melalui pengujian yang jauh lebih ketat. Dengan menerapkan *10-Fold Cross Validation*, di mana model diuji pada 100% variasi data secara bergilir, rata-rata akurasi yang diperoleh tetap mencapai 99,80% dengan variasi antar-*Fold* yang sangat kecil ($\pm 0,10\%$). Hal ini membuktikan bahwa arsitektur EfficientNetV2-S pada dataset ini memang memiliki kualitas fitur yang sangat *distinctive* (mudah dibedakan) dan performanya yang superior bersifat konsisten (*robust*), bukan sekadar hasil dari keberuntungan pembagian data semata.

4.2.3 Pengujian pada 3 Dataset Penelitian Wong

Skenario pengujian ketiga difokuskan pada replikasi studi yang dilakukan oleh Wong, dkk. (2023). Berbeda dengan dua eksperimen sebelumnya yang hanya menggunakan dataset tunggal, penelitian ini mengusulkan penggabungan tiga dataset publik populer, yaitu Figshare, Br35H, dan Brain Tumor Classification (Sartaj). Integrasi ketiga sumber tersebut menghasilkan korpus data berskala besar (*large scale dataset*) dengan total akumulasi sebanyak 6.902 citra MRI. Rincian distribusi persebaran citra dari masing-masing sumber data untuk setiap kelas tumor dapat dilihat pada Tabel 4.6.

Tabel 4.6 Detail Sumber Datasheet

No	Kelas Tumor	Sumber: Figshare	Sumber: Br35H	Sumber: Sartaj	Total Sampel
1	Glioma	1.426	0	0	1.426
2	Meningioma	708	0	937	1.645
3	Pituitary	930	0	901	1.831
4	No tumor	0	1.500	500	2.000

Volume data yang banyak serta tingginya variasi karakteristik citra (heterogenitas) akibat perbedaan sumber akuisisi menjadi tantangan utama pada skenario ini. Pada penelitian aslinya, Wong melaporkan akurasi sebesar 99,24% menggunakan arsitektur VGG16 dengan skema pembagian data 80:10:10. Dalam penelitian ini, skema evaluasi distandarisasi menggunakan *10-Fold Cross Validation* untuk menguji apakah model EfficientNetV2-S mampu menangani kompleksitas data gabungan tersebut dengan lebih baik dan stabil.

Meskipun dihadapkan pada tantangan perbedaan karakteristik data yang cukup nyata antar-sumber, hasil pengujian menunjukkan bahwa arsitektur EfficientNetV2-S mampu beradaptasi dengan sangat baik. Penerapan *10-Fold Cross Validation* pada total 6.902 citra membuktikan kestabilan model, di mana kinerja klasifikasi tetap konsisten tinggi tanpa mengalami penurunan yang drastis pada *Fold* tertentu. Hal ini mengindikasikan bahwa model berhasil mempelajari ciri-ciri utama (fitur) yang konsisten, terlepas dari perbedaan alat MRI maupun teknik pengambilan gambar yang digunakan oleh masing-masing sumber data (Figshare, Br35H, dan Sartaj). Dengan mengacu pada rumus perhitungan yang telah dijabarkan pada Bab 2, rekapitulasi performa kuantitatif yang meliputi metrik akurasi, presisi, *recall*, dan *F1-score* dari setiap iterasi pengujian dapat dilihat secara rinci pada Tabel 4.7.

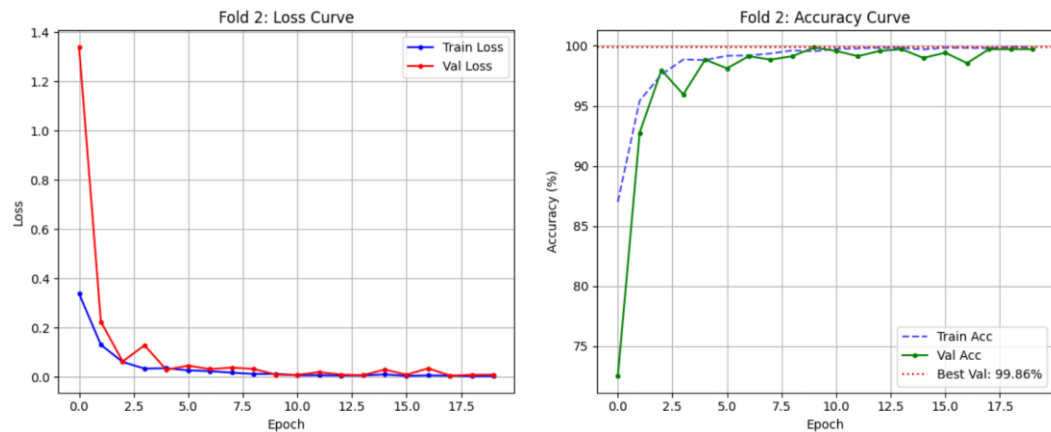
Tabel 4.7 Hasil 10 *Fold* pada Dataset Wong

<i>Fold</i>	<i>Best Epoch</i>	<i>Akurasi</i>	<i>Presisi (Tertimbang)</i>	<i>Recall (Tertimbang)</i>	<i>F1-Score (Tertimbang)</i>
1	18	0,9928	0, 9928	0, 9928	0, 9928
2	10	0,9986	0,9986	0,9986	0,9986
3	12	0,9971	0, 9971	0, 9971	0, 9971
4	18	0,9913	0, 9913	0, 9913	0, 9913
5	17	0,9971	0, 9971	0, 9971	0, 9971
6	10	0,9984	0,9985	0,9984	0,9984
7	15	0,9928	0,9927	0,9928	0,9927
8	8	0,9913	0, 9913	0, 9913	0, 9913
9	15	0,9957	0, 9957	0, 9957	0, 9956
10	13	0,9942	0, 9942	0, 9942	0, 9942
Rata-rata	-	0,9949	0, 9949	0, 9949	0,9949

Berdasarkan data kuantitatif yang tersaji pada Tabel 4.7, model EfficientNetV2-S terbukti mampu menangani kompleksitas dataset gabungan berskala besar dengan sangat impresif. Rata-rata akurasi validasi yang diperoleh dari 10 kali pengujian mencapai angka 99,49%. Hasil ini tidak hanya menunjukkan konsistensi yang tinggi, tetapi juga berhasil melampaui capaian akurasi *baseline* sebesar 99,24% yang dilaporkan oleh (Wong et al., 2023) dalam penelitian aslinya.

Hal yang paling menarik perhatian adalah stabilitas performa antar *Fold*. Meskipun dataset terdiri dari gabungan tiga sumber berbeda (Figshare, Br35H, Sartaj) yang memiliki variasi karakteristik citra, model mampu mempertahankan akurasi di atas ambang 99% pada seluruh lipatan pengujian. Akurasi terendah tercatat pada *Fold* 4 dan 8 (99,13%), sedangkan performa terbaik dicapai pada *Fold* 2 dengan akurasi mencapai 99,86%. Minimnya deviasi kinerja ini menegaskan bahwa model EfficientNetV2-S sangat tangguh (*robust*) dan tidak bias terhadap sumber data tertentu, melainkan sukses mempelajari fitur-fitur general tumor otak

yang valid secara lintas-dataset. Gambar 4.3 berikut menunjukkan grafik *loss* dan akurasi pada *Fold 2*.



Gambar 4.3 Grafik *Loss* dan *Akurasi* pada *Fold 2*

Grafik di atas merepresentasikan dinamika pembelajaran model pada *Fold 2*, yang mencatatkan performa terbaik dalam eksperimen ini. Pada grafik sebelah kiri (*Loss Curve*), terlihat penurunan nilai kerugian yang sangat tajam pada fase awal pelatihan (*Epoch 0* hingga 3). Secara spesifik, *validation loss* turun drastis dari angka di atas 1,3 pada *epoch* awal menjadi mendekati 0,1 hanya dalam waktu kurang dari 5 *epoch*. Penurunan yang curam ini mengindikasikan bahwa bobot *pre-trained* dari EfficientNetV2-S sangat efektif dalam mengenali fitur dasar tumor otak pada dataset gabungan, sehingga proses adaptasi (*transfer learning*) berlangsung sangat efisien.

Fenomena serupa juga teramati pada grafik akurasi (*Akurasi Curve*). Akurasi validasi (garis hijau) melonjak dari sekitar 72% pada *epoch 0* menjadi stabil di atas 95% mulai *epoch* ke-3. Memasuki pertengahan pelatihan (sekitar *epoch* ke-10), kurva pelatihan (biru) dan validasi (hijau) bergerak beriringan dengan jarak (*gap*) yang sangat tipis dan saling berimpit. Kedekatan kedua garis ini merupakan

indikator kuat bahwa model memiliki kemampuan generalisasi yang optimal (*good fit*) dan tidak mengalami *overfitting* maupun *underfitting*.

Puncaknya, model mencapai akurasi validasi tertinggi (99,86%) dengan stabilitas yang terjaga hingga akhir *epoch* ke-20. Hal ini sekaligus membuktikan bahwa variasi karakteristik citra yang berasal dari tiga sumber data berbeda (heterogenitas) tidak menghalangi model untuk mencapai konvergensi yang mulus dan stabil. Hasil rekapitulasi pelatihan dari 10 lipatan data, yang dihitung berdasarkan rumus metrik evaluasi yang telah dijelaskan pada Bab 2, dapat dilihat secara rinci pada Tabel 4.8.

Tabel 4.8 Detail *Epoch* pada *Fold* 2

Epoch	Laju Pembelajaran	Loss Pelatihan	Akurasi Pelatihan (%)	Loss Validasi	Akurasi Validasi (%)
1	0,001000	0,3382	87,01	1,3401	72,50
2	0,001000	0,1305	95,43	0,2242	92,76
3	0,001000	0,0609	97,60	0,0631	97,97
4	0,001000	0,0339	98,86	0,1286	95,95
5	0,001000	0,0356	98,81	0,0293	98,84
6	0,001000	0,0269	99,16	0,0461	98,12
7	0,001000	0,0236	99,19	0,0319	99,13
8	0,001000	0,0174	99,36	0,0379	98,84
9	0,000500	0,0124	99,61	0,0327	99,13
10	0,000500	0,0123	99,53	0,0098	99,86
11	0,000500	0,0067	99,74	0,0084	99,57
12	0,000500	0,0073	99,77	0,0197	99,13
13	0,000500	0,0058	99,84	0,0097	99,57
14	0,000500	0,0063	99,82	0,0070	99,71
15	0,000500	0,0105	99,69	0,0300	98,99
16	0,000500	0,0047	99,84	0,0092	99,42
17	0,000500	0,0066	99,82	0,0353	98,55
18	0,000250	0,0052	99,79	0,0052	99,71
19	0,000250	0,0032	99,90	0,0088	99,71
20	0,000250	0,0038	99,86	0,0094	99,71

Berdasarkan rincian data pada Tabel 4.8, model menunjukkan kemampuan adaptasi yang sangat agresif pada fase awal pelatihan, meskipun dihadapkan pada

tantangan heterogenitas data dari tiga sumber berbeda. Hal ini terindikasi dari lonjakan akurasi validasi yang signifikan, bergerak dari 72,50% pada *epoch* pertama hingga menembus 97,97% hanya pada *epoch* ketiga. Penurunan drastis *Validation Loss* dari 1,3401 menjadi 0,0631 dalam interval singkat tersebut menegaskan bahwa fitur-fitur *pre-trained* pada arsitektur EfficientNetV2-S sangat efektif dalam mengekstraksi representasi visual tumor otak yang kompleks. Momentum pembelajaran ini kemudian dioptimalkan oleh mekanisme *scheduler* (ReduceLROnPlateau) yang mendeteksi perlambatan konvergensi pada pertengahan pelatihan. Sistem secara otomatis menurunkan *learning rate* menjadi 0,0005 pada *epoch* ke-9, yang terbukti menjadi keputusan krusial dalam menyempurnakan bobot model.

Respons model terhadap penyesuaian laju pembelajaran tersebut terlihat instan dan sangat positif. Tepat satu *epoch* setelah penurunan *learning rate* (*epoch* ke-10), model langsung mencatatkan performa puncaknya dengan akurasi validasi tertinggi sebesar 99,86% dan *loss* yang sangat rendah di angka 0,0098. Stabilitas ini terus terjaga hingga akhir proses pelatihan, bahkan ketika *learning rate* kembali diturunkan menjadi 0,00025 pada *epoch* ke-18. Pada tahap akhir ini, model berhasil meminimalkan *validation loss* hingga titik terendah absolutnya (0,0052) dengan akurasi yang tetap stabil di 99,71%. Konsistensi kinerja antara kurva pelatihan dan validasi ini menjadi bukti kuat bahwa model berhasil melakukan generalisasi fitur secara optimal tanpa terjebak dalam kondisi *overfitting*, meskipun dilatih menggunakan dataset gabungan yang masif dan bervariasi.

Secara keseluruhan, hasil eksperimen ini menunjukkan superioritas kinerja arsitektur yang diusulkan dibandingkan dengan penelitian *baseline* oleh (Wong et al., 2023). Meskipun kedua penelitian mengakuisisi data dari tiga sumber publik yang sama (Figshare, Br35H, dan Sartaj), terdapat perbedaan komposisi jumlah sampel pada distribusi kelas yang digunakan. Meskipun terdapat variasi dalam distribusi data tersebut, model EfficientNetV2-S yang dievaluasi menggunakan skema ketat *10-Fold Cross Validation* mampu menghasilkan rata-rata akurasi 99,49% dengan capaian tertinggi 99,86%. Hasil ini melampaui akurasi 99,24% yang dilaporkan dalam studi terdahulu yang menggunakan arsitektur VGG16 dengan skema *single-split*. Hal ini membuktikan bahwa model yang diusulkan memiliki tingkat generalisasi yang tinggi dan mampu beradaptasi dengan baik terhadap variasi komposisi dataset tanpa mengalami penurunan performa.

4.3 Fase Produksi Implementasi Sistem

Untuk memaksimalkan kinerja generalisasi sistem pada lingkungan produksi, dilakukan penggabungan seluruh himpunan data yang tersedia dalam penelitian ini. Dataset final ini merupakan agregasi dari dataset campuran (Wong et al., 2023) yang terdiri dari sumber Figshare, Br35H, dan Sartaj ditambah dengan dataset *Crystal Clean* (Waskita et al., 2023). Integrasi dari keempat sumber data tersebut menghasilkan korpus data terbesar dalam penelitian ini dengan total akumulasi sebanyak 29.500 citra MRI. Rincian data yang digunakan dapat dilihat pada Tabel 4.9.

Tabel 4.9 Distribusi Kelas dan Dataset

Kelas Tumor	Sumber: Figshare	Sumber: Br35H	Sumber: Sartaj	Sumber: Crystal Clean	Total Sampel
Glioma	1.426	0	926	6.307	8.659
Meningioma	708	0	937	6.391	8.036
Pituitary	930	0	901	5.908	7.739
No tumor	0	1.500	500	3.066	5.066
Total	3.064	1.500	3.264	21.672	29.500

Sebagaimana dirincikan pada Tabel 4.9, dataset *Crystal Clean* memberikan kontribusi mayoritas dengan 21.672 citra, yang berperan sebagai basis data berkualitas tinggi (*curated*). Sementara itu, penambahan data dari Figshare, Br35H, dan Sartaj memberikan variasi *noise* dan karakteristik citra yang berbeda. Strategi ini diambil untuk menciptakan model yang tidak hanya akurat, tetapi juga memiliki tingkat ketangguhan (*robustness*) yang tinggi terhadap berbagai variasi input yang mungkin ditemui saat aplikasi digunakan oleh pengguna akhir.

4.3.1 Implementasi Model pada Dataset Campuran (4 Dataset)

Evaluasi kinerja model pada dataset skala masif ini kembali dilakukan menggunakan metode *10-Fold Cross Validation*. Penggunaan validasi silang pada tahap ini bertujuan untuk memastikan bahwa model yang akan ditanamkan ke dalam sistem benar-benar memiliki performa yang konsisten dan tidak bias terhadap subset data tertentu. Rekapitulasi hasil pengujian kuantitatif pada dataset gabungan total berupa akurasi, presisi, recall, dan *F1-Score* yang dihitung berdasarkan rumus pada Bab 2, dapat dilihat secara rinci pada Tabel 4.10.

Tabel 4.10 Hasil 10 *Fold* pada Dataset Campuran

<i>Fold</i>	Akurasi	Presisi (Tertimbang)	Recall (Tertimbang)	<i>F1-Score</i> (Tertimbang)
1	0,9980	0,9980	0,9980	0,9980

<i>Fold</i>	<i>Akurasi</i>	<i>Presisi (Tertimbang)</i>	<i>Recall (Tertimbang)</i>	<i>F1-Score (Tertimbang)</i>
2	0,9973	0,9973	0,9973	0,9973
3	0,9973	0,9973	0,9973	0,9973
4	0,9966	0,9966	0,9966	0,9966
5	0,9976	0,9976	0,9976	0,9976
6	0,9986	0,9986	0,9986	0,9986
7	0,9986	0,9986	0,9986	0,9986
8	0,9990	0,9990	0,9990	0,9990
9	0,9973	0,9973	0,9973	0,9973
10	0,9976	0,9976	0,9976	0,9976
Rata-rata	0,9978	0,9978	0,9978	0,9978

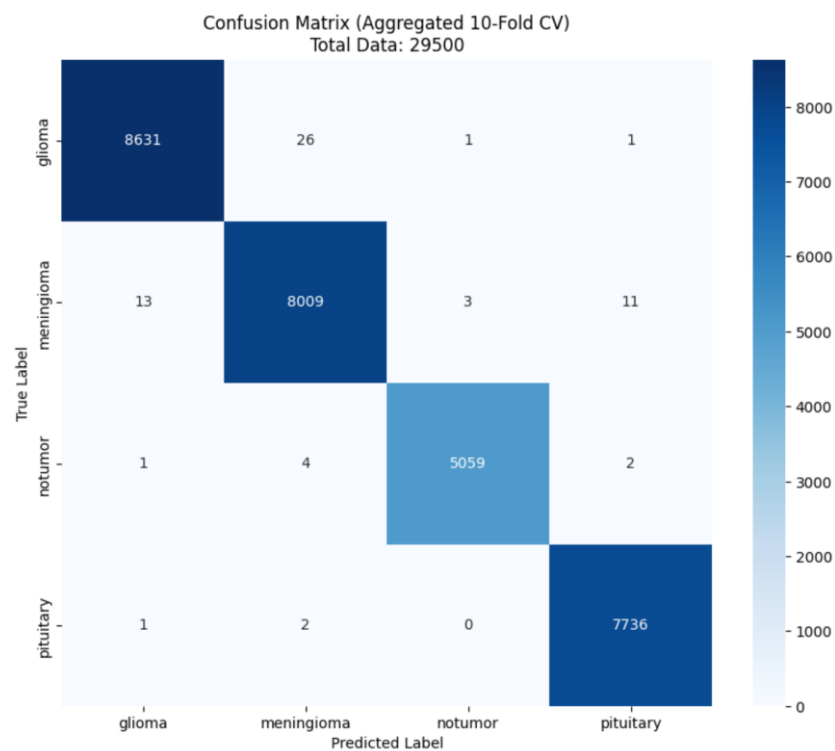
Berdasarkan Tabel 4.10, model EfficientNetV2-S menunjukkan performa yang sangat impresif dengan rata-rata akurasi mencapai 99,78%. Angka ini merefleksikan kemampuan model dalam menyerap pola fitur dari 29.500 citra dengan sangat baik tanpa mengalami degradasi kinerja akibat heterogenitas sumber data. Selisih antara akurasi terendah (*Fold* 4: 99,66%) dan tertinggi (*Fold* 8: 99,90%) hanya sebesar 0,24%, membuktikan bahwa model memiliki stabilitas tinggi dan siap untuk diimplementasikan pada lingkungan produksi.

Mengacu pada hasil tersebut, bobot model dari *Fold* 8 dipilih sebagai model final untuk aplikasi *web*. Keputusan ini didasarkan pada capaian akurasi tertinggi sebesar 99,90% serta nilai presisi dan *recall* yang seimbang, yang mengindikasikan bahwa model pada iterasi ini memiliki kemampuan prediksi paling optimal dibandingkan iterasi lainnya. *File* model kemudian diekspor ke dalam format *.pth* untuk selanjutnya diintegrasikan dengan antarmuka pengguna pada tahap berikutnya.

Keberhasilan model pada *Fold* 8 tersebut merepresentasikan titik optimal dari seluruh rangkaian pengujian yang telah dilakukan. Namun, untuk memastikan

bahwa keunggulan ini tidak bersifat sporadis pada satu partisi data saja, diperlukan peninjauan terhadap performa model secara kolektif. Evaluasi secara menyeluruh terhadap seluruh iterasi pengujian akan memberikan kepastian mengenai objektivitas model dalam menangani data citra yang lebih luas. Hal ini penting dilakukan guna memetakan apakah terdapat pola kesalahan klasifikasi yang spesifik pada kelas tertentu sebelum model benar-benar diintegrasikan ke dalam sistem.

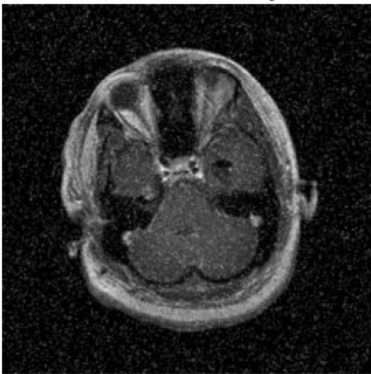
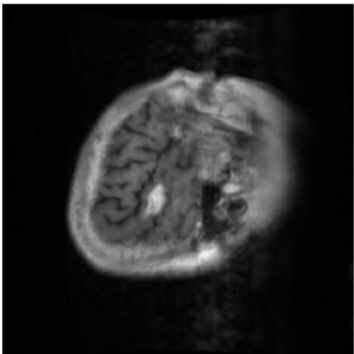
Untuk memverifikasi ketangguhan model secara komprehensif, dilakukan analisis kesalahan klasifikasi menggunakan *Aggregated Confusion Matrix* yang merangkum hasil prediksi dari seluruh 10 *Fold*. Matriks ini merepresentasikan evaluasi terhadap total 29.500 citra, memberikan gambaran utuh mengenai performa model di seluruh variasi data. Visualisasi matriks konfusi tersebut disajikan pada Gambar 4.4.

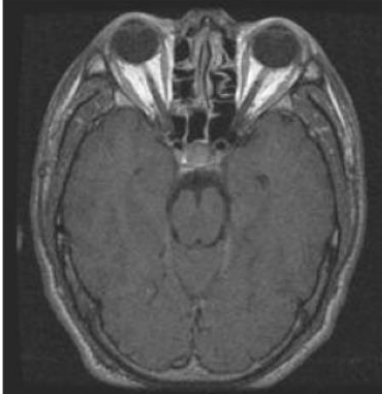
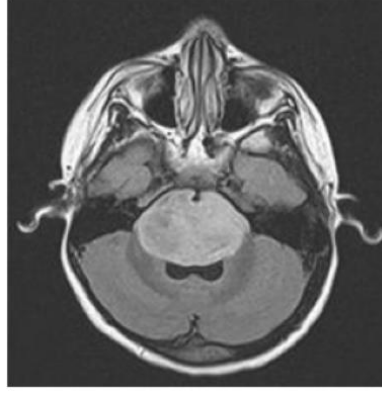
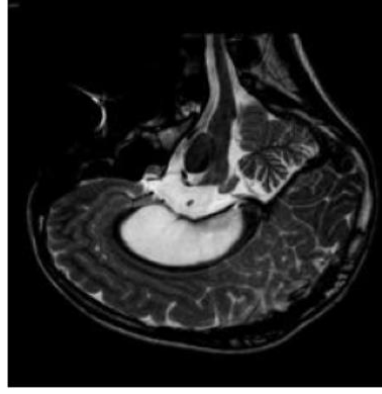


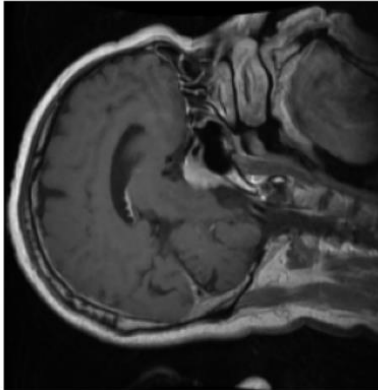
Gambar 4.4 *Confusion Matrix* pada Dataset Campuran

Berdasarkan Gambar 4.4, terlihat dominasi warna biru pekat pada garis diagonal utama yang menandakan tingginya jumlah prediksi benar (*True Positives*). Dari total 29.500 data uji, kesalahan klasifikasi terjadi dalam jumlah yang sangat insignifikan. Sebagai contoh pada kelas Glioma, model berhasil memprediksi 8.631 sampel dengan benar dan hanya meleset pada 28 citra. Pola serupa terlihat pada kelas Meningioma (8.009 benar), Pituitary (7.736 benar), dan *No tumor* (5.059 benar). Minimnya angka di luar diagonal utama (kesalahan prediksi) menegaskan bahwa model EfficientNetV2-S yang dikembangkan memiliki tingkat presisi yang sangat tinggi. Meskipun jumlah kesalahan tersebut sangat kecil secara persentase, peninjauan terhadap sampel citra yang gagal diprediksi tetap krusial untuk dilakukan guna memahami karakteristik visual yang membingungkan model. Oleh karena itu, dilakukan analisis visual secara spesifik terhadap beberapa representasi citra *misclassification*. Sampel citra yang dianalisis pada sub-bab ini diambil dari hasil prediksi menggunakan model terbaik yang merupakan model final untuk diimplementasikan pada prototipe antarmuka sistem. Investigasi terhadap anomali prediksi ini tidak hanya memetakan batas kemampuan model dalam membedakan fitur yang saling tumpang tindih, tetapi juga secara kritis dijadikan instrumen untuk mengidentifikasi potensi kecacatan bawaan dari dataset publik itu sendiri. Transparansi dalam membedah kegagalan klasifikasi ini menjadi langkah esensial untuk menjamin objektivitas evaluasi sebelum sistem dioperasikan pada ranah praktis. Tabel 4.11 berikut menyajikan representasi sampel citra yang mengalami kesalahan prediksi beserta analisis kemungkinan penyebabnya berdasarkan kemiripan fitur.

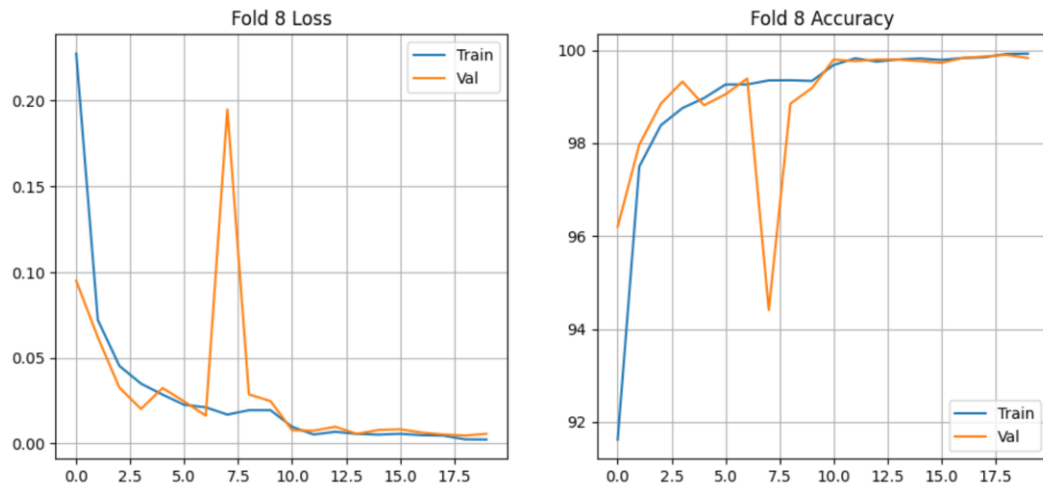
Tabel 4.11 Sampel Kesalahan Prediksi Model

No	Gambar	Analisis
1	Asli: No Tumor Prediksi: Glioma 	Kesalahan ini kemungkinan besar dipicu oleh adanya variasi intensitas piksel. Dalam kacamata <i>Computer Vision</i>, kelas Glioma pada data latih sering kali memiliki fitur berupa perubahan tekstur piksel yang sangat samar. Akibatnya, model AI menjadi <i>oversensitive</i> dan menyalahartikan kumpulan piksel gelap atau terang alami pada anatomi otak normal tersebut sebagai pola fitur (<i>feature maps</i>) dari Glioma. Dalam konteks metrik evaluasi multikelas, kasus misklasifikasi ini merupakan contoh konkret dari <i>False Positive</i> (FP) untuk kelas Glioma dan <i>False Negative</i> (FN) untuk kelas <i>No Tumor</i>, sekaligus menjadi <i>True Negative</i> (TN) bagi kelas Meningioma dan Pituitary.
2	Asli: Glioma Prediksi: Pituitary 	Kesalahan klasifikasi ini dipicu oleh bias lokasi spasial dari distribusi data latih. Berdasarkan observasi visual terhadap dataset yang digunakan, kelas Pituitary secara mayoritas memiliki <i>Region of Interest</i> (ROI) yang terpusat di koordinat tengah bawah citra. Pada sampel ini, piksel Glioma kebetulan berada di koordinat yang tumpang tindih dengan sebaran lokasi langganan kelas Pituitary. Akibatnya, bobot fitur lokasi spasial mengalahkan fitur tekstur tumor itu sendiri saat proses ekstraksi fitur, sehingga model mengklasifikasikannya sebagai Pituitary. Dalam tinjauan metrik evaluasi multikelas, insiden misklasifikasi ini menyumbang nilai <i>False Positive</i> (FP) bagi kelas Pituitary dan <i>False Negative</i> (FN) bagi kelas Glioma, namun di saat yang sama tetap tercatat sebagai <i>True Negative</i> (TN) untuk kelas Meningioma dan <i>No Tumor</i>.
3	Asli: Glioma Pred: Meningioma 	Kualitas ketajaman pada foto ini kurang baik secara keseluruhan. Akibatnya, detail tekstur halus di dalam tumor yang menjadi ciri khas utamanya tidak bisa ditangkap oleh sistem penyaring model. Dalam tinjauan metrik evaluasi multikelas, insiden misklasifikasi ini menyumbang nilai <i>False Positive</i> (FP) bagi kelas Meningioma dan <i>False Negative</i> (FN) bagi kelas Glioma, namun di saat yang sama tetap tercatat sebagai <i>True Negative</i> (TN) untuk kelas Pituitary dan <i>No Tumor</i>.

No	Gambar	Analisis
4	Asli: Pituitary Pred: No Tumor 	Pada citra ini, area tumor tidak memiliki kontras kecerahan (<i>hyperintensity</i>) yang mencolok dibandingkan struktur di sekitarnya. Akibatnya, saat model melakukan ekstraksi fitur dan reduksi dimensi spasial, bobot fitur dari tumor yang samar tersebut terabaikan oleh bentuk otak yang secara keseluruhan tampak normal. Ketidadaan anomali piksel yang signifikan ini membuat model mengklasifikasikannya ke dalam kelas <i>No tumor</i>. Dalam tinjauan metrik evaluasi multikelas, insiden misklasifikasi ini menyumbang nilai <i>False Positive</i> (FP) bagi kelas <i>No Tumor</i> dan <i>False Negative</i> (FN) bagi kelas Pituitary, namun di saat yang sama tetap tercatat sebagai <i>True Negative</i> (TN) untuk kelas Meningioma dan Glioma.
5	Asli: Meningioma Pred: Glioma 	Kegagalan prediksi ini disebabkan oleh distribusi intensitas piksel yang tidak merata di dalam area tumor. Mayoritas kelas Meningioma pada data latih memiliki karakteristik tekstur fitur yang homogen. Namun pada citra ini, terdapat variasi piksel gelap dan terang pada bagian tengah tumor. Model menyalahartikan variasi tekstur internal ini sebagai pola fitur dari kelas Glioma yang memang sering bermanifestasi dengan tekstur piksel yang acak atau tidak beraturan. Dalam tinjauan metrik evaluasi multikelas, insiden misklasifikasi ini menyumbang nilai <i>False Positive</i> (FP) bagi kelas Glioma dan <i>False Negative</i> (FN) bagi kelas Meningioma, namun di saat yang sama tetap tercatat sebagai <i>True Negative</i> (TN) untuk kelas Pituitary dan <i>No Tumor</i>.
6	Asli: Glioma Pred: No Tumor 	Kegagalan prediksi pada citra ini secara dominan disebabkan oleh kualitas visual citra yang rendah dan tingkat kontras piksel yang buruk. Karena gambar tampak buram, batas piksel antara tumor Glioma dan jaringan otak normal tidak memiliki gradien yang tajam. Akibatnya, filter konvolusi pada arsitektur EfficientNetV2-S gagal mengekstrak fitur garis batas (<i>edges</i>) maupun fitur tekstur yang diskriminatif. Sinyal anomali yang sangat lemah ini kemudian semakin memudar saat melewati lapisan <i>pooling</i>, sehingga model mengklasifikasikannya sebagai kelas <i>baseline</i>, yaitu <i>No tumor</i>. Dalam tinjauan metrik evaluasi multikelas, insiden misklasifikasi ini

No	Gambar	Analisis
		menyumbang nilai <i>False Positive</i> (FP) bagi kelas <i>No Tumor</i> dan <i>False Negative</i> (FN) bagi kelas Glioma, namun di saat yang sama tetap tercatat sebagai <i>True Negative</i> (TN) untuk kelas Meningioma dan Pituitary.
7	Asli: Meningioma Pred: Pituitary 	Secara visual, Meningioma pada citra ini berada tepat di koordinat tengah bawah gambar. Berdasarkan distribusi data latih, koordinat tersebut adalah <i>Region of Interest</i> (ROI) utama yang secara persentase didominasi oleh kelas Pituitary. Akibat tumpang tindih letak spasial ini, arsitektur model memberikan bobot probabilitas yang jauh lebih tinggi pada fitur lokasi dibandingkan fitur tekstur piksel itu sendiri, sehingga memicu misklasifikasi menjadi Pituitary. Dalam tinjauan metrik evaluasi multikelas, insiden misklasifikasi ini menyumbang nilai <i>False Positive</i> (FP) bagi kelas Pituitary dan <i>False Negative</i> (FN) bagi kelas Meningioma, namun di saat yang sama tetap tercatat sebagai <i>True Negative</i> (TN) untuk kelas Glioma dan <i>No Tumor</i> .

Sebelum model diekspor ke dalam sistem produksi, dilakukan pemeriksaan kesehatan (*health check*) terhadap riwayat pelatihannya untuk memastikan stabilitas konvergensi. Evaluasi visual terhadap kurva pembelajaran (*learning curve*) ini merupakan metode standar yang krusial untuk mendeteksi secara dini ada tidaknya gejala *overfitting* maupun *underfitting*. Melalui grafik tersebut, pergerakan tren antara fase pelatihan dan validasi dapat diamati secara jelas untuk membuktikan bahwa model telah mencapai titik konvergensi optimal tanpa mengalami degradasi performa. Gambar 4.5 memvisualisasikan dinamika *Loss* dan *Akurasi* pada model final (*Fold 8*) selama proses pelatihan.



Gambar 4.5 Grafik *Loss* dan *Akurasi* pada *Fold 8*

Berdasarkan grafik tersebut, terlihat tren pembelajaran yang sangat positif. Pada fase awal, model beradaptasi dengan cepat yang ditandai oleh penurunan *loss* yang signifikan. Meskipun sempat terjadi lonjakan yang tinggi pada *epoch 7* di mana *loss* melonjak sesaat akibat variabilitas *batch*, data model menunjukkan ketangguhan yang luar biasa dengan melakukan koreksi mandiri pada *epoch* berikutnya. Pasca-pemulihan, kurva validasi bergerak stabil dan berimpit dengan kurva pelatihan pada kisaran akurasi 99,90%, mengonfirmasi bahwa model telah mencapai titik optimal dan aman dari risiko *overfitting*. Hal ini menjadi indikator kuat bahwa model final (*Fold 8*) yang merupakan bagian dari agregasi ini, sangat layak dan aman untuk diimplementasikan sebagai mesin diagnosis utama pada aplikasi *web*.

4.3.2 Implementasi Model pada Validasi Biner Citra Otak

Tahap implementasi model klasifikasi biner dirancang sebagai mekanisme validasi awal untuk menyaring citra masukan dari pengguna. Pelatihan model ini

memanfaatkan total 63.722 citra yang berimbang, terdiri atas 31.861 citra untuk kelas "MRI Otak" dan 31.861 citra untuk kelas "Bukan MRI" yang diambil dari dataset COCO. Keseluruhan himpunan data tersebut kemudian dipartisi menggunakan rasio pembagian 80:10:10. Skema pembagian ini menghasilkan alokasi data yang terstruktur untuk tahap pelatihan (*training*), validasi (*validation*), dan pengujian (*testing*). Rincian distribusi jumlah citra pada masing-masing himpunan data untuk kedua kelas disajikan pada Tabel 4.12.

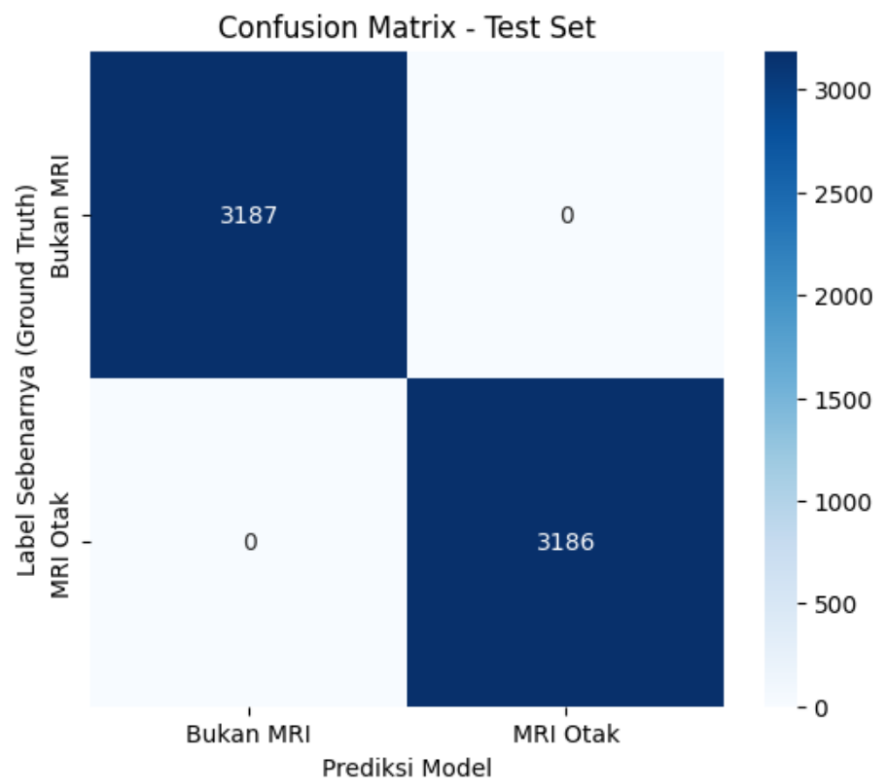
Tabel 4.12 Pembagian Data pada Validasi Biner Citra Otak

	Train	Validation	Test
Citra Otak	25.489	3.186	3.186
Bukan Citra Otak	25.488	3.186	3.187
Total	50.977	6.372	6.373

Proses pelatihan arsitektur EfficientNetV2-S pada tugas klasifikasi biner ini dikonfigurasi untuk berjalan hanya selama 5 *epoch*. Pemilihan jumlah *epoch* yang singkat ini didasarkan pada tingkat kompleksitas klasifikasi yang relatif rendah jika dibandingkan dengan klasifikasi jenis tumor. Perbedaan fitur visual antara citra medis anatomi otak dengan citra objek umum (*RGB*) memiliki kontras yang sangat tinggi. Karakteristik data yang sangat distingtif ini memungkinkan model untuk mempelajari fitur diskriminatif dan mencapai titik konvergensi secara cepat. Penggunaan *epoch* yang sedikit sudah sangat memadai untuk mencapai performa maksimal sekaligus mencegah terjadinya *overfitting* dan menghemat sumber daya komputasi.

Evaluasi kinerja akhir model dilakukan menggunakan himpunan data uji untuk mengukur keandalan sistem dalam mengklasifikasikan data yang belum

pernah dilihat sebelumnya. Hasil pemetaan antara prediksi model dengan label yang sebenarnya divisualisasikan melalui *Confusion Matrix* pada Gambar 4.6.



Gambar 4.6 *Confusion Matrix* Validasi Biner Citra Otak

Ekstraksi nilai dari *Confusion Matrix* tersebut menghasilkan empat metrik performa utama, yaitu akurasi, presisi, *recall*, dan *F1-Score*. Model menunjukkan kinerja yang sangat optimal dengan keberhasilan memprediksi seluruh kelas secara akurat tanpa adanya kesalahan klasifikasi, sehingga keempat metrik evaluasi tersebut mencapai nilai sempurna (100%). Tingkat presisi dan *recall* yang maksimal membuktikan bahwa model tidak memiliki bias terhadap salah satu kelas. Pencapaian ini mengonfirmasi bahwa model validasi biner yang dibangun sangat tangguh dan sepenuhnya siap diimplementasikan untuk menolak unggahan citra yang tidak relevan secara otomatis pada antarmuka sistem.

4.3.3 Implementasi Web

Implementasi antarmuka pengguna (*User Interface*) dikembangkan menggunakan bahasa pemrograman Python dengan bantuan kerangka kerja (*framework*) Streamlit. Streamlit dipilih karena efisiensinya dalam mengubah skrip Python menjadi aplikasi *web* interaktif yang responsif terhadap model *Deep Learning*. Struktur kode program dibagi menjadi tiga komponen utama: manajemen model, pra-pemrosesan data, dan logika antarmuka.

a. Inisialisasi dan Pemuatan Model

Komponen pertama adalah fungsi untuk memuat arsitektur model dan bobot (*weights*) yang telah dilatih. Pada tahap ini, arsitektur EfficientNetV2-S didefinisikan ulang dan lapisan *classifier* disesuaikan agar memiliki 4 *output neuron* (sesuai jumlah kelas). Setelah kerangka arsitektur tersebut terbentuk, bobot parameter terbaik yang diekspor dari tahap *k-Fold Cross Validation* sebelumnya disuntikkan secara presisi ke dalam jaringan melalui proses pemuatan bobot tersimpan.

Fungsi menggunakan dekorator dari Streamlit. Fitur ini berfungsi untuk menyimpan model di dalam *cache* memori setelah pemuatan pertama, sehingga aplikasi tidak perlu memuat ulang model setiap kali pengguna melakukan interaksi, yang secara signifikan mempercepat respons sistem. Manajemen alokasi memori yang proaktif ini sangat krusial untuk mencegah terjadinya kelebihan beban (*memory overload*) pada server, terutama saat sistem harus menangani inferensi dari unggahan citra secara jamak (*batch processing*).

script program pemuatan model:

```

# Potongan kode inisialisasi model
@st.cache_resource
def load_model(model_path='best_model.pth'):
    try:
        model = efficientnet_v2_s(weights=None)
        # Penyesuaian classifier agar sesuai dengan arsitektur
        saat training
        model.classifier = nn.Sequential(
            nn.Dropout(p=0.2, inplace=True),
            nn.Linear(model.classifier[1].in_features, 4)
        )
        # Memuat state dictionary ke CPU
        state_dict = torch.load(model_path,
map_location=torch.device('cpu'))
        model.load_state_dict(state_dict)
        model.eval()
        return model
    except Exception as e:
        st.error(f"Error memuat model: {e}")
        return None

@st.cache_resource
def load_tumor_model(model_path='best_model.pth'):
    """Model 2: Pengklasifikasi Jenis Tumor (4 Kelas)"""
    try:
        model = efficientnet_v2_s(weights=None)
        # Sesuai dengan arsitektur tumor classifier (4 node
        output)
        model.classifier = nn.Sequential(
            nn.Dropout(p=0.2, inplace=True),
            nn.Linear(model.classifier[1].in_features, 4)
        )
        state_dict = torch.load(model_path,
map_location=torch.device('cpu'), weights_only=True)
        model.load_state_dict(state_dict)
        model.eval()
        return model
    except Exception as e:
        st.error(f"Error memuat Model Tumor: {e}")
        return None

```

b. Pra-pemrosesan Citra Input (*Preprocessing*)

Agar model dapat melakukan prediksi dengan akurat, citra yang diunggah oleh pengguna harus memiliki format yang identik dengan data yang digunakan saat pelatihan. Fungsi bertugas mengubah ukuran citra menjadi piksel,

mengonversinya menjadi tensor, dan melakukan normalisasi menggunakan nilai *mean* dan *standard deviation* standar ImageNet.

script program preprocessing:

```
# Potongan kode pra-pemrosesan citra
def preprocess_image(image):
    transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229,
0.224, 0.225])
    ])
    return transform(image).unsqueeze(0) # Menambah dimensi
batch
```

c. Logika Inferensi dan Klasifikasi

Inti dari sistem ini terletak pada fungsi *predict*. Fungsi ini menerima input berupa tensor citra, meneruskannya ke model, dan menghitung probabilitas. Hasil akhir berupa label kelas dengan probabilitas tertinggi ditampilkan sebagai diagnosa final beserta tingkat keyakinannya (*confidence score*). Sesuai dengan pendekatan *multilayer* yang diusulkan, logika prediksi ini secara terstruktur dipisahkan ke dalam dua tahap fungsi independen, yakni validasi citra MRI otak dan klasifikasi jenis tumor. Guna mengoptimalkan alokasi memori dan kecepatan inferensi pada lingkungan produksi, seluruh kalkulasi maju (*forward pass*) dieksekusi tanpa melacak komputasi gradien (*gradient tracking*) karena model tidak lagi berada dalam fase pelatihan. Selain itu, nilai keluaran mentah (*logits*) dari jaringan dikonversi secara matematis menggunakan fungsi aktivasi *Softmax* untuk menghasilkan distribusi probabilitas absolut yang menjadi landasan utama penarikan kesimpulan.

script program logika prediksi:

```
# Potongan kode logika prediksi
def check_is_mri(model, image_tensor):
    """Mengembalikan True jika MRI (Label 1), False jika Bukan
    MRI (Label 0)"""
    with torch.no_grad():
        outputs = model(image_tensor)
        probabilities = torch.nn.functional.softmax(outputs,
dim=1)
        confidence, predicted_class = torch.max(probabilities,
1)
        is_mri = predicted_class.item() == 1 # Asumsi: 1 = MRI, 0 =
Bukan MRI
        return is_mri, confidence.item()

def predict_tumor(model, image_tensor):
    """Klasifikasi Jenis Tumor"""
    class_names = {0: 'Glioma', 1: 'Meningioma', 2: 'No tumor',
3: 'Pituitary'}
    with torch.no_grad():
        outputs = model(image_tensor)
        probabilities = torch.nn.functional.softmax(outputs,
dim=1)
        confidence, predicted_class = torch.max(probabilities,
1)
        return class_names[predicted_class.item()],
confidence.item()
```

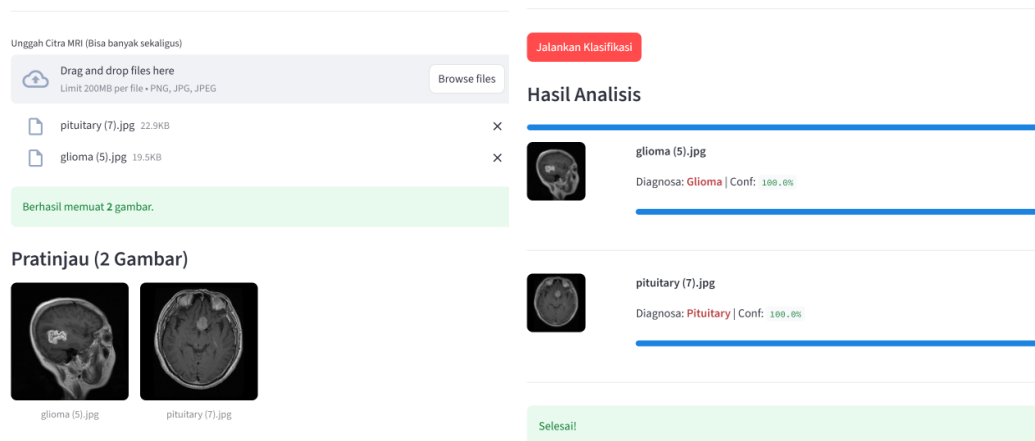
d. Tampilan Antarmuka Pengguna

Sisi antarmuka dirancang dengan pendekatan minimalis dan intuitif untuk mengakomodasi kemudahan penggunaan bagi pengguna medis maupun non-medis. Tata letak aplikasi terbagi menjadi tiga segmen utama yang saling terintegrasi, yaitu area unggah data, area pratinjau (*preview*), dan area hasil analisis. Pada bagian atas halaman, pengguna disuguhkan judul aplikasi beserta deskripsi singkat mengenai model yang digunakan. Interaksi utama dimulai melalui komponen *File Uploader* yang mendukung mekanisme *drag-and-drop* serta kemampuan pemrosesan banyak berkas sekaligus (*batch processing*), sebagaimana terlihat pada Gambar 4.7. Setelah

citra berhasil diunggah, sistem memberikan umpan balik visual berupa pesan konfirmasi dan menampilkan *thumbnail* dari seluruh citra dalam tata letak *grid*. Fitur pratinjau ini berfungsi sebagai langkah verifikasi krusial agar pengguna dapat memastikan kelengkapan dan kesesuaian data sebelum proses klasifikasi dilanjutkan.

Brain Tumor Classification

Aplikasi deteksi tumor otak menggunakan model EfficientNetV2-S.



Gambar 4.7 Klasifikasi Gambar Jamak

Mekanisme inferensi kemudian dieksekusi melalui tombol interaktif "*Jalankan Klasifikasi*", di mana sistem akan memproses citra satu per satu disertai indikator *progress bar*. Hasil diagnosis akhirnya disajikan dalam format kartu informasi terstruktur untuk setiap citra, seperti yang ditunjukkan pada Gambar 4.8. Setiap output hasil memuat elemen esensial meliputi citra asli sebagai referensi, nama berkas, label kelas tumor hasil prediksi (misalnya: *Meningioma*, *Glioma*, *Pituitary*, atau *No tumor*), serta tingkat keyakinan (*confidence score*).

Brain Tumor Classification

Aplikasi deteksi tumor otak menggunakan model EfficientNetV2-S.

Unggah Citra MRI (Bisa banyak sekaligus)

Drag and drop files here
Limit 200MB per file • PNG, JPG, JPEG

Browse files

M_6.jpg 23.7KB

Jalankan Klasifikasi

Berhasil memuat 1 gambar.

Pratinjau (1 Gambar)

M_6.jpg

Hasil Analisis

M_6.jpg

Diagnosa: Meningioma | Conf: 100.0%

Selesai!

Gambar 4.8 Klasifikasi Gambar Tunggal

4.4 Evaluasi *Black Box Testing*

Pengujian fungsionalitas sistem dilakukan dengan menerapkan metode *Black Box Testing* untuk memverifikasi bahwa antarmuka aplikasi *web* yang dibangun telah beroperasi sesuai dengan spesifikasi kebutuhan tanpa meninjau kode internal program. Fokus utama pengujian ini mencakup validasi mekanisme input data citra MRI, respons sistem terhadap format file, proses eksekusi model EfficientNetV2-S, hingga ketepatan tampilan hasil prediksi pada antarmuka pengguna. Sebanyak delapan skenario uji telah dirancang untuk merepresentasikan interaksi pengguna, yang mencakup kondisi input normal (*valid*) maupun kondisi penanganan kesalahan (*invalid*) guna memastikan ketahanan sistem. Rincian skenario pengujian tersebut beserta perbandingan antara hasil yang diharapkan dan hasil aktual yang diperoleh dari aplikasi disajikan selengkapnya dalam Tabel 4.13.

Tabel 4.13 Hasil *Black Box Testing*

No	Fitur	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengujian (Aktual)	Kesimpulan
1	Widget Pengunggahan (Format Valid)	Mengunggah file citra dengan format .jpg, .jpeg, atau .png	File berhasil dimuat dan tidak muncul pesan kesalahan	File citra berhasil dimuat masuk ke dalam sistem tanpa kendala	100% Berhasil
2	Widget Pengunggahan (Format Invalid)	Mengunggah file dokumen (.pdf, .docx) untuk menguji validasi	Sistem menolak file atau menampilkan peringatan kesalahan	Sistem menolak file dan menampilkan pesan "Invalid file format"	100% Berhasil
3	Mode Unggahan Tunggal	Mengunggah satu file citra MRI secara individual	Citra berhasil diunggah dan siap diproses	Satu citra berhasil terunggah dan tampil pada widget	100% Berhasil
4	Mode Unggahan Jamak	Mengunggah 3-5 file citra sekaligus untuk pemrosesan batch	Seluruh citra berhasil terunggah dan masuk antrian	Seluruh file yang dipilih berhasil dimuat secara bersamaan	100% Berhasil
5	Area Pratinjau	Memverifikasi tampilan visual citra setelah diunggah	Citra ditampilkan pada layar utama untuk verifikasi	Gambar citra MRI muncul pada antarmuka pengguna sesuai input	100% Berhasil
6	Tombol Eksekusi Klasifikasi	Menekan tombol "Proses Klasifikasi" untuk memulai inferensi	Sistem menampilkan indikator <i>loading</i> dan menjalankan model	Indikator proses berjalan dan sistem melakukan komputasi model	100% Berhasil
7	Panel Hasil Prediksi (Label)	Memeriksa keluaran teks kelas hasil klasifikasi	Muncul teks label kategori tumor (misal: Glioma)	Sistem menampilkan label kelas prediksi dengan tepat	100% Berhasil
8	Panel Hasil Prediksi (Skor)	Memeriksa tampilan	Muncul nilai persentase	Nilai persentase	100% Berhasil

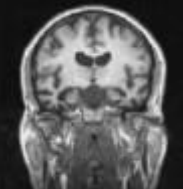
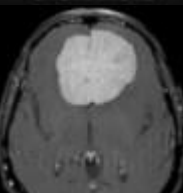
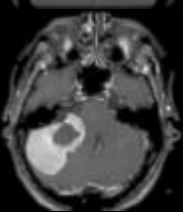
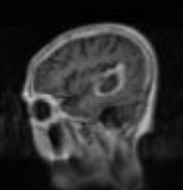
No	Fitur	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengujian (Aktual)	Kesimpulan
		tingkat keyakinan (<i>confidence score</i>)	probabilitas yang menyertai prediksi	akurasi muncul di samping label kelas	
9	Validasi Input	Memeriksa apakah gambar input merupakan citra MRI otak	File citra format .jpg, .jpeg, atau .png	Sistem menolak klasifikasi jika gambar yang diinput bukan citra MRI otak	100% Berhasil

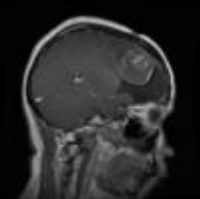
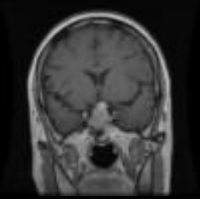
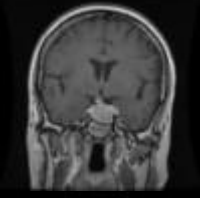
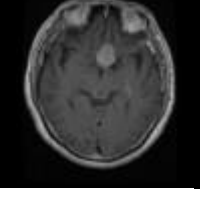
Berdasarkan hasil pengujian yang tercantum pada Tabel 4.13 di atas, dapat disimpulkan bahwa seluruh fitur fungsional utama pada aplikasi *web* klasifikasi tumor otak ini telah beroperasi sepenuhnya sesuai dengan skenario yang dirancang. Sistem terbukti mampu menangani berbagai interaksi pengguna, mulai dari memvalidasi format unggahan file citra hingga menolak input yang tidak sesuai dengan memberikan peringatan yang tepat. Selain itu, fitur inti berupa eksekusi prediksi model EfficientNetV2-S berhasil menampilkan label klasifikasi tumor beserta nilai probabilitas (*confidence score*) secara akurat pada layar antarmuka tanpa mengalami kegagalan fungsi. Dengan terpenuhinya seluruh kriteria hasil yang diharapkan pada setiap poin skenario uji, aplikasi ini dinyatakan lolos uji fungsionalitas dan siap untuk digunakan dalam implementasi nyata.

Untuk membuktikan keandalan sistem dalam menjalankan skenario pengujian nomor 9 pada Tabel 4.13, dilakukan pengujian fungsionalitas lanjutan secara spesifik terhadap fitur validasi biner. Pengujian ini bertujuan untuk melihat secara langsung respons penolakan maupun penerimaan aplikasi ketika dihadapkan pada berbagai jenis gambar acak yang mungkin diunggah oleh pengguna. Pengujian

dilakukan dengan mengunggah 20 sampel gambar yang terdiri dari beberapa citra MRI otak yang didapat dari salah satu dataset yg digunakan dalam pengembangan model dan citra bukan MRI otak (objek umum seperti hewan, barang, dan lain-lain) yang didapat dari mesin pencari. Hasil dari pengujian input gambar tersebut disajikan selengkapnya pada Tabel 4.14.

Tabel 4.14 Uji Validasi Input

No.	Gambar	Kategori	Output
1		Citra MRI otak.	Diterima: Tervalidasi sebagai MRI Otak.
2		Citra MRI otak.	Diterima: Tervalidasi sebagai MRI Otak.
3		Citra MRI otak.	Diterima: Tervalidasi sebagai MRI Otak.
4		Citra MRI otak.	Diterima: Tervalidasi sebagai MRI Otak.
5		Citra MRI otak.	Diterima: Tervalidasi sebagai MRI Otak.
6		Citra MRI otak/	Diterima: Tervalidasi sebagai MRI Otak.

No.	Gambar	Kategori	Output
7		Citra MRI otak.	Diterima: Tervalidasi sebagai MRI Otak.
8		Citra MRI otak.	Diterima: Tervalidasi sebagai MRI Otak.
9		Citra MRI otak.	Diterima: Tervalidasi sebagai MRI Otak.
10		Citra MRI otak.	Diterima: Tervalidasi sebagai MRI Otak.
11		Bukan citra MRI otak.	Ditolak: Citra terdeteksi BUKAN MRI Otak.
12		Bukan citra MRI otak.	Ditolak: Citra terdeteksi BUKAN MRI Otak.
13		Bukan citra MRI otak.	Ditolak: Citra terdeteksi BUKAN MRI Otak.
14		Bukan citra MRI otak.	Ditolak: Citra terdeteksi BUKAN MRI Otak.

No.	Gambar	Kategori	Output
15		Bukan citra MRI otak.	Ditolak: Citra terdeteksi BUKAN MRI Otak.
16		Bukan citra MRI otak.	Ditolak: Citra terdeteksi BUKAN MRI Otak.
17		Bukan citra MRI otak.	Ditolak: Citra terdeteksi BUKAN MRI Otak.
18		Bukan citra MRI otak.	Ditolak: Citra terdeteksi BUKAN MRI Otak.
19		Bukan citra MRI otak.	Ditolak: Citra terdeteksi BUKAN MRI Otak.
20		Bukan citra MRI otak.	Ditolak: Citra terdeteksi BUKAN MRI Otak.

Berdasarkan hasil pengujian yang disajikan pada Tabel 4.14, dapat disimpulkan bahwa mekanisme validasi input berbasis klasifikasi biner telah berfungsi secara optimal dan sesuai dengan skenario yang diharapkan. Sistem terbukti mampu mengenali dan menerima 10 sampel citra MRI otak secara akurat untuk diproses lebih lanjut, sekaligus berhasil memblokir 10 sampel citra bukan MRI (objek umum) dengan menampilkan pesan penolakan yang tepat. Keberhasilan sistem dalam merespons keseluruhan sampel uji secara sempurna ini menegaskan bahwa fitur *gatekeeper* sangat andal dalam mencegah masuknya data yang tidak valid

akibat kelalaian pengguna, sehingga integritas pada proses klasifikasi jenis tumor utama dapat terus terjaga.

BAB V

KESIMPULAN DAN SARAN

Bab ini merupakan bagian penutup yang merangkum hasil akhir dari seluruh rangkaian penelitian yang telah dilaksanakan. Pada bab ini, penulis menguraikan kesimpulan yang ditarik berdasarkan analisis, implementasi, dan pengujian model *Deep Learning* menggunakan arsitektur *EfficientNetV2-S* terhadap citra MRI tumor otak yang telah dipaparkan pada Bab IV. Kesimpulan ini disusun secara sistematis untuk menjawab rumusan masalah serta tujuan penelitian yang telah ditetapkan pada Bab I. Selain itu, berdasarkan temuan dan keterbatasan yang ada, penulis juga menyampaikan saran-saran konstruktif yang diharapkan dapat menjadi rekomendasi bermanfaat bagi pengembangan penelitian selanjutnya maupun implementasi praktis di masa mendatang.

5.1 Kesimpulan

Berdasarkan hasil penelitian, implementasi, dan pengujian yang telah dilakukan mengenai klasifikasi jenis tumor otak menggunakan model *Deep Learning* *EfficientNetV2-S* pada citra MRI, dapat ditarik kesimpulan sebagai berikut:

1. Implementasi model *transfer learning* *EfficientNetV2-S* untuk klasifikasi tumor otak berhasil dilakukan dengan menyesuaikan lapisan klasifikasi akhir (*fine-tuning*) pada empat kelas target. Konfigurasi *hyperparameter* menggunakan *optimizer* AdamW, fungsi kerugian *CrossEntropy*, dan

mekanisme *Early Stopping* terbukti efisien. Model mampu mencapai konvergensi optimal secara stabil dalam batas pelatihan maksimal 20 *epoch* tanpa mengalami *overfitting* yang signifikan.

2. Mekanisme validasi input berhasil dibangun dengan mengimplementasikan model klasifikasi biner tambahan berbasis EfficientNetV2-S sebagai *gatekeeper*. Sistem ini terbukti mampu membedakan citra anatomi medis dengan citra objek umum secara akurat. Sistem juga secara efektif menolak unggahan citra yang tidak relevan (bukan MRI otak), sehingga mencegah terjadinya kesalahan pemrosesan akibat kelalaian pengguna.
3. Model EfficientNetV2-S terbukti sangat efektif dalam mengklasifikasikan empat jenis kelas (glioma, meningioma, pituitary, dan *no tumor*). Berdasarkan pengujian *10-Fold Cross Validation*, model mencapai akurasi tertinggi sebesar 99,86% pada *fold* terbaik. Metrik evaluasi rata-rata tertimbang (*tertimbang average*) untuk *Presisi*, *Recall*, dan *F1-Score* juga menunjukkan performa yang sangat konsisten di angka 99,78%, dengan *F1-Score* untuk masing-masing kelas berada di atas 99%.
4. Performa model EfficientNetV2-S terbukti sangat tangguh dan kompetitif apabila dibandingkan dengan arsitektur *Deep Learning* lain dari berbagai penelitian terdahulu yang menggunakan dataset identik. Model ini tidak hanya mampu menyamai, tetapi juga memberikan tingkat akurasi klasifikasi yang lebih unggul dibandingkan beberapa model lain.
5. Strategi penggabungan dataset dari berbagai sumber (Figshare, Br35H, Sartaj, dan Brain Tumors Dataset) secara umum tidak memberikan perubahan

performa yang signifikan terhadap model EfficientNetV2-S. Hasil evaluasi membuktikan bahwa model konsisten memberikan performa yang sama baiknya pada pengujian menggunakan dataset *baseline* maupun dataset campuran. Kondisi ini menunjukkan bahwa ketangguhan, kemampuan generalisasi, dan akurasi dalam klasifikasi pola tumor lebih dipengaruhi oleh keunggulan arsitektur EfficientNetV2-S itu sendiri dibandingkan dengan penambahan volume dan variasi data dari luar.

5.2 Saran

Berdasarkan keterbatasan dan hasil yang diperoleh dalam penelitian ini, penulis mengajukan beberapa saran untuk pengembangan penelitian selanjutnya:

1. Implementasi *Explainable AI* (XAI): Untuk meningkatkan kepercayaan tenaga medis terhadap prediksi sistem, disarankan untuk menambahkan modul XAI seperti *Grad-CAM*. Fitur ini dapat memvisualisasikan *heatmap* pada citra MRI yang menjadi dasar keputusan model, sehingga dokter dapat memverifikasi lokasi tumor yang dideteksi.
2. Uji Coba Data Klinis Nyata: Meskipun model mencapai akurasi tinggi pada dataset publik, disarankan untuk melakukan validasi eksternal menggunakan data citra MRI asli dari rumah sakit mitra. Hal ini penting untuk menguji ketahanan model terhadap variasi alat pemindai dan protokol pengambilan gambar di lapangan.
3. Pengembangan Sistem Aplikasi: Prototipe aplikasi (misalnya berbasis Streamlit) dapat dikembangkan lebih lanjut menjadi sistem berbasis *cloud*

atau aplikasi *mobile* yang terintegrasi dengan sistem rekam medis elektronik rumah sakit untuk memudahkan penggunaan oleh radiolog.

4. Komparasi Model Terbaru: Penelitian selanjutnya dapat membandingkan performa EfficientNetV2 dengan arsitektur lainnya, seperti Vision Transformer (ViT) atau ConvNeXt, untuk mengevaluasi apakah ada peningkatan efisiensi komputasi atau akurasi yang lebih baik lagi.

DAFTAR PUSTAKA

- Adugna, T. D., Ramu, A., & Haldorai, A. (2024). A Review of Pattern Recognition and Machine Learning. In *Journal of Machine and Computing* (Vol. 4, Number 1). <https://doi.org/10.53759/7669/jmc202404020>
- Azzahra, T. S., Jessica Jesslyn Cerelia, Farid Azhar Lutfi Nugraha, & Anindya Apriliyanti Pravitasari. (2023). MRI-Based Brain Tumor Classification Using Inception Resnet V2. *Enthusiastic: International Journal of Applied Statistics and Data Science*, 3(2), 163–175. <https://doi.org/10.20885/enthusiastic.vol3.iss2.art4>
- Badža, M. M., & Barjaktarović, M. C. (2020). Classification of brain tumors from mri images using a convolutional neural network. *Applied Sciences (Switzerland)*, 10(6). <https://doi.org/10.3390/app10061999>
- Bhuvaji, S., Khandelwal, A., Chakole, R., & Jain, P. (2020). Brain tumor Classification (MRI) [Dataset]. Kaggle. <https://www.kaggle.com/datasets/sartajbhuvaji/brain-tumor-classification-mri>
- Cheng, J. (2017). brain tumor dataset [Dataset]. Figshare. <https://doi.org/10.6084/m9.figshare.1512427.v8>
- Friedl, H., Stampfer, E., El-shaarawi, A. H., & Piegorsch, W. W. (2002). Cross-validation Cross-validation. 1, 452–460. <https://doi.org/10.1002/9780470057339.vac062>
- Gencer, K. (2025). A Comparative Analysis of EfficientNetB0 and EfficientNetV2 Variants for Brain Tumor Classification Using MRI Images. *International Journal of Innovative Engineering Applications*, 9(1), 1–7. <https://doi.org/10.46460/ijiea.1523782>
- Ghasemi, M. H. (2023). Uncovering knowledge: A clean brain tumor dataset for advanced medical research [Dataset]. Kaggle. <https://www.kaggle.com/datasets/mohammadhossein77/brain-tumors-dataset>
- Hamada, A. (2020). Brain tumor detection [Dataset]. Kaggle. <https://www.kaggle.com/datasets/ahmedhamada0/brain-tumor-detection>
- Hamidja, K. B., Koffi, K., Pacôme, B., Asseu, O., & Oumtanaga, S. (2024). An Adapted Convolutional Neural Network for Brain Tumor Detection. *Open Journal of Applied Sciences*, 14(10), 2809–2825. <https://doi.org/10.4236/ojapps.2024.1410184>

- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hesaraki, S. (2017). 2017-2017 [Dataset]. Kaggle. <https://www.kaggle.com/datasets/sabahasarak/2017-2017>
- Iqbal, A., Jaffar, M. A., & Jahangir, R. (2024). Enhancing Brain Tumour Multi-Classification Using Efficient-Net B0-Based Intelligent Diagnosis for Internet of Medical Things (IoMT) Applications. *Information (Switzerland)*, 15(8). <https://doi.org/10.3390/info15080489>
- Karno A S B, Hastomo W, Arif D, Wardhana I S K, Kamilia N, Yulianto R, Digdoyo A, & Surawan T. (2023). Brain Tumor Classification Using Four Versions of EfficientNet. *Insearch (Information System Research Journal)*, 3(1), 16–23. <https://ejournal.uinib.ac.id/jurnal/index.php/insearch/article/view/5810>
- Khan, M. E. (2011). Different approaches to white box testing technique for finding errors. *International Journal of Software Engineering and Its Applications*, 5(3), 1–14. <https://doi.org/10.5121/ijsea.2011.2404>
- Kingma, D. P., & Ba, J. L. (2015). Adam: A method for stochastic optimization. *3rd International Conference on Learning Representations, ICLR 2015 - Conference Track Proceedings*, 1–15. <https://doi.org/10.48550/arXiv.1412.6980>
- Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *7th International Conference on Learning Representations, ICLR 2019*. <https://doi.org/10.48550/arXiv.1711.05101>
- Nidhra, S. (2012). Black Box and White Box Testing Techniques - A Literature Review. *International Journal of Embedded Systems and Applications*, 2(2), 29–50. <https://doi.org/10.5121/ijesa.2012.2204>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32(NeurIPS). <https://doi.org/10.48550/arXiv.1912.01703>
- PyTorch Developers. (2024). Models and pre-trained weights. PyTorch Documentation. <https://pytorch.org/vision/stable/models.html>

- Tan, M., & Le, Q. V. (2019). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. <https://doi.org/10.48550/arXiv.1905.11946>
- Tan, M., & Le, Q. V. (2021). *EfficientNetV2: Smaller Models and Faster Training*. <https://doi.org/10.48550/arXiv.2104.00298>
- Waskita, A. A., Amda, J. M., Sihono, D. S. K., & Prasetyo, H. (2023). EfficientNetV2 based for MRI brain tumor image classification. *Proceedings - 2023 10th International Conference on Computer, Control, Informatics and Its Applications: Exploring the Power of Data: Leveraging Information to Drive Digital Innovation, IC3INA 2023*, 171–176. <https://doi.org/10.1109/IC3INA60834.2023.10285782>
- Wong, Y., Su, E. L. M., Yeong, C. F., Holderbaum, W., & Yang, C. (2025). Brain tumor classification using MRI images and deep learning techniques. *PLoS ONE*, 20(5 May), 1–19. <https://doi.org/10.1371/journal.pone.0322624>

LAMPIRAN

Lampiran 1 Link Github Berisi *Source Code*

<https://github.com/Roberttwil/skripsi.git>

RIWAYAT HIDUP